

결과리포트

LAB1_FPGA 실험

제출일: 2026년 9월 27일

이준화 교수님

전자전기컴퓨터공학부

팀 B

학번: 2025440031

이름: 김영서

목차

<예비보고서>

1. 실험 개요와 목적
2. 실험의 이론적 배경
3. 실험에 필요한 장치
4. 모드별 입출력 표
5. 각 verillog 파일의 역할
6. 사전 시뮬레이션 결과

<결과보고서>

7. 포트 및 핀 번호 입력

8. Vivado 내의 실험 과정 및 결과

9. 실험 결과와 예상 결과 간의 비교

10. 증빙 및 소스 commit 링크(git hub)

1. 실험 개요와 목적

1.1. 카운터, 클록 분주기, 레지스터, 시프트 레지스터, PISO 등의 회로를 설계하여 클록에 따른 데이터 저장과 상태 변화의 원리를 이해한다. 특히 순차회로에서는 현재 입력뿐만 아니라 이전 상태가 출력에 영향을 준다는 점과, 상승 에지를 기준으로 저장값이 갱신되는 과정을 파형을 통해 확인한다.

1.2. Moore FSM과 Mealy FSM을 포함한 여러 순차논리회로를 Verilog HDL로 구현하고, 테스트벤치를 이용해 입력과 클록 변화에 따른 상태 및 출력 변화를 검증한다. 정상 동작뿐 아니라 일부 RTL을 의도적으로 수정하여 자기검사 실패를 확인하고, 오류 시점의 입력과 현재 상태를 분석한 후 회로를 복구함으로써 순차논리 설계의 검증 방법을 익힌다.

1.3. VS Code에서 수행한 사전 시뮬레이션 결과를 바탕으로 Vivado에서 회로를 합성하고 FPGA에 구현한 뒤, 실제 보드의 스위치·버튼·LED·7-segment 등을 이용하여 설계한 순차논리회로의 동작을 확인한다. 이를 통해 RTL 시뮬레이션 결과와 실제 FPGA 하드웨어의 동작을 비교하고, 클록 입력과 상태 변화가 실제 장치에서 어떻게 나타나는지 확인하는 것을 목적으로 한다.

2. 실험의 이론적 배경

2.1. 순차논리회로와 클록

- ▶ 순차논리회로는 현재 입력뿐만 아니라 이전에 저장된 상태에 따라 출력이 결정되는 회로이다.
- ▶ 즉, 입력이 변한다고 즉시 저장값이 변경되는 것이 아니라, 정해진 클록 에지를 기준으로 상태가 갱신된다.
- ▶ LAB2의 테스트벤치에서는 클록 주기를 10 ns로 설정하며 상승 에지는 5, 15, 25, ... ns에서 발생한다.
- ▶ 입력이 변경되더라도 레지스터의 값은 다음 상승 에지에서 갱신되며, 테스트벤치는 nonblocking assignment에 의한 갱신이 완료된 뒤 값을 검사하기 위해 상승 에지 이후 1 ns를 기다린 후 결과를 비교한다.
- ▶ 또한 시뮬레이션에서 사용하는 10 ns 클록과 실제 FPGA 보드에서 사용하는 클록 조건은 서로 구분하여 해석해야 한다.

2.2. 동기식 Reset과 Enable

- ▶ 순차논리회로에서는 rst와 enable 등의 제어 신호를 이용해 상태 갱신을 제어한다.
- ▶ 본 실험에서 사용하는 동기식 reset은 rst=1인 순간 즉시 상태가 변하는 것이 아니라 다음 상승 에지에서 초기값으로 변경된다.
- ▶ 반면 rst=0, enable=1일 때는 회로가 정의한 다음 상태로 갱신되고, enable=0이면 기존 상태를 그대로 유지한다.
- ▶ 따라서 파형을 분석할 때는 reset 신호가 변하는 순간과 실제 상태가 바뀌는 상승 에지를 구분해야 하며, 각 회로에서 reset과 enable의 우선순위도 확인해야 한다.

2.3. Nonblocking Assignment와 저장값 갱신

- ▶ Verilog에서 순차논리의 저장값은 주로 nonblocking assignment인 `<=`를 이용하여 갱신한다.
- ▶ 여러 레지스터가 동일한 상승 에지에서 갱신되더라도 오른쪽 항은 에지 직전의 값을 기준으로 계산된다.
- ▶ 예를 들어 같은 상승 에지에서 A에 새로운 입력을 저장하고 B에 A를 전달하면, B에는 새로 갱신된 A가 아니라 상승 에지 직전의 A가 저장된다.
- ▶ 교안에서는 순차회로의 저장값에는 `<=`를 사용하고, 별도의 조합논리 `always @*` 블록에서는 `=`을 사용하도록 구분하고 있다.
- ▶ 이러한 특성은 Register, Shift Register, PISO, FSM 등의 파형을 해석할 때 중요하다.

2.4. 외부 버튼의 동기화, Debounce와 One-pulse

- ▶ FPGA 보드의 기계식 버튼은 한 번 눌러도 접점이 짧은 시간 동안 여러 번 변할 수 있는 바운스(bounce) 현상이 발생하며, FPGA 내부 클록과도 동기화되어 있지 않다.
- ▶ 따라서 교안에서는 외부 입력을 바로 상태 변경 신호로 사용하는 대신 동기화 => 안정 시간 확인 => 한 클록 펄스 생성의 순서로 처리한다.
- ▶ 두 단계의 동기화 플립플롭은 비동기 입력으로 인한 메타안정성이 내부 회로로 전달될 가능성을 낮추고, debounce 과정에서는 일정 시간 동안 입력이 안정적으로 유지되는지 확인하여 버튼의 반복적인 접점 변화를 제거한다.
- ▶ 마지막으로 실제 회로에는 한 번의 버튼 입력을 한 클록 폭의 펄스로 전달하여 한 번 눌렀을 때 상태가 여러 번 변하는 것을 방지한다.

2.5. Clock Divider와 Clock Enable

- ▶ 클록 분주기는 빠른 입력 클록을 이용하여 더 느린 동작 주기를 만드는 회로이다.
- ▶ 본 실험에서는 느린 동작을 LED나 파형으로 관찰하기 위해 분주 출력을 사용하며, 다른 순차회로에는 별도의 새로운 클록을 직접 공급하기보다 공통 clk를 유지하고 tick이 1인 상승 에지에서만 상태를 갱신하는 clock-enable 방식을 사용한다.
- ▶ 예를 들어 DIVISOR=10이면 tick은 10개의 클록마다 한 번 사용되며, 실제 클록 주파수와 분주 수의 비를 이용하여 최종 동작 속도를 계산할 수 있다.
- ▶ 클록 분주 실험에서는 단순히 주기만 확인하는 것이 아니라 첫 번째 출력 전환 시점과 duty 특성도 함께 확인한다.

2.6. Counter, Register, Shift Register와 PISO

- ▶ Up/Down Counter는 클록 에지마다 저장된 값을 증가 또는 감소시키는 순차회로이다.
- ▶ LAB2에서는 증가 시 $0 \Rightarrow \dots \Rightarrow 15 \Rightarrow 0$ 으로 순환하고, 0에서 감소하면 15가 되며, enable=0에서는 값을 유지한다.
- ▶ 또한 reset과 enable이 동시에 활성화된 경우 reset이 우선하도록 설계한다.
- ▶ Register는 입력 데이터를 클록 에지에 맞추어 저장하는 회로이다.
- ▶ 본 실험의 Register 회로에서는 load가 활성화되면 입력값을 stored에 저장하고, transfer가 활성화되면 저장된 값을 다른 레지스터로 전달한다.
- ▶ load와 transfer가 동시에 활성화될 경우 nonblocking assignment의 특성에 의해 전달받는 값은 같은 에지에서 새로 저장되는 값이 아니라 이전 stored 값이다.
- ▶ Shift Register는 클록 에지마다 저장된 여러 비트의 위치를 한 칸씩 이동시키는 회로이다.

- ▶ 본 실험에서는 새로운 직렬 입력이 bit 3으로 들어오고 기존 데이터는 bit 0 방향으로 이동한다.
- ▶ 따라서 연속된 입력에 따라 내부 저장값이 단계적으로 변화하는 모습을 파형에서 확인할 수 있다.
- ▶ PISO(Parallel-In Serial-Out)는 여러 비트의 데이터를 한 번에 병렬로 저장한 뒤 한 비트씩 직렬로 출력하는 회로이다.
- ▶ 예를 들어 1010을 병렬 로드하면 직렬 출력은 최상위 비트부터 $1 \rightarrow 0 \rightarrow 1 \rightarrow 0$ 순서로 나타난다.
- ▶ 또한 load와 enable이 동시에 활성화된 경우에는 load 동작이 우선한다.

2.7. FSM과 Moore/Mealy 상태 머신

- ▶ FSM(Finite State Machine)은 현재 상태와 입력에 따라 다음 상태를 결정하며 정해진 상태 사이를 전이하는 순차논리회로이다.
- ▶ Moore FSM에서는 출력이 현재 상태에 의해 결정된다. LAB2의 Moore 회로는 $00 \Rightarrow 01 \Rightarrow 10 \Rightarrow 00$ 의 순서로 상태가 전이하며, enable과 advance가 모두 활성화된 유효 에지에서만 다음 상태로 이동한다.
- ▶ 출력은 상태 자체에 의해 결정되므로 클록 사이에서 advance 입력만 변경하더라도 출력은 즉시 바뀌지 않는다.
- ▶ 반면 Mealy FSM에서는 출력이 현재 상태와 현재 입력에 모두 의존한다.
- ▶ 따라서 상태가 바뀌지 않더라도 입력이 바뀌면 조합 출력은 변경될 수 있다.
- ▶ 본 실험에서는 입력이 1인 유효 에지에서 $S0 \leftrightarrow S1$ 로 상태가 전이하고 입력이 0이면 상태를 유지한다.
- ▶ 또한 enable=0은 상태 갱신만 정지시키므로 입력 변화에 따른 조합 출력의 변화는 계속 발생할 수 있다.

2.8. 7-Segment Dynamic Scanning

- ▶ 여러 자리의 7-segment를 제한된 신호선으로 구동하기 위해 자리 스캔 방식을 사용한다.
- ▶ LAB2에서는 8자리 중 한 자리만 순간적으로 활성화하고 이를 빠르게 순환하여 사람의 눈에는 여러 자리가 동시에 켜져 있는 것처럼 보이게 한다.
- ▶ 자리와 자리 사이에는 모든 자리 선택 신호를 비활성화하는 blank 구간을 두어 다음 자리의 segment 데이터가 안정될 시간을 확보한다.
- ▶ 또한 기능 코어에서는 active-high 형태의 논리값을 생성하고, 실제 보드에서 필요한 자리 선택 및 segment의 극성 변환은 핀 연결 부분에서 처리한다.

3. 실험에 필요한 장치

3.1. Visual Studio Code

- 본 실험에서 Verilog HDL 파일과 SystemVerilog Testbench를 작성하는 주된 편집 환경이다.
- src 폴더의 각 조합논리회로, 통합 모듈, 버튼 및 LCD 제어 모듈을 작성하고, sim 폴더의 Testbench와 FPGA 핀 설정을 위한 XDC 파일도 관리한다.
- 즉, 회로를 HDL이라는 소프트웨어 형태로 기술하는 첫 단계에 사용된다.

3.2. Icarus Verilog

- HDL로 작성한 회로를 실제 FPGA에 구현하기 전에 기능이 올바른지 확인하는 시뮬레이터이다.
- 이번 실험에서는 10개 mode와 각각의 256개 DIP 입력 조합을 검사하여 총 2560개의 경우를 자동으로 비교한다.
- 또한 reset, button bounce, long press, MODE 10에서 MODE 01로의 순환 및 LCD 동작도 Testbench에서 함께 검증한다.

3.3. VaporView

- Icarus Verilog 시뮬레이션에서 생성된 VCD 파일을 파형으로 표시하는 도구이다.
- 본 실험에서는 여러 신호를 관찰하여 버튼 입력 이후 모드가 바뀌는 과정과 LCD 데이터가 전송되는 과정을 시간축에서 확인할 예정이다.

3.4. AMD Vivado 2026.1

- Vivado는 Verilog로 기술한 논리회로를 실제 AMD/Xilinx FPGA에서 사용할 수 있는 회로로 변환하는 FPGA 설계 프로그램이다.
- 사전 시뮬레이션 이후 회로를 합성하고, FPGA 내부 자원에 회로를 배치하고 연결하는 과정을 수행하며, 마지막으로 FPGA에 기록할 비트 스트림을 생성한다.

****FPGA와 이러한 소프트웨어 도구의 관계****

- FPGA는 ASIC과 달리 반도체가 제조된 이후에도 논리 기능을 다시 설정할 수 있는 소자이다.
- ASIC은 특정 기능에 맞는 회로를 실제 반도체 제조 공정을 통해 고정하여 제작하므로, 설계에 문제가 발생하면 경우에 따라 설계를 수정한 뒤 다시 제작해야 한다.
- 반면, FPGA는 이미 제조된 칩의 programmable logic과 interconnect를 Bitstream으로 설정하기 때문에 새로운 반도체 제조 공정을 반복하지 않고도 회로를 변경할 수 있다.
- 이러한 위험성 측면, 시간적 측면에 따라 반도체 제조 공정에서는 FPGA를 사용한다.

4. 모드별 입출력 표

회로	주요 입력	실행 조건	주요 출력	예상 동작
Up/Down Counter	SW8 = sw[0]	N4 1회	LED[3:0] = count	SW8=0이면 +1, SW8=1이면 -1
Clock Divider	별도 스위치 입력 없음	자동 동작	LED0 = / 2 , LED1 = / 10 , LED2 = / 50 , LED3 = / 1000 , LED4=tick	1 kHz 입력 클록을 여러 비율로 분주
Register Pair	SW1~4 = sw[7:4] data, SW7=sw[1] transfer, SW8=sw[0] load	N4 1회	LED[7:4]=stored, LED[3:0]=registered(value)	입력 저장 및 저장값 전달
Shift Register	SW1 = sw[7] serial input	N4 1회	LED[3:0]=shifted value	직렬 입력을 넣으며 4비트 저장값 이동
PISO	SW1~4=sw[7:4] 병렬 데이터, SW8=sw[0] load/shift 선택	N4 1회	LED[7:4]=parallel 저장값, LED0=serial_out	SW8=1이면 load, SW8=0이면 shift
Moore FSM	SW1=sw[7] advance	N4 1회	LED[1:0]=state/output	advance = 1 에서 00=>01=>10=>00
Mealy FSM	SW1=sw[7] input	N4로 상태 전이	LED2 = state , LED[1:0]=output	출력이 현재 상태와 입력에 모두 의존
8 - Digit Segment Scan	SW1~4=sw[7:4] 첫 자리 데이터	자동 동작, N4 불필요	7 - segment seg_com , seg_data , LED[2:0]=scan_index	8개 자리를 순차적으로 자동 스캔

5. 각 verilog 파일의 역할

파일명	역할
counter4.v	4비트 Up/Down Counter를 구현한다. enable이 활성화된 클록 에지에서 설정된 방향에 따라 값을 1씩 증가 또는 감소시키고, reset 시 0으로 초기화한다.
clock_divider.v	입력 클록을 설정된 분주비에 따라 낮은 주파수의 출력 및 tick 신호로 변환한다. 통합 회로에서는 여러 속도의 분주 신호를 관찰하는 데 사용한다.
register_pair.v	두 개의 4비트 레지스터를 이용하여 입력 데이터를 저장하고, 저장된 값을 다른 레지스터로 전달하는 동작을 수행한다.
shift_register4.v	4비트 시프트 레지스터를 구현한다. 한 번의 유효 클록마다 새로운 직렬 입력을 저장하면서 기존 비트들을 한 방향으로 이동시킨다.
piso4.v	4비트 PISO(Parallel-In Serial-Out) 레지스터를 구현한다. 병렬 입력을 한 번에 저장한 뒤 저장된 데이터를 한 비트씩 직렬로 출력한다.
moore_cycle.v	Moore 방식의 FSM을 구현한다. 입력 조건과 유효 클록에 따라 상태가 순차적으로 전이하며, 출력은 현재 상태에 의해 결정된다.
mealy_toggle.v	Mealy 방식의 FSM을 구현한다. 상태 전이를 수행하며 출력은 현재 상태와 현재 입력에 의해 결정된다.
segment_scan8.v	8자리 7-segment를 동적 스캔 방식으로 구동한다. 한 번에 한 자리씩 선택하고 자리 사이에 blank 구간을 두면서 8자리를 반복적으로 표시한다.
input_frontend.v	외부 버튼 및 스위치 입력을 FPGA 내부에서 안정적으로 사용할 수 있도록 입력 동기화, 안정 상태 확인 등의 전처리를 수행한다.
button_onepulse.v	버튼 입력을 처리하여 한 번의 버튼 조작마다 한 클록 폭의 펄스를 생성한다. 버튼을 길게 눌러도 한 번의 동작만 발생하도록 한다.
lcd_lab2_modes.v	현재 LAB2 모드 번호와 각 회로의 이름을 LCD에 표시하기 위한 LCD 제어 신호와 데이터를 생성한다.
lab2_integrated.v	LAB2 통합 회로의 최상위 모듈이다. 각 기능 코어를 인스턴스화하고 mode에 따라 해당 회로를 활성화하며, 버튼·스위치 입력과 LED·LCD·7-segment 출력을 연결하고 선택한다.

6. 사전 시뮬레이션 결과

vivado버전: vivado 2026.1

FPFA Part: xc7s75fgga484-1

6.1. Counter

작성한 코드는 다음과 같다.

```
src > ≡ counter4.v
1  `timescale 1ns/1ps
2  module counter4 (
3      input wire clk, rst, enable, down,
4      output reg [3:0] value
5  );
6
7      always @(posedge clk) begin
8          if (rst) value <= 4'd0;
9          else if (enable) begin
10             if (down) value <= value - 4'd1;
11             else value <= value + 4'd1;
12         end
13     end
14 endmodule
```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.

Problems Output Debug Console **Terminal** Ports

Python 3.14.7

RUN "C:\Program Files\Git\cmd\git.EXE" --version
git version 2.55.0.windows.5

RUN C:\msys64\mingw64\bin\iverilog.EXE -V ...

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/.sim/.constraints/의 파일을 직접 작성하세요.

* Terminal will be reused by tasks, press any key to close it.

check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.

```
● RUN C:\msys64\mingw64\bin\iverilog.EXE -g2012 -Wall -I C:\Users\young\Documents\lab2_01_counter\src -I C:\Users\young\Documents\lab2_01_counter\sim -s tb_counter4 -o C:\Users\young\Documents\lab2_01_counter\build\sim\run-22e81847d9a045f78e5b1e34c89b3075\sim.vvp C:\Users\young\Documents\lab2_01_counter\src\counter4.v C:\Users\young\Documents\lab2_01_counter\src\input_frontend.v C:\Users\young\Documents\lab2_01_counter\src\lab2_counter.v C:\Users\young\Documents\lab2_01_counter\sim\tb_counter4.sv

RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\lab2_01_counter\build\sim\run-22e81847d9a045f78e5b1e34c89b3075\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS counter4 checks=36
C:\Users\young\Documents\lab2_01_counter\sim\tb_counter4.sv:40: $finish called at 356000 (1ps)

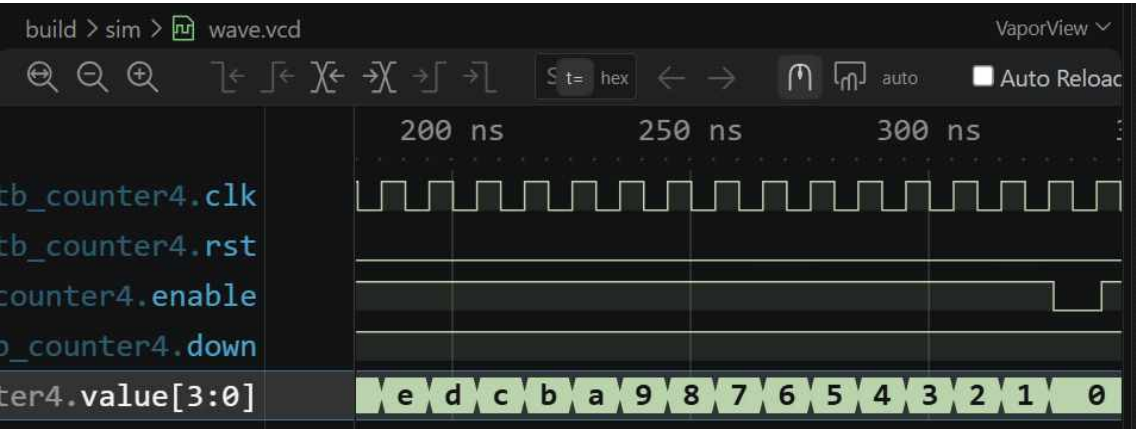
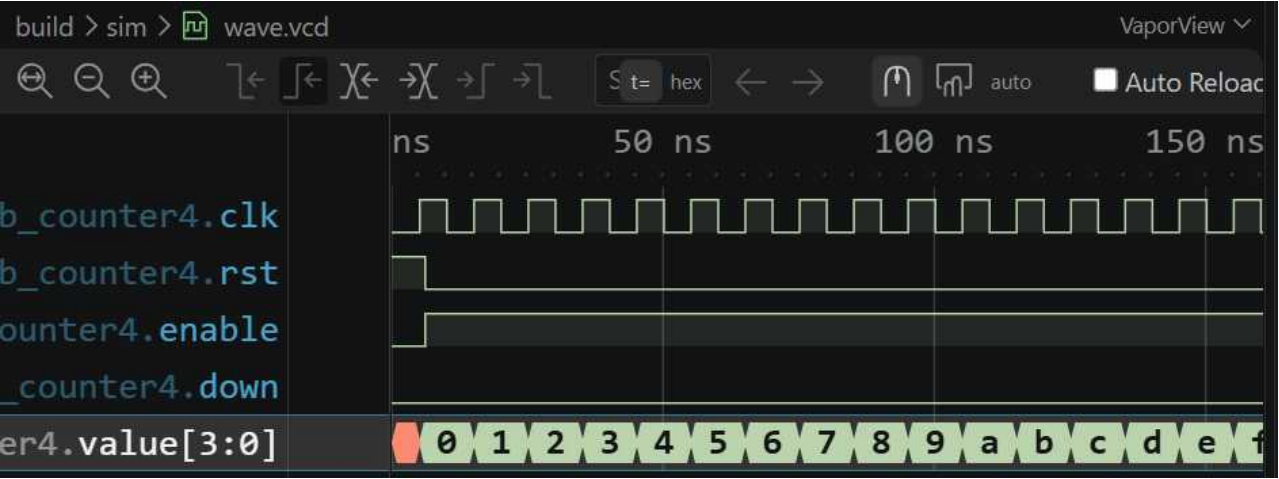
시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\young\Documents\lab2_01_counter\build\sim\wave.vcd
* Terminal will be reused by tasks, press any key to close it.
```

위 터미널 응답에서 알 수 있듯이 36개의 경우 모두 올바르게 출력되었다는 것을 알 수 있다.
또한 출력 시간 또한 356ns로 올바르게 출력되었다.

이후 파형 출력을 통해 코드가 정상 작동하는지
확인하였다.

구간	rst	enable	down	value 예상
초기 Reset	1	0	0	0
Up Count	0	1	0	0 → 1 → 2 → ... → 15 → 0
Down Count	0	1	1	0 → 15 → 14 → ... → 1 → 0
Hold	0	0	1	0 유지
Down Wrap	0	1	1	0 → 15
마지막 Reset	1	1	1	15 → 0

위는 이론에 따른 시뮬레이션 예상 결과이다.



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

```
always @(posedge clk) begin
    if (rst) value <= 4'd0;
    else if (enable) begin
        if (down) value <= value - 4'd1;
        else value <= value + 4'd1;
    end
end
```

+부분을 -로 바꾸어 실행하였다.

시뮬레이션 결과 오류가 발생하였다.

```
④ RUN C:\msys64\mingw64\bin\iverilog.EXE -g2012 -Wall -I C:\Users\young\Documents\lab2_01_counter\src -I C:\Users\young\Documents\lab2_01_counter\sim -s tb_counter4 -o C:\Users\young\Documents\lab2_01_counter\build\sim\run-5279cf15958c4ba3aaa8db0220a163d5\sim.vvp C:\Users\young\Documents\lab2_01_counter\src\counter4.v C:\Users\young\Documents\lab2_01_counter\src\input_frontend.v C:\Users\young\Documents\lab2_01_counter\src\lab2_counter.v C:\Users\young\Documents\lab2_01_counter\sim\tb_counter4.sv

RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\lab2_01_counter\build\sim\run-5279cf15958c4ba3aaa8db0220a163d5\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_FAIL up including 15 to 0 time=16000
FATAL: C:\Users\young\Documents\lab2_01_counter\sim\tb_counter4.sv:29: check failed
      Time: 16000 Scope: tb_counter4.check

ERROR: 종료 코드 1: C:\Users\young\Documents\lab2_01_counter\build\sim\run-5279cf15958c4ba3aaa8db0220a163d5\simulation.log

* The terminal process "C:\Users\young\AppData\Local\Microsoft\WindowsApps\python.EXE 'tools/lab1.py', 'simulate'" terminated with exit code: 1.
* Terminal will be reused by tasks, press any key to close it.
```

이후 오류를 수정하여 다시 시뮬레이트한 결과 복구에 성공하였다.

```
else if (enable) begin
    if (down) value <= value - 4'd1;
    else value <= value + 4'd1;
end
```

```
● RUN C:\msys64\mingw64\bin\iverilog.EXE -g2012 -Wall -I C:\Users\young\Documents\lab2_01_counter\src -I C:\Users\young\Documents\lab2_01_counter\sim -s tb_counter4 -o C:\Users\young\Documents\lab2_01_counter\build\sim\run-859ea3d7ffdb423794e35fa1906ff79a\sim.vvp C:\Users\young\Documents\lab2_01_counter\src\counter4.v C:\Users\young\Documents\lab2_01_counter\src\input_frontend.v C:\Users\young\Documents\lab2_01_counter\src\lab2_counter.v C:\Users\young\Documents\lab2_01_counter\sim\tb_counter4.sv
```

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\lab2_01_counter\build\sim\run-859ea3d7ffdb423794e35fa1906ff79a\sim.vvp
```

```
VCD info: dumpfile wave.vcd opened for output.
```

```
LAB2_PASS counter4 checks=36
```

```
C:\Users\young\Documents\lab2_01_counter\sim\tb_counter4.sv:40: $finish called at 356000 (1ps)
```

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.

```
WAVE C:\Users\young\Documents\lab2_01_counter\build\sim\wave.vcd
```

```
* Terminal will be reused by tasks, press any key to close it.
```

- 오류를 분석해보면, 테스트 벤치는 그대로 유지한 상태로 코드에 오류만 추가하였기 때문에, 테스트벤치에서의 기댓값과 코드의 출력값이 일치하지 않아 오류가 발생하였다고 분석할 수 있다.
- 즉, 원래는 down이 0일 때 0부터 시작하여 1씩 커져야 하지만, 0에서 시작하여 F, E, D처럼 감소하게 되어 기댓값과 다른 출력을 얻어 오류가 발생한 것이다.

6.2. Clock Divider

작성한 코드는 다음과 같다.

```
circuits > 02_clock_divider > src > ≡ clock_divider.v
1  `timescale 1ns/1ps
2  // DIVISOR must be an even integer >= 2.
3  // divided is an observation output; use tick as an enable on clk
4  module clock_divider #(parameter integer DIVISOR = 10) (
5      input wire clk, rst,
6      output reg divided,
7      output wire tick
8  );
9      localparam integer WIDTH = $clog2(DIVISOR);
10     reg [WIDTH-1:0] count;
11     assign tick = (!rst && (count == DIVISOR - 1));
12     always @(posedge clk) begin
13         if (rst) begin
14             count <= 0;
15             divided <= 1'b0;
16         end else begin
17             if (count == DIVISOR - 1) count <= 0;
18             else count <= count + 1'b1;
19             if (count == DIVISOR/2 - 1) divided <= 1'b1;
20             else if (count == DIVISOR - 1) divided <= 1'b0;
21         end
22     end
23 endmodule
24
```


이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -V
Icarus Verilog runtime version 13.0 (stable) (v13_0)

Copyright (c) 2001-2026 Stephen Williams (steve@icarus.com)

This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; either version 2 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License along
with this program; if not, write to the Free Software Foundation, Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/.sim/.constraints/의 파일을 직접 작성하세요.
* Terminal will be reused by tasks, press any key to close it.
```

check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.

```
RUN C:\msys64\mingw64\bin\iverilog.EXE -g2012 -Wall -I C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\src -I C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\sim -s tb_clock_divider -o C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\build\sim\run-1e1cf1721ebd4090844771d52a3c6caf\sim.vvp
p C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\src\clock_divider.v C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\src\input_frontend.v C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\src\lab2_clock_divider.v C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\sim\tb_clock_divider.sv
```

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\build\sim\run-1e1cf1721ebd4090844771d52a3c6caf\sim.vvp
```

```
VCD info: dumpfile wave.vcd opened for output.
```

```
LAB2_PASS clock_divider checks=129
```

```
C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\sim\tb_clock_divider.sv:19: $finish called at 436000 (1ps)
```

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.

```
WAVE C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\build\sim\wave.vcd
```

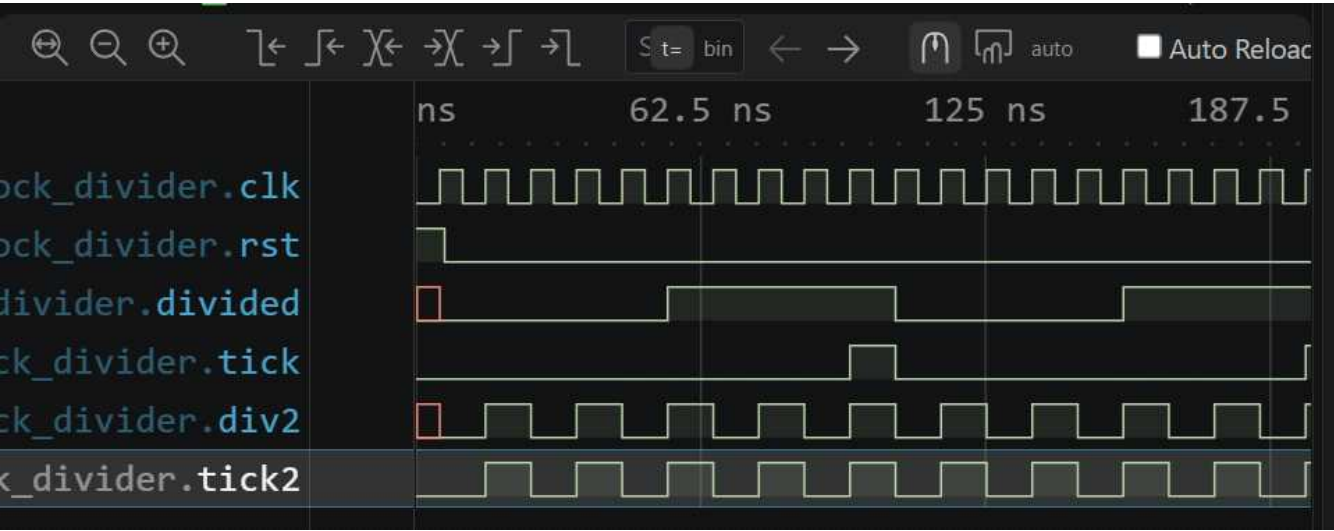
위 터미널 응답에서 알 수 있듯이 129개의 경우 모두 올바르게 출력되었다는 것을 알 수 있다.

또한 출력 시간 또한 436ns로 올바르게 출력되었다.

이후 파형 출력을 통해 시간 구간별로 올바른 출력을 내놓는지 분석하였다.

clk 상승엿지	count 변화 후	divided	tick	의미
15 ns	1	0	0	Low 구간
25 ns	2	0	0	Low
35 ns	3	0	0	Low
45 ns	4	0	0	Low
55 ns	5	1	0	divided 상승
65 ns	6	1	0	High
75 ns	7	1	0	High
85 ns	8	1	0	High
95 ns	9	1	1	tick 발생
105 ns	0	0	0	다음 주기 시작

위는 이론에 따른 시뮬레이션 예상 결과이다.



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

```
end else begin
    if (count == DIVISOR - 1) count <= 0;
    else count <= count + 1'b1;
    if (count == DIVISOR/2) divided <= 1'b1;
    else if (count == DIVISOR - 1) divided <= 1'
```

코드 중 -1 부분을 제거하여 오류를 발생시켰다.

시뮬레이션 결과 오류가 발생하였다.

```
④ RUN C:\msys64\mingw64\bin\iverilog.EXE -g2012 -Wall -I C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\src -I C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\sim -s tb_clock_divider -o C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\build\sim\run-00db54cffd2544f89edcdb4bd9728ffc\sim.vvp C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\src\clock_divider.v C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\src\input_frontend.v C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\src\lab2_clock_divider.v C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\sim\tb_clock_divider.sim.vv

RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\build\sim\run-00db54cffd2544f89edcdb4bd9728ffc\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_FAIL minimum divisor duty time=16000
FATAL: C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\sim\tb_clock_divider.sim:12: check failed
Time: 16000 Scope: tb_clock_divider.check

ERROR: 종료 코드 1: C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\build\sim\run-00db54cffd2544f89edcdb4bd9728ffc\simulation.log

* The terminal process "C:\Users\young\AppData\Local\Microsoft\WindowsApps\python.EXE 'tools/lab1.py', 'simulate'" terminated with exit code: 1.
* Terminal will be reused by tasks, press any key to close it.
```

이후 오류를 수정하여 다시 시뮬레이트한 결과 복구에 성공하였다.

```
        divided <= 1'b0;
    end else begin
        if (count == DIVISOR - 1) count <= 0;
        else count <= count + 1'b1;
        if (count == DIVISOR/2 - 1) divided <= 1'b1;
        else if (count == DIVISOR - 1) divided <= 1'b0;
    end
end
```

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\build\sim\run-ded6bf3d496d41b9a08cbdf03cf48156\sim.vvp
```

```
VCD info: dumpfile wave.vcd opened for output.
```

```
LAB2_PASS clock_divider checks=129
```

```
C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\sim\tb_clock_divider.sv:19: $finish called at 436000 (1ps)
```

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.

```
WAVE C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\build\sim\wave.vcd
```

```
* Terminal will be reused by tasks, press any key to close it.
```

- 오류를 분석해보면, 테스트 벤치는 그대로 유지한 상태로 코드에 오류만 추가하였기 때문에, 테스트벤치에서의 기댓값과 코드의 출력값이 일치하지 않아 오류가 발생하였다고 분석할 수 있다.
- 이렇게 바꾸면 divided의 상승 시점이 원래보다 한 클럭 늦어져, 특히 DIVISOR=2 인스턴스의 듀티 동작이 TB 예상값과 어긋나게 되어 오차가 발생하게 된 것이다.

6.3. Register

작성한 코드는 다음과 같다.

```
1  `timescale 1ns/1ps
2  module register_pair (
3      input wire clk, rst, load, transfer,
4      input wire [3:0] data_in,
5      output reg [3:0] stored, value
6  );
7      always @(posedge clk) begin
8          if (rst) begin
9              stored <= 4'd0;
10             value <= 4'd0;
11         end else begin
12             if (load) stored <= data_in;
13             if (transfer) value <= stored;
14         end
15     end
16 endmodule
17
```


이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -V
Icarus Verilog runtime version 13.0 (stable) (v13_0)

Copyright (c) 2001-2026 Stephen Williams (steve@icarus.com)

This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; either version 2 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License along
with this program; if not, write to the Free Software Foundation, Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/.sim/.constraints/의 파일을 직접 작성하세요.
* Terminal will be reused by tasks, press any key to close it.
```

check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\03_register\build\sim\run-ed2c12b68bec4dae8147c870b3093d4b\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS register_pair checks=7
C:\Users\young\Documents\fpga_lab2\circuits\03_register\sim\tb_register_pair.sv:19: $finish called at 66000 (1ps)

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\young\Documents\fpga_lab2\circuits\03_register\build\sim\wave.vcd
* Terminal will be reused by tasks, press any key to close it.
```

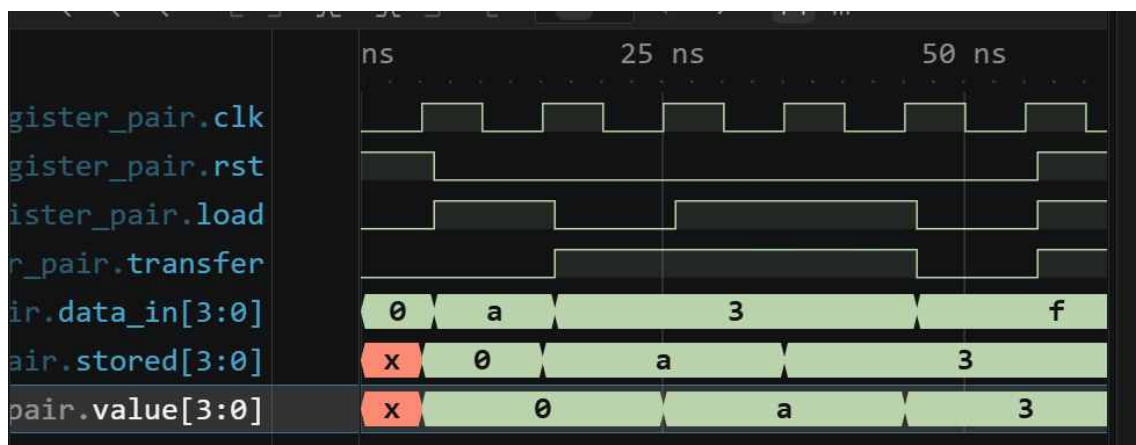
위 터미널 응답에서 알 수 있듯이 7개의 경우 모두 올바르게 출력되었다는 것을 알 수 있다.

또한 출력 시간 또한 66ns로 올바르게 출력되었다.

이후 파형 출력을 통해 시간 구간별로 올바른 출력을 내놓는지 분석하였다.

검사 시각	동작	stored	value
6 ns	reset	0	0
16 ns	A 저장	A	0
26 ns	입력은 3, 기존 저장값 전달	A	A
36 ns	3 저장 + 전달 동시	3	A
46 ns	다음 edge에도 동시	3	3
56 ns	둘 다 0, 입력 F	3	3
66 ns	reset 우선	0	0

위의 표의 결과가 시뮬레이션의 예상 결과값이다.



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

```
11      end else begin
12          if (load) stored <= data_in;
13          if (transfer) value <= data in;
14      end
15  end
```

코드의 'stored' 부분을 'data in'으로 바꾸어 오류를 발생시키겠다.

시뮬레이션 결과 오류가 발생하였다.

```
⊙ RUN C:\msys64\mingw64\bin\iverilog.EXE -g2012 -Wall -I C:\Users\young\Documents\fpga_lab2\circuits\03_register\src -I C:\Users\young\Documents\fpga_lab2\circuits\03_register\sim -s tb_register_pair -o C:\Users\young\Documents\fpga_lab2\circuits\03_register\build\sim\run-7c150e77be854fa8bf6d92f4f72dcdf2\sim.vvp C:\Users\young\Documents\fpga_lab2\circuits\03_register\src\register_pair.v C:\Users\young\Documents\fpga_lab2\circuits\03_register\src\input_frontend.v C:\Users\young\Documents\fpga_lab2\circuits\03_register\src\lab2_register.v C:\Users\young\Documents\fpga_lab2\circuits\03_register\sim\tb_register_pair.sv
C:\Users\young\Documents\fpga_lab2\circuits\03_register\src\register_pair.v:13: syntax error
C:\Users\young\Documents\fpga_lab2\circuits\03_register\src\register_pair.v:13: error: Malformed statement

ERROR: 종료 코드 2: C:\Users\young\Documents\fpga_lab2\circuits\03_register\build\sim\run-7c150e77be854fa8bf6d92f4f72dcdf2\compile.log

* The terminal process "C:\Users\young\AppData\Local\Microsoft\WindowsApps\python.EXE 'tools/lab1.py', 'simulate'" terminated with exit code: 1.
* Terminal will be reused by tasks, press any key to close it.
```


이후 오류를 수정하여 다시 시뮬레이션한 결과 복구에 성공하였다.

```
        end else begin
            if (load) stored <= data_in;
            if (transfer) value <= stored;
        end
    end
end
```

```
03_register\sim\tb_register_pair.vv

RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\03_register\build\sim\run-70b9d5280a9e41d7b1994d424ce2f316\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS register_pair checks=7
C:\Users\young\Documents\fpga_lab2\circuits\03_register\sim\tb_register_pair.vv:19: $finish called at 66000 (1ps)

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\young\Documents\fpga_lab2\circuits\03_register\build\sim\wave.vcd
* Terminal will be reused by tasks, press any key to close it.
```

- 오류를 분석해보면, 테스트 벤치는 그대로 유지한 상태로 코드에 오류만 추가하였기 때문에, 테스트벤치에서의 기댓값과 코드의 출력값이 일치하지 않아 오류가 발생하였다고 분석할 수 있다.
- 위에서와 같이 코드를 변경하게 되면, 저장된 값을 전달하는 것이 아닌 현재 입력을 바로 전달하게 된다. 이에 따라 register의 원래 동작과 다르게 되어 오류가 발생한 것이다.

6.4. Shift register

작성한 코드는 다음과 같다.

```
1  `timescale 1ns/1ps
2
3  module shift_register4 (
4      input wire clk, rst, enable, serial_in,
5      output reg [3:0] value
6  );
7      always @(posedge clk) begin
8          if (rst) value <= 4'd0;
9          else if (enable) value <= {serial_in, value[3:1]};
10     end
11 endmodule
12
```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -V
Icarus Verilog runtime version 13.0 (stable) (v13_0)

Copyright (c) 2001-2026 Stephen Williams (steve@icarus.com)

This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; either version 2 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License along
with this program; if not, write to the Free Software Foundation, Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/.sim/.constraints/의 파일을 직접 작성하세요.
```

check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\04_shift_register\build\sim\run-ad36d0b652524646b-aa6d5ca10cf280b\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS shift_register4 checks=8
C:\Users\young\Documents\fpga_lab2\circuits\04_shift_register\sim\tb_shift_register4.sv:30: $finish called at 106000 (1ps)

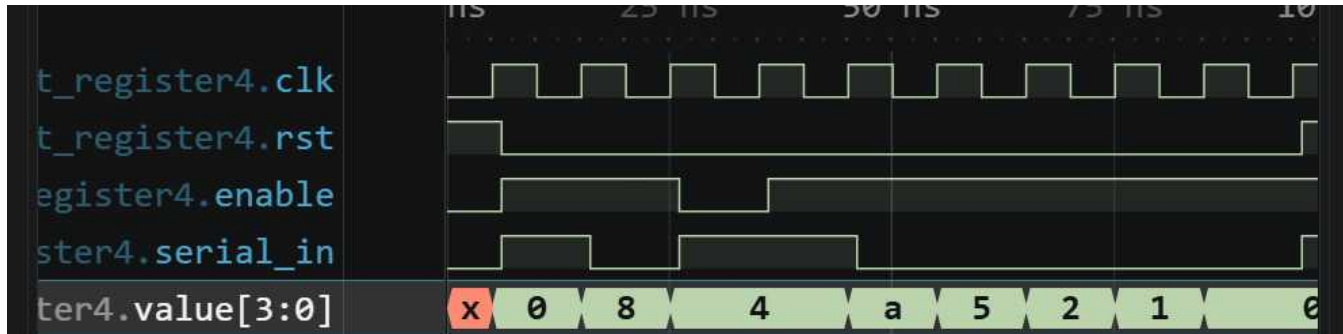
시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\young\Documents\fpga_lab2\circuits\04_shift_register\build\sim\wave.vcd
```

위 터미널 응답에서 알 수 있듯이 8개의 경우 모두 올바르게 출력되었다는 것을 알 수 있다.
또한 출력 시간 또한 106ns로 올바르게 출력되었다.

이후 파형 출력을 통해 시간 구간별로 올바른 출력을 내놓는지 분석하였다.

검사 시각	동작	value
6 ns	reset	0000
16 ns	serial_in=1, enable=1	1000 = 8
26 ns	serial_in=0	0100 = 4
36 ns	enable=0, 입력은 1	0100 유지
46 ns	enable=1, 입력 1	1010 = A
56 ns	입력 0	0101 = 5
96 ns	0을 네 번 더 입력	0000
106 ns	rst=1	0000

위는 이론에 따른 시뮬레이션 예상 결과이다.



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

```
8         if (rst) value <= 4'd0;
9         else if (enable) value <= {value[2:0], value[3:1]};
10        end
11    endmodule
```

코드의 부등호 'serial_in' 을 'value[2:0]' 으로 바꾸어 오류를 발생시켰다.

시뮬레이션 결과 오류가 발생하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\04_shift_register\build\sim\run-809f180ef10843a79f830aaec7916e3\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_FAIL shift 1 time=16000
FATAL: C:\Users\young\Documents\fpga_lab2\circuits\04_shift_register\sim\tb_shift_register4.sv:22: check failed
      Time: 16000  Scope: tb_shift_register4.check

ERROR: 종료 코드 1: C:\Users\young\Documents\fpga_lab2\circuits\04_shift_register\build\sim\run-809f180ef10843a79f830aaec7916e3\simulation.log

* The terminal process "C:\Users\young\AppData\Local\Microsoft\WindowsApps\python.EXE 'tools/lab1.py', 'simulate'" terminated with exit code: 1.
* Terminal will be reused by tasks, press any key to close it.
```

이후 오류를 수정하여 다시 시뮬레이트한 결과 복구에 성공하였다.

```
8      if (rst) value <= 4'd0;
9      else if (enable) value <= {serial_in, value[3:1]};
```

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\04_shift_register\build\sim\run-4462d92ff2e640aaae2f84404adb38f5\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS shift_register4 checks=8
C:\Users\young\Documents\fpga_lab2\circuits\04_shift_register\sim\tb_shift_register4.sv:30: $finish called at 106000 (1ps)

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\young\Documents\fpga_lab2\circuits\04_shift_register\build\sim\wave.vcd
* Terminal will be reused by tasks, press any key to close it.
```

- 오류를 분석해보면, 테스트 벤치는 그대로 유지한 상태로 코드에 오류만 추가하였기 때문에, 테스트벤치에서의 기댓값과 코드의 출력값이 일치하지 않아 오류가 발생하였다고 분석할 수 있다.

- 의도적으로 코드를 바꾸게 되면서 시프트 방향을 반대로 바꾸었다. 이에 따라 예측값과 다른 출력값을 얻어 오차가 발생한 것이다.

6.5. PISO

작성한 코드는 다음과 같다.

```
1  `timescale 1ns/1ps
2
3  module piso4 (
4      input wire clk, rst, load, enable,
5      input wire [3:0] data_in,
6      output wire serial_out,
7      output reg [3:0] value
8  );
9      assign serial_out = value[3];
10
11     always @(posedge clk) begin
12         if (rst) value <= 4'd0;
13         else if (load) value <= data_in;
14         else if (enable) value <= {value[2:0], 1'b0};
15     end
16 endmodule
```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.

```
Copyright (c) 2001-2026 Stephen Williams (steve@icarus.com)

This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; either version 2 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License along
with this program; if not, write to the Free Software Foundation, Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/.sim/.constraints/의 파일을 직접 작성하세요.
```

check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\05_piso\build\sim\run-49736e3815e642d59838167928320f74\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS piso4 checks=114
C:\Users\young\Documents\fpga_lab2\circuits\05_piso\sim\tb_piso4.sv:30: $finish called at 976000 (1ps)

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\young\Documents\fpga_lab2\circuits\05_piso\build\sim\wave.vcd
```

위 터미널 응답에서 알 수 있듯이 114개의 경우 모두 올바르게 출력되었다는 것을 알 수 있다.
또한 출력 시간 또한 976ns로 올바르게 출력되었다.

이후 파형 출력을 통해 시간 구간별로 올바른 출력을 내놓는지 분석하였다.

이론에 따르면,

만약 입력이 A=1010인 구간은

616 ns → A 병렬 load 확인

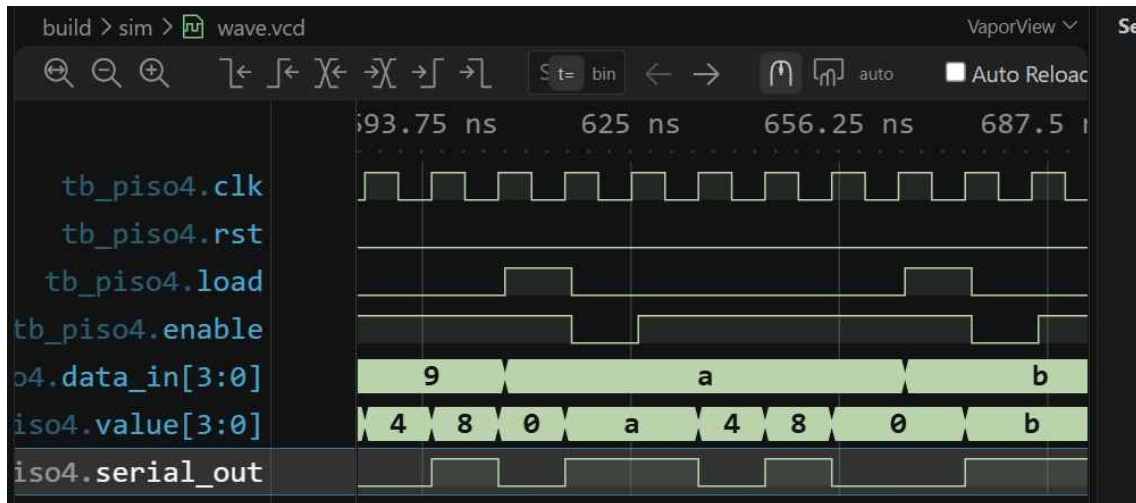
626 ns → hold, serial_out = 1

636 ns → serial_out = 0

646 ns → serial_out = 1

656 ns → serial_out = 0

666 ns → 네 번 shift 후 value = 0000 으로 출력되어야 한다.



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

```

8  );
9  assign serial_out = value[0];
10
11 always @(posedge clk) begin

```

코드의 value값을 3에서 0으로 바꾸어 오류를 발생시켰다.

시뮬레이션 결과 오류가 발생하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\05_piso\build\sim\run-7b5ea152f2094b8fa3734faf7db8ee0d\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_FAIL MSB first before edge time=86000
FATAL: C:\Users\young\Documents\fpga_lab2\circuits\05_piso\sim\tb_piso4.sv:22: check failed
      Time: 86000  Scope: tb_piso4.check

ERROR: 종료 코드 1: C:\Users\young\Documents\fpga_lab2\circuits\05_piso\build\sim\run-7b5ea152f2094b8fa3734faf7db8ee0d\simulation.log
```

이후 오류를 수정하여 다시 시뮬레이트한 결과 복구에 성공하였다.

```
9      assign serial_out = value[3];
10
11     always @(posedge clk) begin
```

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\05_piso\build\sim\run-9a8e9aa46abe420294069f5106d55526\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS piso4 checks=114
C:\Users\young\Documents\fpga_lab2\circuits\05_piso\sim\tb_piso4.sv:30: $finish called at 976000 (1ps)
```

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.

- 오류를 분석해보면, 테스트 벤치는 그대로 유지한 상태로 코드에 오류만 추가하였기 때문에, 테스트벤치에서의 기댓값과 코드의 출력값이 일치하지 않아 오류가 발생하였다고 분석할 수 있다.
- value값을 의도적으로 3에서 0으로 바꾸면서 MSB가 아닌 LSB를 출력하도록 하였다. 이에 따라 오류가 발생한 것이다.

6.6. Moore

작성한 코드는 다음과 같다.

```
circuits > 06_moore > src > ≡ moore_cycle.v
1  `timescale 1ns/1ps
2
3  module moore_cycle (
4      input wire clk, rst, enable, advance,
5      output reg [1:0] value
6  );
7      always @(posedge clk) begin
8          if (rst) value <= 2'b00;
9          else if (enable && advance) begin
10             case (value)
11                 2'b00: value <= 2'b01;
12                 2'b01: value <= 2'b10;
13                 default: value <= 2'b00;
14             endcase
15         end
16     end
17 endmodule
18
```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.

```
Copyright (c) 2001-2026 Stephen Williams (steve@icarus.com)

This program is free software; you can redistribute it and/or
modify it under the terms of the GNU General Public License as published
by the Free Software Foundation; either version 2 of the License,
or (at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with this program; if not, write to the Free Software Foundation,
Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/.sim/.constraints/의 파일을 직접 작성
하세요.
```

check tools를 실행한 결과 오류가 발생하지 않았으므로
다음으로 simulate를 실행하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\06_moore\build\sim\run-17a4eb22e4cf4df3b832f6294466e5e3\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS moore_cycle checks=23
C:\Users\young\Documents\fpga_lab2\circuits\06_moore\sim\tb_moore_cycle.sv:30: $finish called at 226000 (1ps)

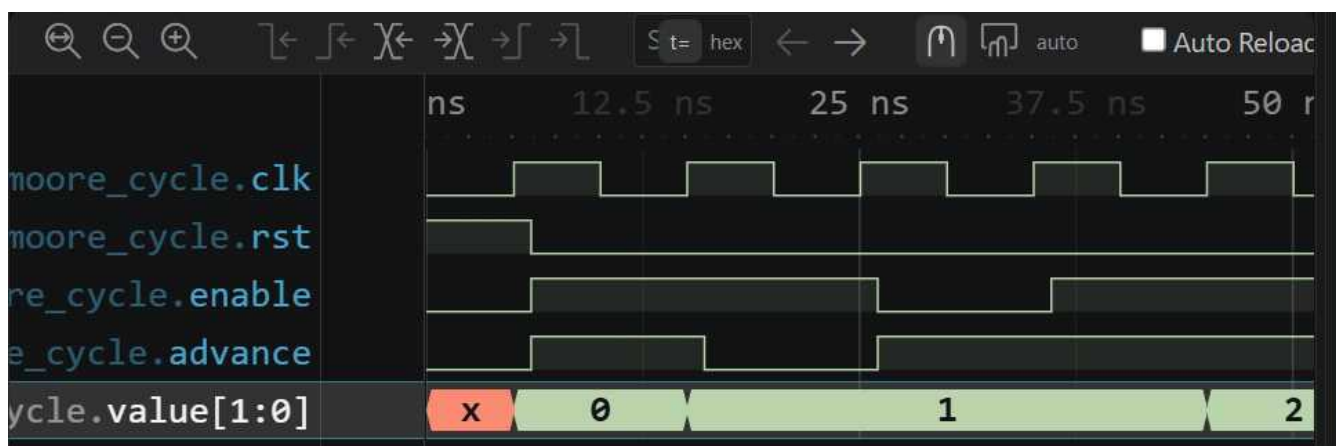
시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\young\Documents\fpga_lab2\circuits\06_moore\build\sim\wave.vcd
```

위 터미널 응답에서 알 수 있듯이 23개의 경우 모두
올바르게 출력되었다는 것을 알 수 있다.
또한 출력 시간 또한 226ns로 올바르게 출력되었다.

이후 파형 출력을 통해 시간 구간별로 올바른 출력을 내놓는지 분석하였다.

이론에 따르면 첫 번째 상태 순환은 다음과 같아야 한다.

6 ns : reset → 00
7 ns : advance=1 → 여전히 00
16 ns : 유효 edge → 01
26 ns : advance=0 → 01 유지
36 ns : enable=0 → 01 유지
46 ns : 둘 다 1 → 10
56 ns : 다음 유효 edge → 00



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

```
10      case (value)
11      |      2'b00: value <= 2'b01;
12      |      2'b01: value <= 2'b00;
13      |      default: value <= 2'b00;
```

코드의 '10' 을 '00' 으로 바꾸어 오류를 발생시켰다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\06_moore\build\sim\run-27cee18768454f44a26da51602fb109b\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_FAIL S1 to S2 time=46000
FATAL: C:\Users\young\Documents\fpga_lab2\circuits\06_moore\sim\tb_moore_cycle.sv:22: check failed
      Time: 46000  Scope: tb_moore_cycle.check

ERROR: 종료 코드 1: C:\Users\young\Documents\fpga_lab2\circuits\06_moore\build\sim\run-27cee18768454f44a26da51602fb109b\simulation.log

* The terminal process "C:\Users\young\AppData\Local\Microsoft\WindowsApps\python.EXE 'tools/lab1.py', 'simulate'" terminated with exit code: 1.
```

시뮬레이션 결과 오류가 발생하였다.

이후 오류를 수정하여 다시 시뮬레이트한 결과 복구에 성공하였다.

```
10      case (value)
11      |    2'b00: value <= 2'b01;
12      |    2'b01: value <= 2'b10;
13      |    default: value <= 2'b00;
14      endcase
```

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\06_moore\build\sim\run-a37d16003da54e87947ea3643873b8aa\sim.vvp
```

```
VCD info: dumpfile wave.vcd opened for output.
```

```
LAB2_PASS moore_cycle checks=23
```

```
C:\Users\young\Documents\fpga_lab2\circuits\06_moore\sim\tb_moore_cycle.sv:30: $finish called at 226000 (1ps)
```

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.

```
WAVE C:\Users\young\Documents\fpga_lab2\circuits\06_moore\build\sim\wave.vcd
```

- 오류를 분석해보면, 테스트 벤치는 그대로 유지한 상태로 코드에 오류만 추가하였기 때문에, 테스트벤치에서의 기댓값과 코드의 출력값이 일치하지 않아 오류가 발생하였다고 분석할 수 있다.
- 의도적으로 코드를 변형하여 상태가 s1에서 s2로 넘어가야 하는 상황에서 s0으로 넘어가게 되어 오류가 발생한 것이다.

6.7. Mealy

작성한 코드는 다음과 같다.

```
1  `timescale 1ns/1ps
2
3  module mealy_toggle (
4      input wire clk, rst, enable, bit_in,
5      output reg state,
6      output wire [1:0] value
7  );
8      always @(posedge clk) begin
9          if (rst) state <= 1'b0;
10         else if (enable && bit_in) state <= ~state;
11     end
12
13     // Reset changes state at the rising edge; value is
14     assign value = !bit_in ? 2'b00 : (state ? 2'b01 : 2'b10);
15 endmodule
16
```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -V
Icarus Verilog runtime version 13.0 (stable) (v13_0)

Copyright (c) 2001-2026 Stephen Williams (steve@icarus.com)

This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; either version 2 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License along
with this program; if not, write to the Free Software Foundation, Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/.sim/.constraints/의 파일을 직접 작성하세요.
```

check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\07_mealy\build\sim\run-967cc398478c44fcb3cd4ba63f90773d\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS mealy_toggle checks=11
C:\Users\young\Documents\fpga_lab2\circuits\07_mealy\sim\tb_mealy_toggle.sv:30: $finish called at 67000 (1ps)
```

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.

위 터미널 응답에서 알 수 있듯이 11개의 경우 모두 올바르게 출력되었다는 것을 알 수 있다.
또한 출력 시간 또한 67ns로 올바르게 출력되었다.

이론에 따르면 각 상태별 결과는 다음과 같이 얻어져야한다.

state = 0

bit_in = 0

value = 00

일 때, bit_in = 1이 되면,

state = 0

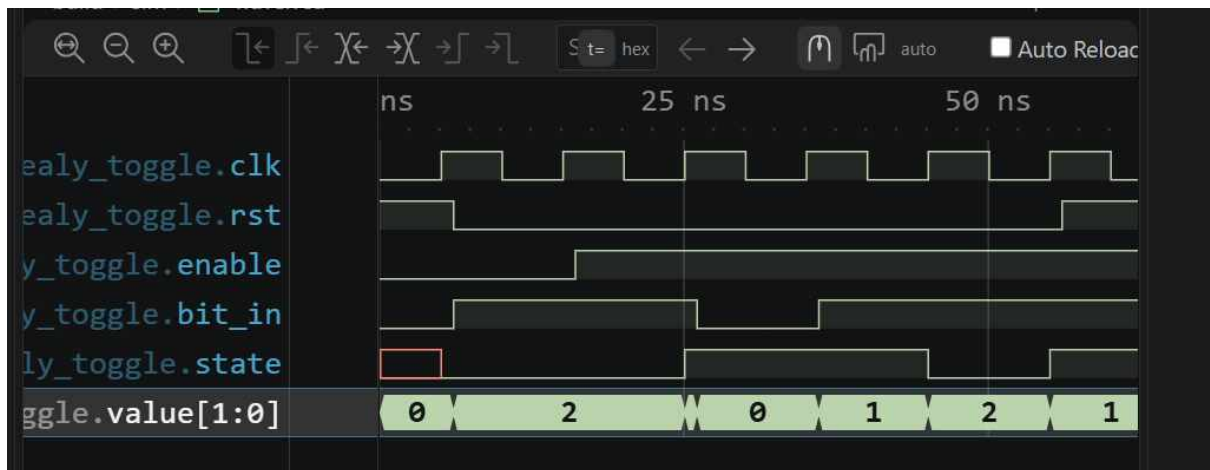
value = 10 으로 바뀐다.

반대로 state = 1이면,

bit_in = 1 => value = 01

bit_in = 0 => value = 00

로 변화한다.



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

```

11     end
12
13     // Reset changes state at the rising edge; value is combinational
14     assign value = !bit_in ? 2'b00 : (state ? 2'b10 : 2'b01);
15 endmodule
16

```

코드의 부등호 '01' 과 '10' 을 서로 바꾸어 오류를 발생시켰다.

시뮬레이션 결과 오류가 발생하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\07_mealy\build\sim\run-4657059703f64c3b945d39a3fc7f1f81\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_FAIL S0 input changes between clocks time=7000
FATAL: C:\Users\young\Documents\fpga_lab2\circuits\07_mealy\sim\tb_mealy_toggle.sv:
22: check failed
      Time: 7000 Scope: tb_mealy_toggle.check

ERROR: 종료 코드 1: C:\Users\young\Documents\fpga_lab2\circuits\07_mealy\build\sim\
run-4657059703f64c3b945d39a3fc7f1f81\simulation.log

* The terminal process "C:\Users\young\AppData\Local\Microsoft\WindowsApps\python
.EXE 'tools/lab1.py', 'simulate'" terminated with exit code: 1.
* Terminal will be reused by tasks, press any key to close it.
```

이후 오류를 수정하여 다시 시뮬레이트한 결과 복구에 성공하였다.

```
11     end
12
13     // Reset changes state at the rising edge; value is combinational
14     assign value = !bit_in ? 2'b00 : (state ? 2'b01 : 2'b10);
15 endmodule
```

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\07_mealy\build\sim\run-fad8ce1e69e043ee9d295ab0f92503ed\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS mealy_toggle checks=11
C:\Users\young\Documents\fpga_lab2\circuits\07_mealy\sim\tb_mealy_toggle.sv:30: $finish called at 67000 (1ps)

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\young\Documents\fpga_lab2\circuits\07_mealy\build\sim\wave.vcd
* Terminal will be reused by tasks, press any key to close it.
```


- 오류를 분석해보면, 테스트 벤치는 그대로 유지한 상태로 코드에 오류만 추가하였기 때문에, 테스트벤치에서의 기댓값과 코드의 출력값이 일치하지 않아 오류가 발생하였다고 분석할 수 있다.
- 정상은 state=0, bit_in=1 => 10, state=1, bit_in=1 => 01 이지만, 오류 회로에서는 state=0, bit_in=1 => 01 , state=1, bit_in=1 => 10 으로 바뀌게 되므로 오류가 발생한 것이다.

6.8. Segment Scan

작성한 코드는 다음과 같다.

```
1  `timescale 1ns/1ps
2  // Logical outputs are active-high. Apply board polarity in the wrapper.
3  // Each slot contains one active interval followed by one blank interval.
4  module segment_scan8 (
5      input wire clk, rst, enable,
6      input wire [31:0] digits,
7      output reg [7:0] select,
8      output reg [7:0] segments,
9      output reg [2:0] index
10 );
11     reg blank;
12     reg [3:0] nibble;
13
14     always @(posedge clk) begin
15         if (rst) begin
16             index <= 0;
17             blank <= 1'b1;
18         end else if (enable) begin
19             blank <= ~blank;
20             if (!blank)
21                 index <= index + 1'b1;
22         end
23     end
24
25     always @* begin
26         nibble = digits >> (index * 4);
27         select = blank ? 8'h00 : (8'h01 << index);
28
29         // abcdefg, dp; decimal point remains off.
30         case (nibble)
31             4'h0: segments = 8'b11111100;
32             4'h1: segments = 8'b01100000;
33             4'h2: segments = 8'b11011010;
34             4'h3: segments = 8'b11110010;
35             4'h4: segments = 8'b01100110;
36             4'h5: segments = 8'b10110110;
37             4'h6: segments = 8'b10111110;
38             4'h7: segments = 8'b11100000;
39             4'h8: segments = 8'b11111110;
40             4'h9: segments = 8'b11110110;
41             4'ha: segments = 8'b11101110;
42             4'hb: segments = 8'b00111110;
43             4'hc: segments = 8'b10011100;
44             4'hd: segments = 8'b01111010;
45             4'he: segments = 8'b10011110;
46             4'hf: segments = 8'b10001110;
47             default: segments = 8'h00;
48         endcase
49     end
50 endmodule
```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -V
Icarus Verilog runtime version 13.0 (stable) (v13_0)

Copyright (c) 2001-2026 Stephen Williams (steve@icarus.com)

This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; either version 2 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License along
with this program; if not, write to the Free Software Foundation, Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/.sim/.constraints/의 파일을 직접 작성하세요.
```

check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\08_segment_scan\build\sim\run-2ff706ec968648809f206456b6d23a57\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS segment_scan8 checks=194
C:\Users\young\Documents\fpga_lab2\circuits\08_segment_scan\sim\tb_segment_scan8.sv:30: $finish called at 1306000 (1ps)

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\young\Documents\fpga_lab2\circuits\08_segment_scan\build\sim\wave.vcd
* Terminal will be reused by tasks, press any key to close it.
```

위 터미널 응답에서 알 수 있듯이 194개의 경우 모두
올바르게 출력되었다는 것을 알 수 있다.
또한 출력 시간 또한 1306ns로 올바르게 출력되었다.

이론에 따르면 다음과 같은 결과가 얻어져야한다.

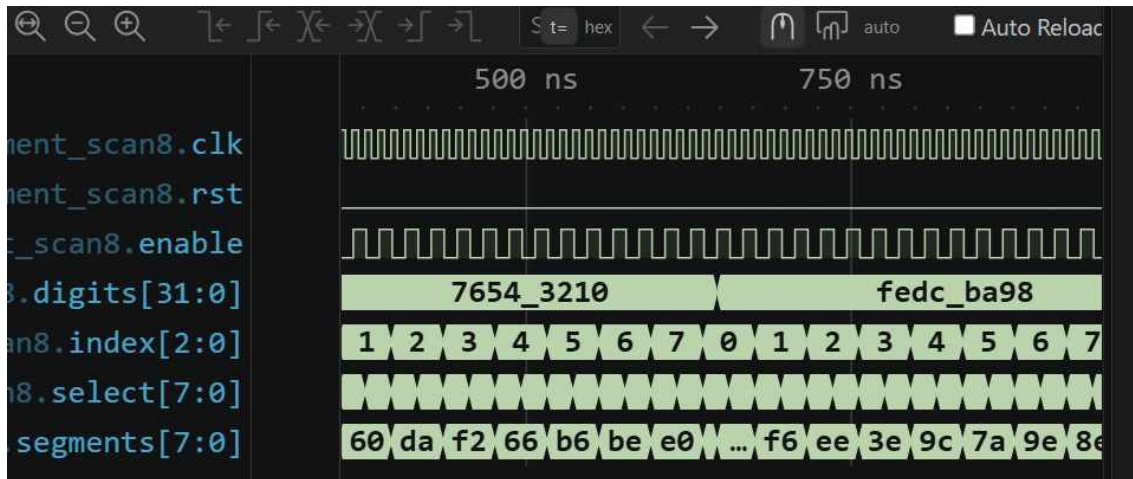
자리 활성화

=> enable=0이라도 활성화 상태 유지

=> blank

=> enable=0이라도 blank 유지

=> 다음 자리



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

```

24
25     always @* begin
26         nibble = digits >> (index * 4);
27         select = (8'h01 << index);
28
29         // abcdefg, dp; decimal point remains off.
30         case (nibble)

```

코드의 'select = blank ? 8'h00 : (8'h01 << index);' 을 'select = (8'h01 << index);' 으로 바꾸어 오류를 발생시켰다.

시뮬레이션 결과 오류가 발생하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\08_segment_scan\build\sim\run-592dc825a1384b3fa46381614b6eee8\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_FAIL reset blanks digit zero time=6000
FATAL: C:\Users\young\Documents\fpga_lab2\circuits\08_segment_scan\sim\tb_segment_scan8.sv:22: check failed
      Time: 6000 Scope: tb_segment_scan8.check

ERROR: 종료 코드 1: C:\Users\young\Documents\fpga_lab2\circuits\08_segment_scan\build\sim\run-592dc825a1384b3fa46381614b6eee8\simulation.log

* The terminal process "C:\Users\young\AppData\Local\Microsoft WindowsApps\python.EXE 'tools/lab1.py', 'simulate'" terminated with exit code: 1.
```

이후 오류를 수정하여 다시 시뮬레이트한 결과 복구에 성공하였다.

```
25     always @* begin
26         nibble = digits >> (index * 4);
27         select = blank ? 8'h00 : (8'h01 << index);
28
29         // abcdefg, dp; decimal point remains off.
30         case (nibble)
```

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\08_segment_scan\build\sim\run-e4f47a9ba056455dbfde19e2d66b6da0\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS segment_scan8 checks=194
C:\Users\young\Documents\fpga_lab2\circuits\08_segment_scan\sim\tb_segment_scan8.sv:30: $finish called at 1306000 (1ps)

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.

WAVE C:\Users\young\Documents\fpga_lab2\circuits\08_segment_scan\build\sim\wave.vcd
```


- 오류를 분석해보면, 테스트 벤치는 그대로 유지한 상태로 코드에 오류만 추가하였기 때문에, 테스트벤치에서의 기댓값과 코드의 출력값이 일치하지 않아 오류가 발생하였다고 분석할 수 있다.
- 즉 blank=1이어도 항상 현재 자리 하나가 켜지게 만들어 오류가 발생한 것이다.

7. 포트 및 핀 번호 입력

7.1. Counter

아래와 같이 대상 부품 및 핀 제약을 설정했다.

```
1  # Combo II-DLD S75 / xc7s75fgga484-1 / main clock 1 kHz.
2  # Pin source and array polarity: evidence/board-pin-provenance.json.
3  set_property PACKAGE_PIN B6 [get_ports {clk}]
4  set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
5  set_property PACKAGE_PIN K4 [get_ports {rst}]
6  set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
7  set_property PACKAGE_PIN N8 [get_ports {button}]
8  set_property IOSTANDARD LVCMOS33 [get_ports {button}]
9  set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
11 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
13 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
15 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
17 set_property PACKAGE_PIN T1 [get_ports {sw[4]}]
18 set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
19 set_property PACKAGE_PIN U2 [get_ports {sw[5]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
21 set_property PACKAGE_PIN W3 [get_ports {sw[6]}]
22 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
23 set_property PACKAGE_PIN Y1 [get_ports {sw[7]}]
24 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
25 set_property PACKAGE_PIN N5 [get_ports {led[0]}]
26 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
27 set_property PACKAGE_PIN M1 [get_ports {led[1]}]
28 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
29 set_property PACKAGE_PIN M3 [get_ports {led[2]}]
30 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
31 set_property PACKAGE_PIN M7 [get_ports {led[3]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
33 set_property PACKAGE_PIN N7 [get_ports {led[4]}]
34 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
35 set_property PACKAGE_PIN M2 [get_ports {led[5]}]
36 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
37 set_property PACKAGE_PIN M4 [get_ports {led[6]}]
38 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
39 set_property PACKAGE_PIN L4 [get_ports {led[7]}]
40 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
41 create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
42 set_false_path -from [get_ports {rst button sw[*]}]
43
```

7.2. Clock divider

아래와 같이 대상 부품 및 핀 제약을 설정했다.

```
ircuits > 02_clock_divider > constraints > lab2_clock_divider.xdc
1  # Combo II-DLD S75 / xc7s75fgga484-1 / main clock 1 kHz.
2  # Pin source and array polarity: evidence/board-pin-provenance.json.
3  set_property PACKAGE_PIN B6 [get_ports {clk}]
4  set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
5  set_property PACKAGE_PIN K4 [get_ports {rst}]
6  set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
7  set_property PACKAGE_PIN N8 [get_ports {button}]
8  set_property IOSTANDARD LVCMOS33 [get_ports {button}]
9  set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
11 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
13 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
15 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
17 set_property PACKAGE_PIN T1 [get_ports {sw[4]}]
18 set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
19 set_property PACKAGE_PIN U2 [get_ports {sw[5]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
21 set_property PACKAGE_PIN W3 [get_ports {sw[6]}]
22 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
23 set_property PACKAGE_PIN Y1 [get_ports {sw[7]}]
24 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
25 set_property PACKAGE_PIN N5 [get_ports {led[0]}]
26 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
27 set_property PACKAGE_PIN M1 [get_ports {led[1]}]
28 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
29 set_property PACKAGE_PIN M3 [get_ports {led[2]}]
30 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
31 set_property PACKAGE_PIN M7 [get_ports {led[3]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
33 set_property PACKAGE_PIN N7 [get_ports {led[4]}]
34 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
35 set_property PACKAGE_PIN M2 [get_ports {led[5]}]
36 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
37 set_property PACKAGE_PIN M4 [get_ports {led[6]}]
38 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
39 set_property PACKAGE_PIN L4 [get_ports {led[7]}]
40 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
41 create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
42 set_false_path -from [get_ports {rst button sw[*]}]
43
```


7.3. Register

아래와 같이 대상 부품 및 핀 제약을 설정했다.

```
1  # Combo II-DLD S75 / xc7s75fpga484-1 / main clock 1 kHz.
2  # Pin source and array polarity: evidence/board-pin-provenance.json.
3  set_property PACKAGE_PIN B6 [get_ports {clk}]
4  set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
5  set_property PACKAGE_PIN K4 [get_ports {rst}]
6  set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
7  set_property PACKAGE_PIN N8 [get_ports {button}]
8  set_property IOSTANDARD LVCMOS33 [get_ports {button}]
9  set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
11 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
13 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
15 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
17 set_property PACKAGE_PIN T1 [get_ports {sw[4]}]
18 set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
19 set_property PACKAGE_PIN U2 [get_ports {sw[5]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
21 set_property PACKAGE_PIN W3 [get_ports {sw[6]}]
22 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
23 set_property PACKAGE_PIN Y1 [get_ports {sw[7]}]
24 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
25 set_property PACKAGE_PIN N5 [get_ports {led[0]}]
26 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
27 set_property PACKAGE_PIN M1 [get_ports {led[1]}]
28 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
29 set_property PACKAGE_PIN M3 [get_ports {led[2]}]
30 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
31 set_property PACKAGE_PIN M7 [get_ports {led[3]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
33 set_property PACKAGE_PIN N7 [get_ports {led[4]}]
34 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
35 set_property PACKAGE_PIN M2 [get_ports {led[5]}]
36 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
37 set_property PACKAGE_PIN M4 [get_ports {led[6]}]
38 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
39 set_property PACKAGE_PIN L4 [get_ports {led[7]}]
40 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
41 create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
42 set_false_path -from [get_ports {rst button sw[*]}]
43
```

7.4. Shift register

아래와 같이 대상 부품 및 핀 제약을 설정했다.

```
1  # Combo II-DLD S75 / xc7s75fpga484-1 / main clock 1 kHz.
2  # Pin source and array polarity: evidence/board-pin-provenance.json.
3
4  set_property PACKAGE_PIN B6 [get_ports {clk}]
5  set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
6
7  set_property PACKAGE_PIN K4 [get_ports {rst}]
8  set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
9
10 set_property PACKAGE_PIN N8 [get_ports {button}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {button}]
12
13 set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
15
16 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
18
19 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
21
22 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
23 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
24
25 set_property PACKAGE_PIN T1 [get_ports {sw[4]}]
26 set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
27
28 set_property PACKAGE_PIN U2 [get_ports {sw[5]}]
29 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
30
31 set_property PACKAGE_PIN W3 [get_ports {sw[6]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
33
34 set_property PACKAGE_PIN Y1 [get_ports {sw[7]}]
35 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
36
37 set_property PACKAGE_PIN N5 [get_ports {led[0]}]
38 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
39
40 set_property PACKAGE_PIN M1 [get_ports {led[1]}]
41 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
42
43 set_property PACKAGE_PIN M3 [get_ports {led[2]}]
44 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
45
46 set_property PACKAGE_PIN M7 [get_ports {led[3]}]
47 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
48
49 set_property PACKAGE_PIN N7 [get_ports {led[4]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
51
52 set_property PACKAGE_PIN M2 [get_ports {led[5]}]
53 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
54
55 set_property PACKAGE_PIN M4 [get_ports {led[6]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
57
58 set_property PACKAGE_PIN L4 [get_ports {led[7]}]
59 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
60
61 create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
62
63 set_false_path -from [get_ports {rst button sw[*]}]
64
```


7.5. PISO

아래와 같이 대상 부품 및 핀 제약을 설정했다.

```
rcuits > US_piso > constraints > lab2_piso.xdc
1  # Combo II-DLD S75 / xc7s75fgga484-1 / main clock 1 kHz.
2  # Pin source and array polarity: evidence/board-pin-provenance.json.
3
4  set_property PACKAGE_PIN B6 [get_ports {clk}]
5  set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
6
7  set_property PACKAGE_PIN K4 [get_ports {rst}]
8  set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
9
10 set_property PACKAGE_PIN N8 [get_ports {button}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {button}]
12
13 set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
15
16 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
18
19 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
21
22 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
23 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
24
25 set_property PACKAGE_PIN T1 [get_ports {sw[4]}]
26 set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
27
28 set_property PACKAGE_PIN U2 [get_ports {sw[5]}]
29 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
30
31 set_property PACKAGE_PIN W3 [get_ports {sw[6]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
```

```
34 set_property PACKAGE_PIN Y1 [get_ports {sw[7]}]
35 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
36
37 set_property PACKAGE_PIN N5 [get_ports {led[0]}]
38 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
39
40 set_property PACKAGE_PIN M1 [get_ports {led[1]}]
41 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
42
43 set_property PACKAGE_PIN M3 [get_ports {led[2]}]
44 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
45
46 set_property PACKAGE_PIN M7 [get_ports {led[3]}]
47 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
48
49 set_property PACKAGE_PIN N7 [get_ports {led[4]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
51
52 set_property PACKAGE_PIN M2 [get_ports {led[5]}]
53 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
54
55 set_property PACKAGE_PIN M4 [get_ports {led[6]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
57
58 set_property PACKAGE_PIN L4 [get_ports {led[7]}]
59 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
60
61 create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
62
63 set_false_path -from [get_ports {rst button sw[*]}]
64
```


7.6. Moore

아래와 같이 대상 부품 및 핀 제약을 설정했다.

```
rcuits > 06_moore > constraints > ≡ lab2_moore.xdc
1  # Combo II-DLD S75 / xc7s75fgga484-1 / main clock 1 kHz.
2  # Pin source and array polarity: evidence/board-pin-provenance.json.
3
4  set_property PACKAGE_PIN B6 [get_ports {clk}]
5  set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
6
7  set_property PACKAGE_PIN K4 [get_ports {rst}]
8  set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
9
10 set_property PACKAGE_PIN N8 [get_ports {button}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {button}]
12
13 set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
15
16 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
18
19 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
21
22 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
23 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
24
25 set_property PACKAGE_PIN T1 [get_ports {sw[4]}]
26 set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
27
28 set_property PACKAGE_PIN U2 [get_ports {sw[5]}]
29 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
30
31 set_property PACKAGE_PIN W3 [get_ports {sw[6]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
33
34 set_property PACKAGE_PIN Y1 [get_ports {sw[7]}]
35 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
36
37 set_property PACKAGE_PIN N5 [get_ports {led[0]}]
38 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
39
40 set_property PACKAGE_PIN M1 [get_ports {led[1]}]
41 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
42
43 set_property PACKAGE_PIN M3 [get_ports {led[2]}]
44 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
45
46 set_property PACKAGE_PIN M7 [get_ports {led[3]}]
47 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
48
49 set_property PACKAGE_PIN N7 [get_ports {led[4]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
51
52 set_property PACKAGE_PIN M2 [get_ports {led[5]}]
53 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
54
55 set_property PACKAGE_PIN M4 [get_ports {led[6]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
57
58 set_property PACKAGE_PIN L4 [get_ports {led[7]}]
59 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
60
61 create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
62
63 set_false_path -from [get_ports {rst button sw[*]}]
64
```

7.7. Mealy

아래와 같이 대상 부품 및 핀 제약을 설정했다.

```
1  # Combo II-DLD S75 / xc7s75fgga484-1 / main clock 1 kHz.
2  # Pin source and array polarity: evidence/board-pin-provenance.json.
3
4  set_property PACKAGE_PIN B6 [get_ports {clk}]
5  set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
6
7  set_property PACKAGE_PIN K4 [get_ports {rst}]
8  set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
9
10 set_property PACKAGE_PIN N8 [get_ports {button}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {button}]
12
13 set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
15
16 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
18
19 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
21
22 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
23 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
24
25 set_property PACKAGE_PIN T1 [get_ports {sw[4]}]
26 set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
27
28 set_property PACKAGE_PIN U2 [get_ports {sw[5]}]
29 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
30
31 set_property PACKAGE_PIN W3 [get_ports {sw[6]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
33
34 set_property PACKAGE_PIN Y1 [get_ports {sw[7]}]
35 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
```

```
37 set_property PACKAGE_PIN N5 [get_ports {led[0]}]
38 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
39
40 set_property PACKAGE_PIN M1 [get_ports {led[1]}]
41 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
42
43 set_property PACKAGE_PIN M3 [get_ports {led[2]}]
44 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
45
46 set_property PACKAGE_PIN M7 [get_ports {led[3]}]
47 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
48
49 set_property PACKAGE_PIN N7 [get_ports {led[4]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
51
52 set_property PACKAGE_PIN M2 [get_ports {led[5]}]
53 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
54
55 set_property PACKAGE_PIN M4 [get_ports {led[6]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
57
58 set_property PACKAGE_PIN L4 [get_ports {led[7]}]
59 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
60
61 create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
62
63 set_false_path -from [get_ports {rst button sw[*]}]
64
```


7.8. Segment scan

아래와 같이 대상 부품 및 핀 제약을 설정했다.

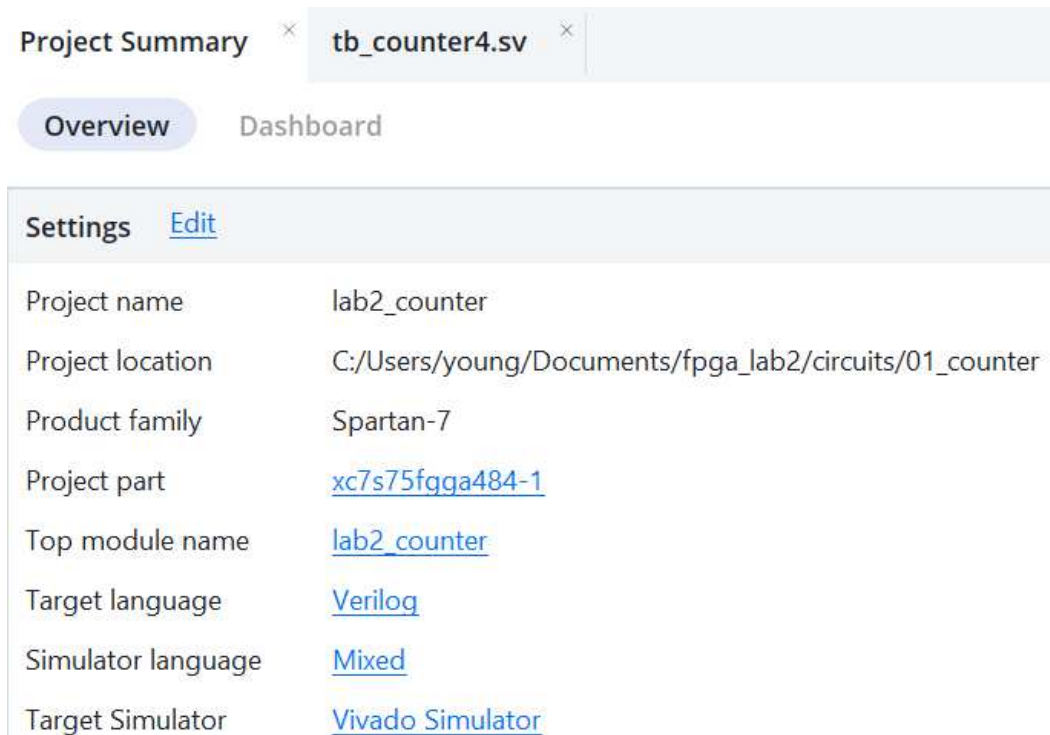
```
1  # Combo II-DLD S75 / xc7s75fgga484-1 / main clock 1 kHz.
2  # Pin source and array polarity: evidence/board-pin-provenance.json.
3
4  set_property PACKAGE_PIN B6 [get_ports {clk}]
5  set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
6  set_property PACKAGE_PIN K4 [get_ports {rst}]
7  set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
8  set_property PACKAGE_PIN N8 [get_ports {button}]
9  set_property IOSTANDARD LVCMOS33 [get_ports {button}]
10
11 set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
13 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
15 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
17 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
18 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
19 set_property PACKAGE_PIN T1 [get_ports {sw[4]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
21 set_property PACKAGE_PIN U2 [get_ports {sw[5]}]
22 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
23 set_property PACKAGE_PIN W3 [get_ports {sw[6]}]
24 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
25 set_property PACKAGE_PIN Y1 [get_ports {sw[7]}]
26 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
27
28 set_property PACKAGE_PIN N5 [get_ports {led[0]}]
29 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
30 set_property PACKAGE_PIN M1 [get_ports {led[1]}]
31 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
32 set_property PACKAGE_PIN M3 [get_ports {led[2]}]
33 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
34 set_property PACKAGE_PIN M7 [get_ports {led[3]}]
35 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
36 set_property PACKAGE_PIN N7 [get_ports {led[4]}]
37 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
38 set_property PACKAGE_PIN M2 [get_ports {led[5]}]
39 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
40 set_property PACKAGE_PIN M4 [get_ports {led[6]}]
41 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
42 set_property PACKAGE_PIN L4 [get_ports {led[7]}]
43 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
```

```
45 set_property PACKAGE_PIN H2 [get_ports {seg_data[0]}]
46 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[0]}]
47 set_property PACKAGE_PIN J7 [get_ports {seg_data[1]}]
48 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[1]}]
49 set_property PACKAGE_PIN J3 [get_ports {seg_data[2]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[2]}]
51 set_property PACKAGE_PIN J1 [get_ports {seg_data[3]}]
52 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[3]}]
53 set_property PACKAGE_PIN E4 [get_ports {seg_data[4]}]
54 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[4]}]
55 set_property PACKAGE_PIN E2 [get_ports {seg_data[5]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[5]}]
57 set_property PACKAGE_PIN F5 [get_ports {seg_data[6]}]
58 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[6]}]
59 set_property PACKAGE_PIN F1 [get_ports {seg_data[7]}]
60 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[7]}]
61
62 set_property PACKAGE_PIN K5 [get_ports {seg_com[0]}]
63 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[0]}]
64 set_property PACKAGE_PIN K3 [get_ports {seg_com[1]}]
65 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[1]}]
66 set_property PACKAGE_PIN K1 [get_ports {seg_com[2]}]
67 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[2]}]
68 set_property PACKAGE_PIN L6 [get_ports {seg_com[3]}]
69 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[3]}]
70 set_property PACKAGE_PIN G3 [get_ports {seg_com[4]}]
71 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[4]}]
72 set_property PACKAGE_PIN G1 [get_ports {seg_com[5]}]
73 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[5]}]
74 set_property PACKAGE_PIN H6 [get_ports {seg_com[6]}]
75 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[6]}]
76 set_property PACKAGE_PIN H4 [get_ports {seg_com[7]}]
77 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[7]}]
78
79 create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
80
81 set_false_path -from [get_ports {rst button sw[*]}]
```


8. vivado 내의 실험 과정 및 결과

8.1. Counter

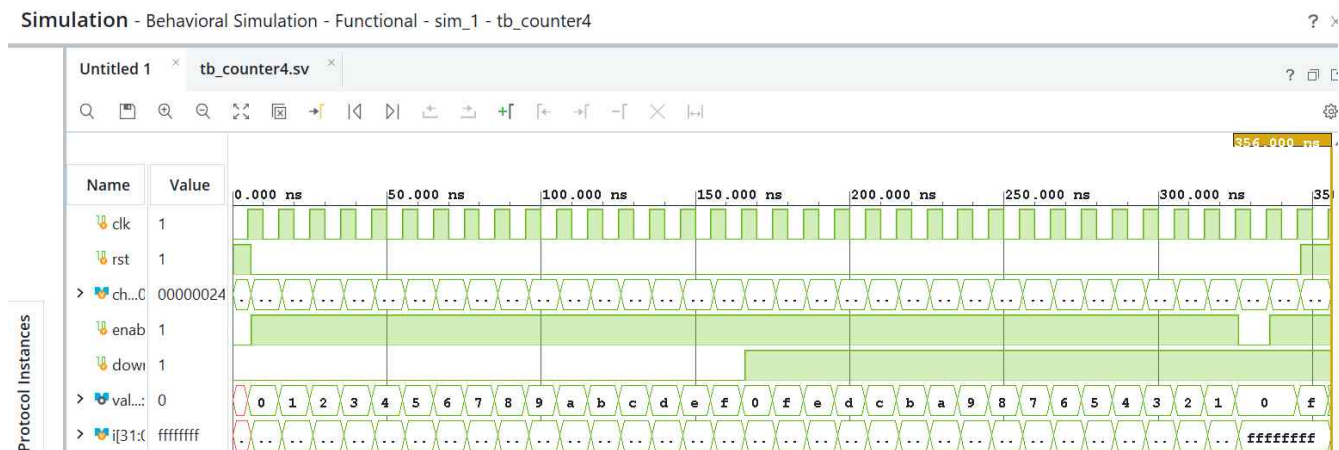
- Vivado 내의 settings는 다음과 같다.



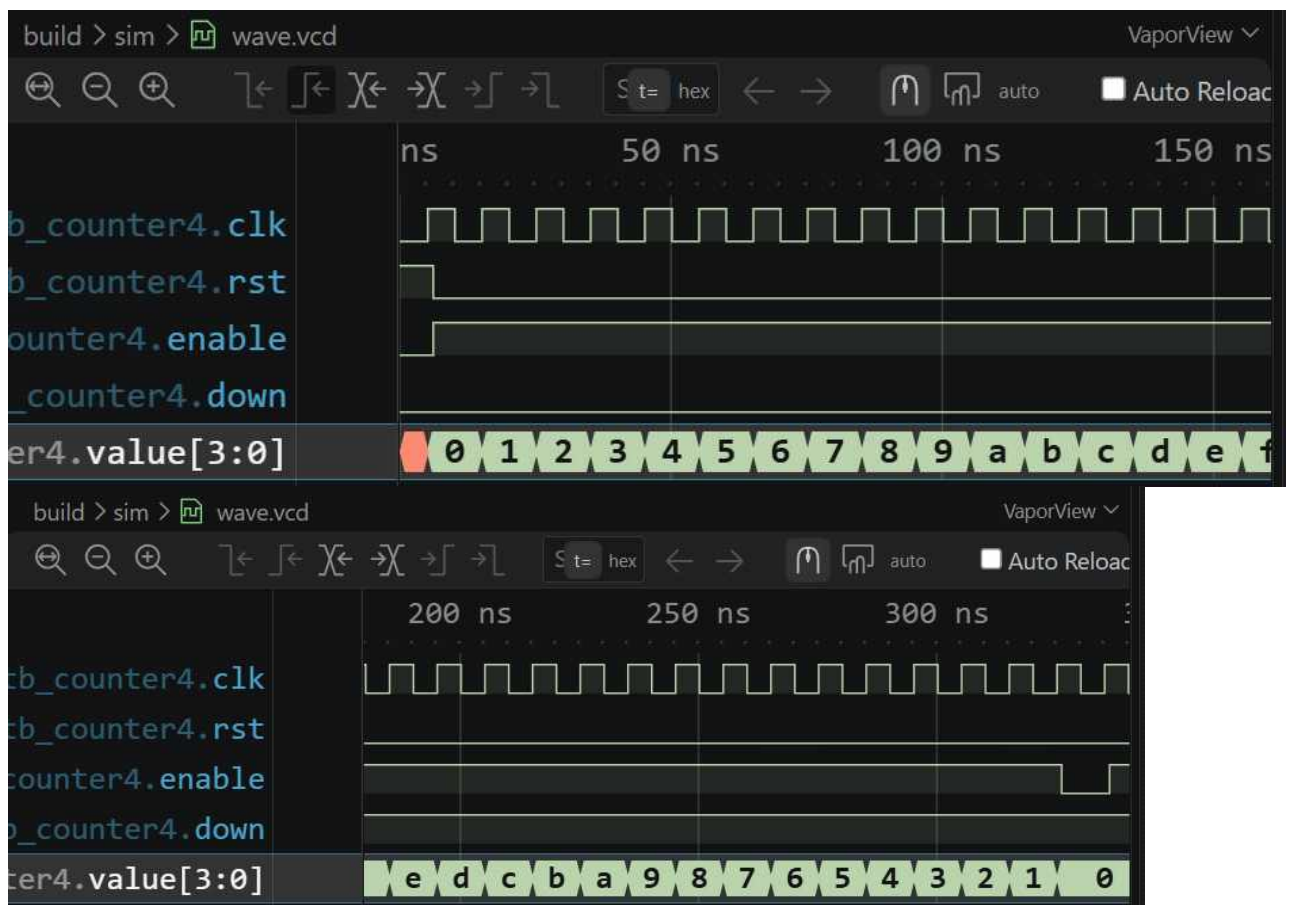
첨부한 사진들과 같이 Top설정 또한 알맞게 진행하였다.

- ▼ Design Sources (1)
 - > ● **lab2_counter** (lab2_counter.v) (2)
- ▼ Constraints (1)
 - ▼ constrs_1 (1)
 - lab2_counter.xdc
- ▼ Simulation Sources (2)
 - ▼ **sim_1** (2) (active)
 - > ● **tb_counter4** (tb_counter4.sv) (1)
 - > ● lab2_counter (lab2_counter.v) (2)
- ▼ Utility Sources
 - utils_1

이를 Run simulation 하여 얻은 파형이다.



이렇게 얻은 파형이 올바른지 확인하기 위해 위에서 작성한 vscode 내에서의 시뮬레이션 결과와 비교하였다.



이론에 부합한 결과는 위에서 설명하였고, 그와 vscode의 파형이 일치하였기에 vscode에서의 파형이 올바른 파형이라고 할 수 있다.

이를 vivado파형과 비교해보면, 정확히 같은 파형의 결과를 얻은 것을 알 수 있다.

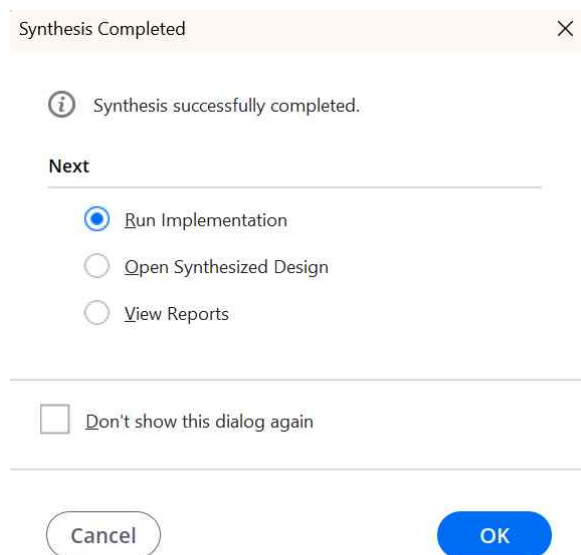
즉, 문제 없이 알맞은 파형을 얻었다.

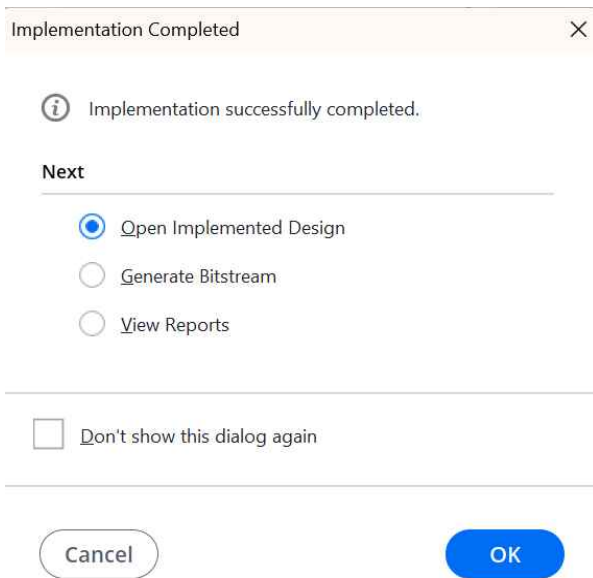
위의 파형을 얻기 위해 시뮬레이션을 행한 결과 아래와 같은 Tcl Console의 결과값을 얻었다.

```
LAB2_PASS counter4 checks=36
$finish called at time : 356 ns : File "C:/Users/young/Documents/fpga_lab2/circuits/01_counter/sim/tb_counter4.sv" Line 19
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_counter4_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:03 ; elapsed = 00:00:07 . Memory (MB): peak = 1952.039 ; gain = 124.723
```

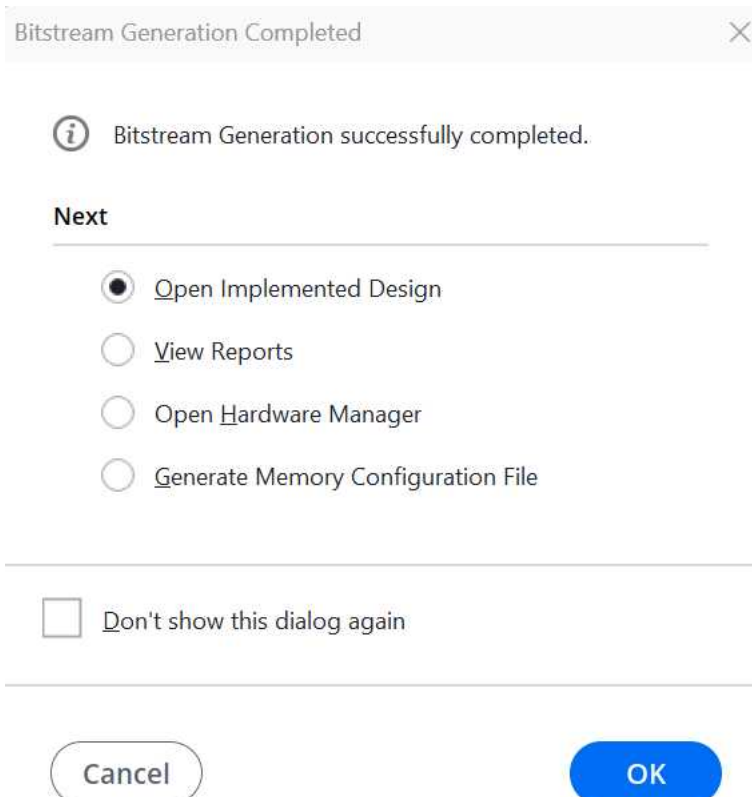
정상적으로 36개의 경우를 연산하였고, 출력시간 또한 356ns로 기록되었다.

이후 Run synthesis와 Run Implementation을 진행하였다.





두 결과 모두 성공하였고, 이후 generate bitstream을 진행하였다.



최종적으로 아래와 같은 결과를 오류 없이 얻었다.

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF
✓ synth_1	constrs_1	synth_design Complete!												10	31
✓ impl_1	constrs_1	write_bitstream Complete!	9999%	0.00%	0.122	0.00%		0.000	0.103	0	8 Warn			11	17

사용한 커밋은 이후 차례 <증빙 및 소스 commit 링크>에 주소로 기록하였다.

- Bit 해시:

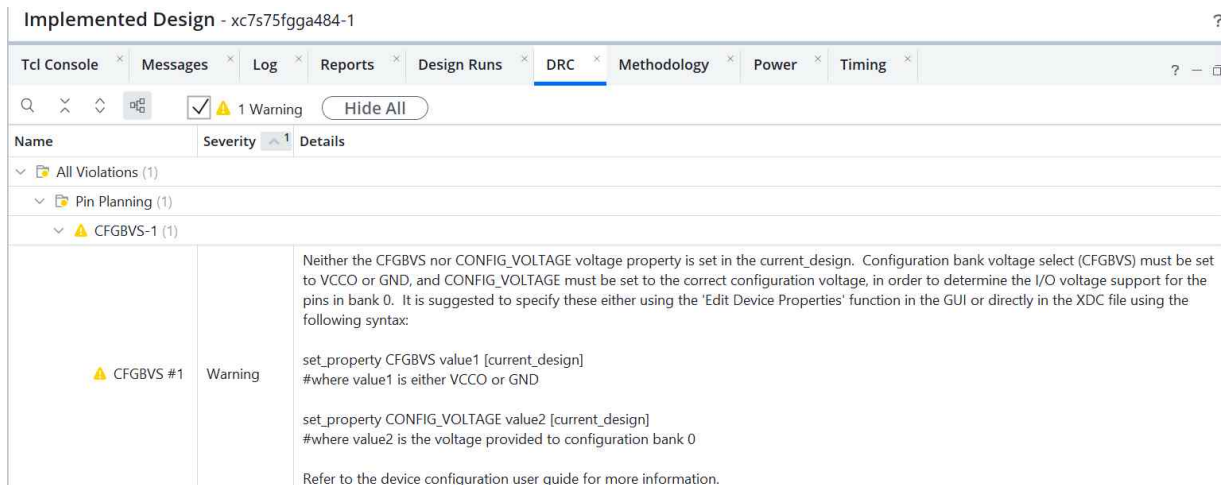
SHA-256:

4A174988A83F6401F1EB6D932568F8759FCF377F08C52
AC7F09B3B324F997168

- Bit 경로

Bitstream:

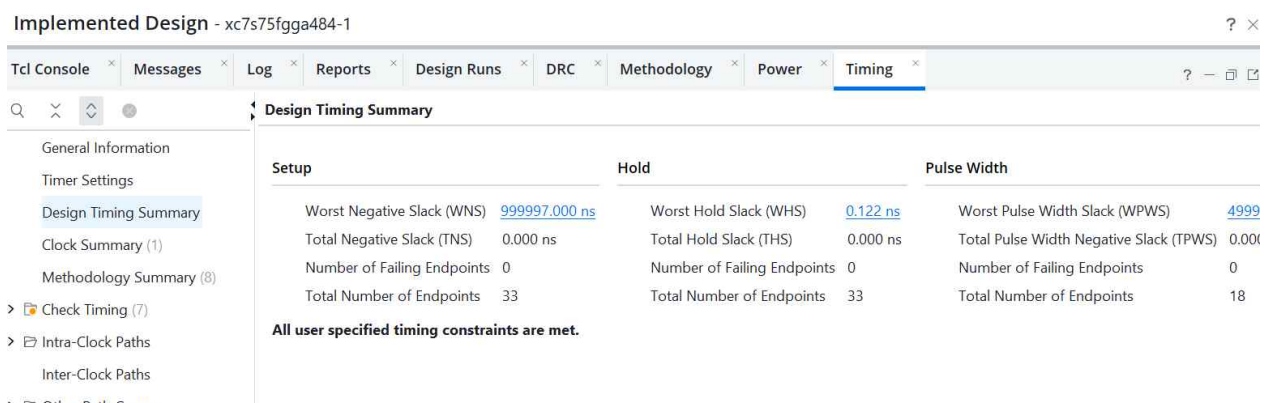
circuits\01_counter\lab2_counter.runs\impl_1\lab2_counter.bit



- DRC 검사 결과 Error는 0건이었으며, CFGBVS-1 Warning 1건이 발생하였다.

해당 경고는 CFGBVS 및 CONFIG_VOLTAGE 속성이 설정되지 않아 발생한 것으로, 실제 보드 구성 전압과 대조가 필요한 항목이다.

- Timing 결과 검증



Vivado의 Implementation 이후 Timing Summary를 확인한 결과, Worst Negative Slack(WNS)은 999997.000 ns, Total Negative Slack(TNS)은 0.000 ns로 나타났으며 Setup Timing에서 실패한 Endpoint는 0개였다. 또한 Worst Hold Slack(WHS)은 0.122 ns, Total Hold Slack(THS)은 0.000 ns이며 Hold Timing에서도 실패한

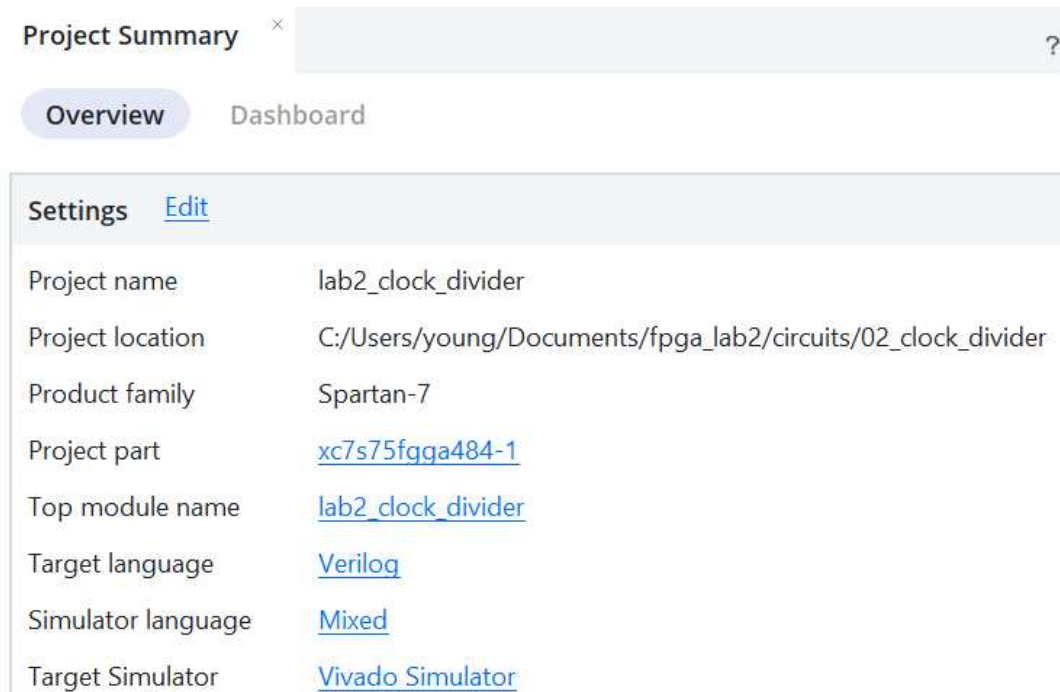
Endpoint는 0개였다.

따라서 Setup 및 Hold Timing에서 모두 timing violation이 발생하지 않았으며, Vivado에서도 ****"All user specified timing constraints are met."****가 확인되었다.

이를 통해 설계된 회로가 설정된 클럭 조건에서 요구되는 타이밍 제약을 만족함을 확인하였다.

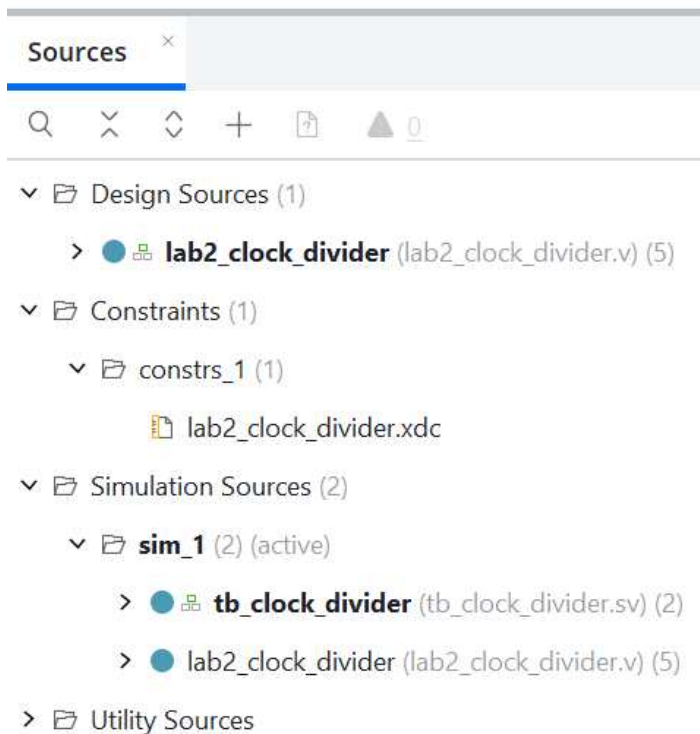
8.2. Clock divider

- Vivado 내의 settings는 다음과 같다.

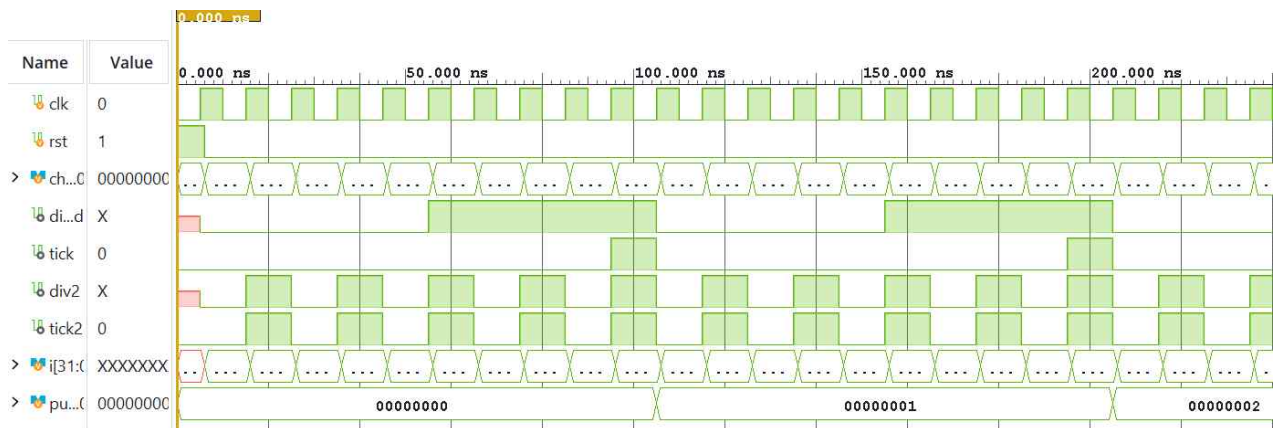


첨부한 사진들과 같이 Top설정 또한 알맞게 진행하였다.

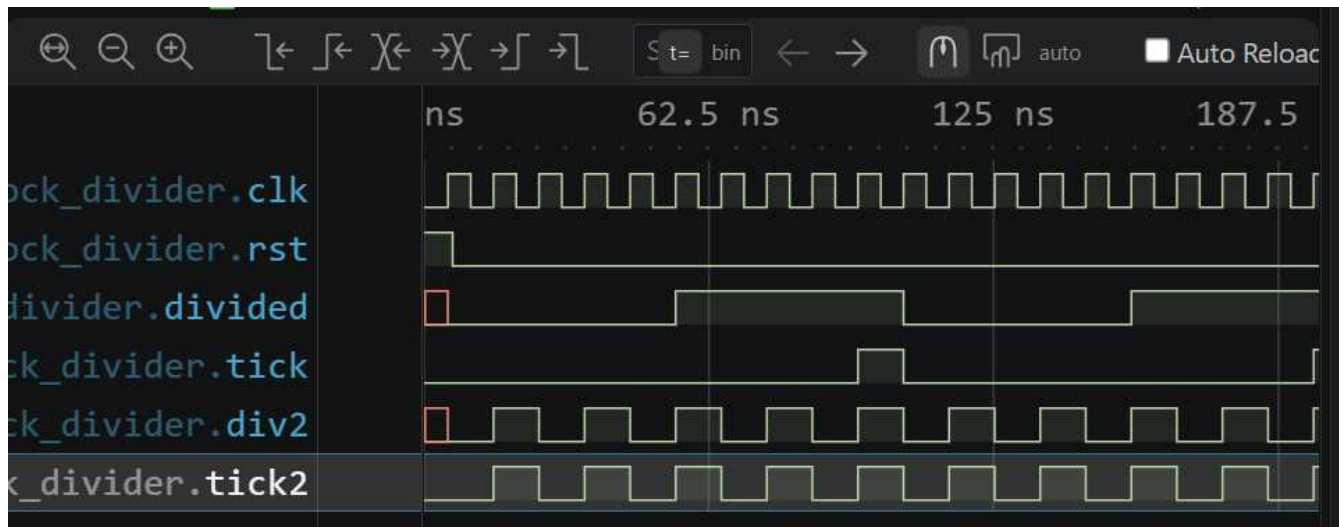
Project Manager - lab2_clock_divider



이를 Run simulation 하여 얻은 파형이다.



이렇게 얻은 파형이 올바른지 확인하기 위해 위에서 작성한 vscode 내에서의 시뮬레이션 결과와 비교하였다.



이론에 부합한 결과는 위에서 설명하였고, 그와 vscode의 파형이 일치하였기에 vscode에서의 파형이 올바른 파형이라고 할 수 있다.

이를 vivado 파형과 비교해보면, 정확히 같은 파형의 결과를 얻은 것을 알 수 있다.

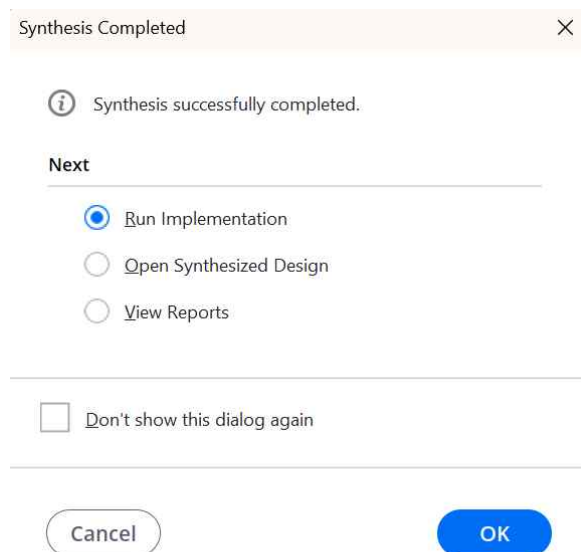
즉, 문제 없이 알맞은 파형을 얻었다.

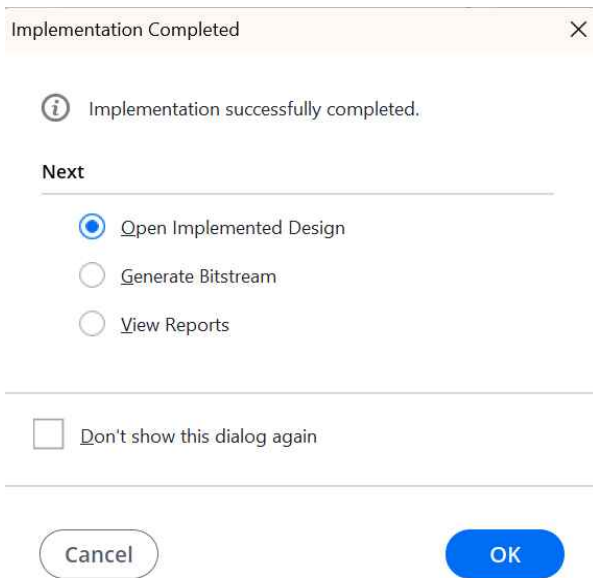
위의 파형을 얻기 위해 시뮬레이션을 행한 결과 아래와 같은 Tcl Console의 결과값을 얻었다.

```
LAB2_PASS clock_divider checks=129
$finish called at time : 436 ns : File "C:/Users/young/Documents/fpga_lab2/circuits/02_clock_divider/sim/tb_clock_divider.sv" Line 19
xsim: Time (s): cpu = 00:00:06 ; elapsed = 00:00:06 . Memory (MB): peak = 1766.805 ; gain = 4.641
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_clock_divider_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
} launch_simulation: Time (s): cpu = 00:00:07 ; elapsed = 00:00:10 . Memory (MB): peak = 1766.805 ; gain = 4.641
```

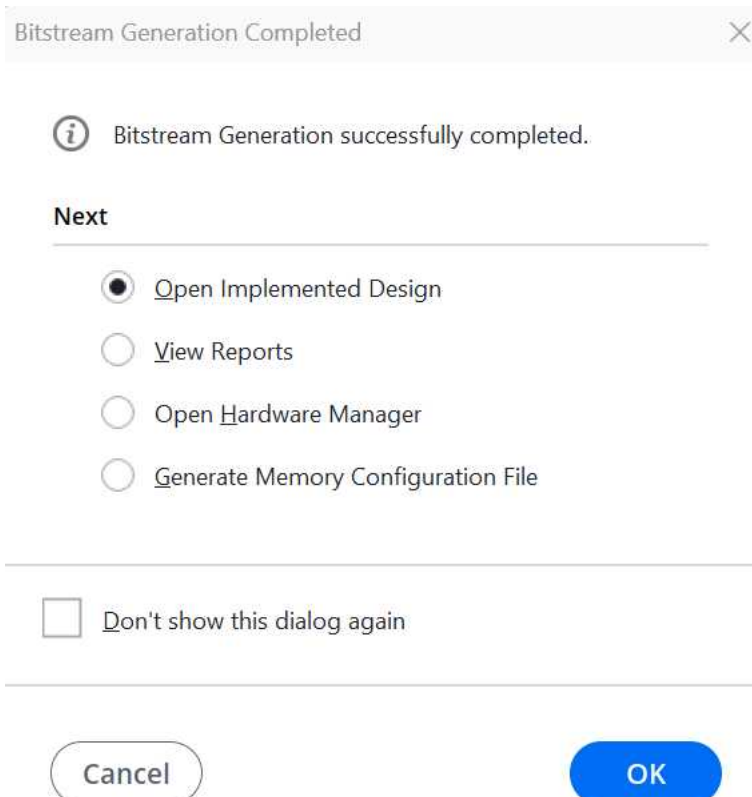
정상적으로 129개의 경우를 연산하였고, 출력시간 또한 436ns로 기록되었다.

이후 Run synthesis와 Run Implementation을 진행하였다.





두 결과 모두 성공하였고, 이후 generate bitstream을 진행하였다.



최종적으로 아래와 같은 결과를 오류 없이 얻었다.

Tcl Console Messages Log Reports Design Runs													
Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggest
✓ synth_1	constrs_1	synth_design Complete!											
✓ impl_1	constrs_1	write_bitstream Complete!	999996.313	0.000	0.122	0.000		0.000	0.103	0	8 Warn		

사용한 커밋은 이후 차례 <증빙 및 소스 commit 링크>에 주소로 기록하였다.

- Bit 해시:

SHA-256:

E99CF49A28ACF1DFF867C45F74A223CBCCEBDB68C81
3A1B544283C25B9C3A72B

- Bit 경로

Bitstream:

circuits\W02_clock_divider\Wlab2_clock_divider.runs\Wimpl_1
Wlab2_clock_divider.bit

Tcl Console × Messages × Log × Reports × Design Runs × DRC × Methodology × Power × Timing ×		
1 Warning Hide All		
Name	Severity	Details
All Violations (1)		
Pin Planning (1)		
CFGBVS-1 (1)		
CFGBVS #1	Warning	Neither the CFGBVS nor CONFIG_VOLTAGE voltage property is set in the current_design. Configuration bank voltage select (CFGBVS) must be set to VCCO or GND, and CONFIG_VOLTAGE must be set to the correct configuration voltage, in order to determine the I/O voltage support for the pins in bank 0. It is suggested to specify these either using the 'Edit Device Properties' function in the GUI or directly in the XDC file using the following syntax: <pre>set_property CFGBVS value1 [current_design] #where value1 is either VCCO or GND set_property CONFIG_VOLTAGE value2 [current_design] #where value2 is the voltage provided to configuration bank 0</pre> Refer to the device configuration user guide for more information.

- DRC 검사 결과 Error는 0건이었으며, CFGBVS-1 Warning 1건이 발생하였다.

해당 경고는 CFGBVS 및 CONFIG_VOLTAGE 속성이 설정되지 않아 발생한 것으로, 실제 보드 구성 전압과 대조가 필요한 항목이다.

● Timing 결과 검증

Implemented Design - xc7s75fgga484-1

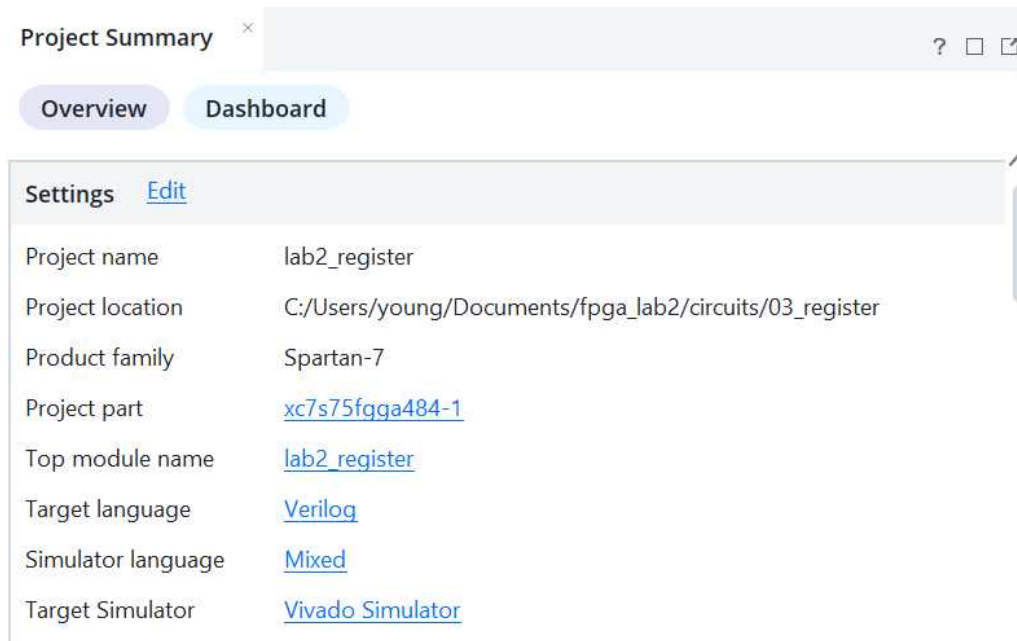
Tcl Console	Messages	Log	Reports	Design Runs	DRC	Methodology	Power	Timing
Design Timing Summary								
General Information Timer Settings Design Timing Summary Clock Summary (1) Methodology Summary (8) > Check Timing (6) > Intra-Clock Paths Inter-Clock Paths Other Path Groups > User Ignored Paths > Unconstrained Paths								
Setup			Hold			Pulse Width		
Worst Negative Slack (WNS) 999996.312 ns			Worst Hold Slack (WHS) 0.122 ns			Worst Pulse Width Slack (WPWS) 49995		
Total Negative Slack (TNS) 0.000 ns			Total Hold Slack (THS) 0.000 ns			Total Pulse Width Negative Slack (TPWS) 0.000		
Number of Failing Endpoints 0			Number of Failing Endpoints 0			Number of Failing Endpoints 0		
Total Number of Endpoints 51			Total Number of Endpoints 51			Total Number of Endpoints 27		
All user specified timing constraints are met.								

Vivado Implementation 후 Timing Summary를 확인한 결과, Worst Negative Slack(WNS)은 999996.312 ns, Total Negative Slack(TNS)은 0.000 ns로 나타났으며 Setup Timing의 Failing Endpoint는 0개였다. 또한 Worst Hold Slack(WHS)은 0.122 ns, Total Hold Slack(THS)은 0.000 ns이며 Hold Timing의 Failing Endpoint 역시 0개였다.

따라서 Setup 및 Hold Timing에서 모두 타이밍 위반이 발생하지 않았으며, Vivado의 "All user specified timing constraints are met." 메시지를 통해 본 회로가 설정된 타이밍 제약을 모두 만족함을 확인하였다.

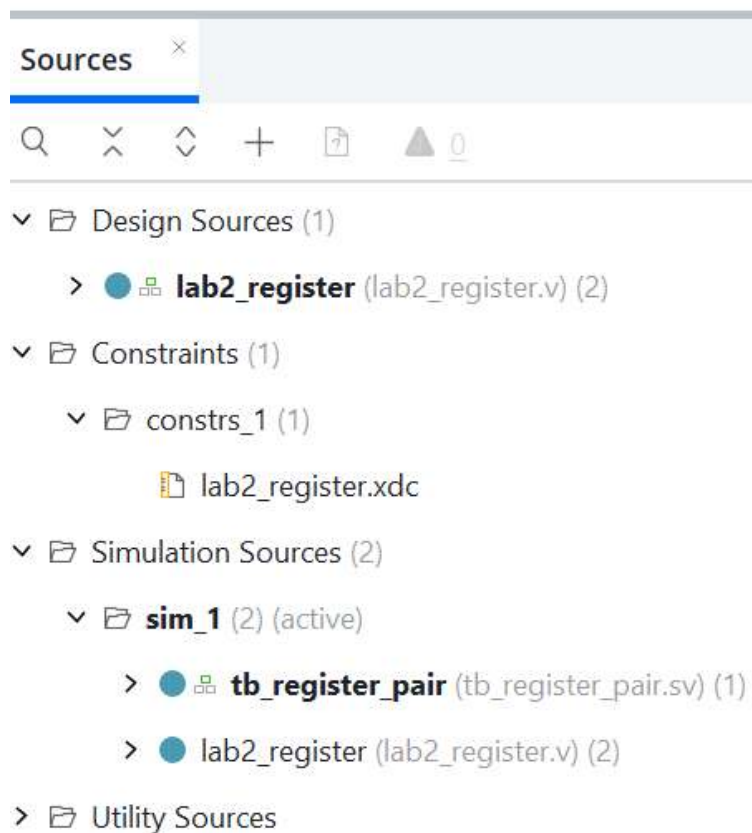
8.3. Register

- Vivado 내의 settings는 다음과 같다.

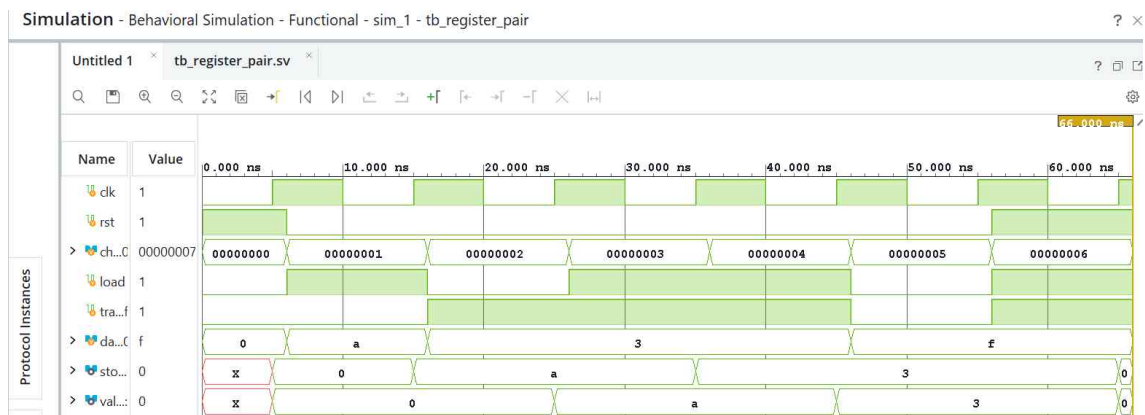


첨부한 사진들과 같이 Top설정 또한 알맞게 진행하였다.

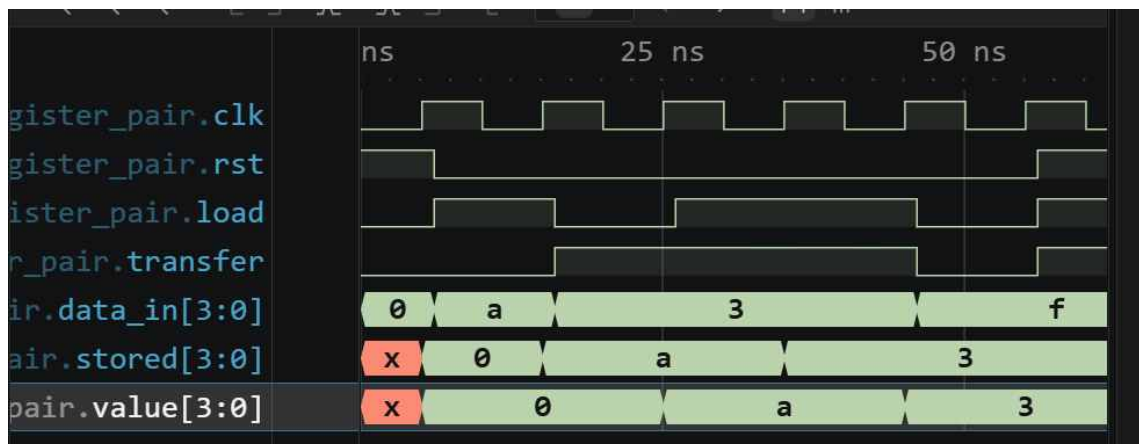
Project Manager - lab2_register



이를 Run simulation 하여 얻은 파형이다.



이렇게 얻은 파형이 올바른지 확인하기 위해 위에서 작성한 vscode 내에서의 시뮬레이션 결과와 비교하였다.



이론에 부합한 결과는 위에서 설명하였고, 그와 vscode의 파형이 일치하였기에 vscode에서의 파형이 올바른 파형이라고 할 수 있다.

이를 vivado파형과 비교해보면, 정확히 같은 파형의 결과를 얻은 것을 알 수 있다.

즉, 문제 없이 알맞은 파형을 얻었다.

위의 파형을 얻기 위해 시뮬레이션을 행한 결과 아래와 같은 Tcl Console의 결과값을 얻었다.

```
LAB2_PASS register_pair checks=7
$finish called at time : 66 ns : File "C:/Users/young/Documents/fpga_lab2/circuits/03_register/sim/tb_register_pair.sv" Line 19
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_register_pair_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:04 ; elapsed = 00:00:09 . Memory (MB): peak = 1803.633 ; gain = 0.000
,
```

그 결과 정상적으로 7개의 경우를 연산하였고, 출력시간 또한 66ns로 기록되었다.

이후 Run synthesis와 Run Implementation을
진행하였다.

Synthesis Completed

×

Synthesis successfully completed.

Next

☒ Run Implementation

☐ Open Synthesized Design

☐ View Reports

☐ Don't show this dialog again

Cancel

OK

Implementation Completed

×

Implementation successfully completed.

Next

☒ Open Implemented Design

☐ Generate Bitstream

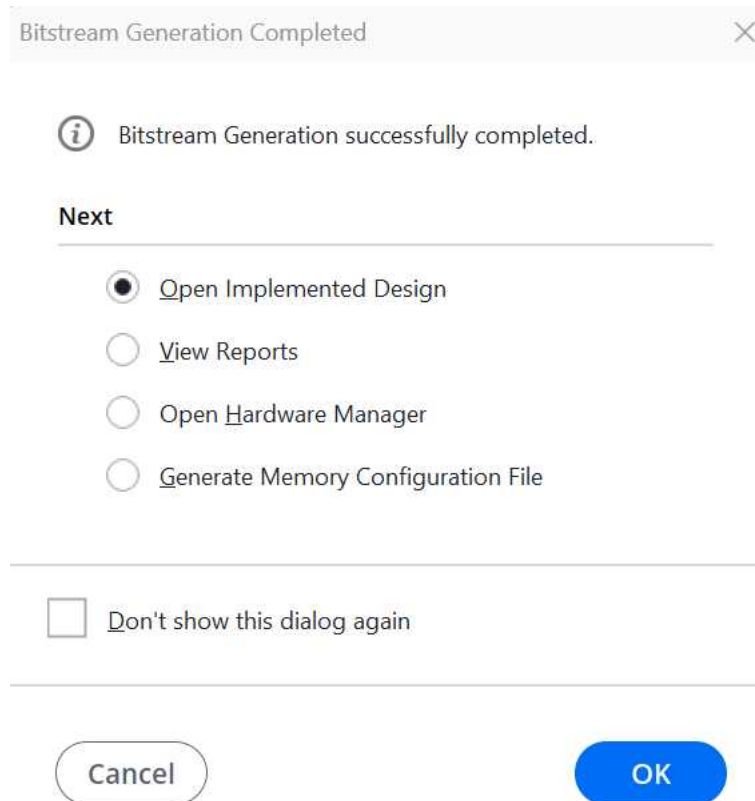
☐ View Reports

☐ Don't show this dialog again

Cancel

OK

두 결과 모두 성공하였고, 이후 generate bitstream을 진행하였다.



최종적으로 아래와 같은 결과를 오류 없이 얻었다.

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF
✓ synth_1	constrs_1	synth_design Complete!												10	35
✓ impl_1	constrs_1	write_bitstream Complete!	9999%	0.00%	0.119	0.00%		0.000	0.103	0	8 Warn			11	35

사용한 커밋은 이후 차례 <증빙 및 소스 commit 링크>에 주소로 기록하였다.

- Bit 해시:

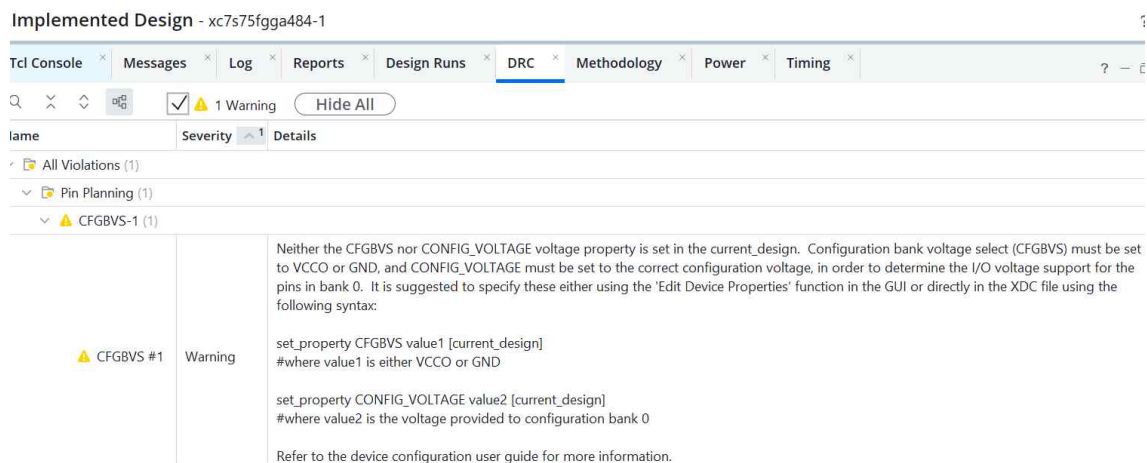
SHA-256:

FC267281E64185B9A600284CB49E87872E7ECCAC9C855
82736654E0711F1B3AE

- Bit 경로

Bitstream:

circuitsW03_registerWlab2_register.runsWimpl_1Wlab2_regis
ter.bit



- DRC 검사 결과 Error는 0건이었으며, CFGBVS-1 Warning 1건이 발생하였다.

해당 경고는 CFGBVS 및 CONFIG_VOLTAGE 속성이
설정되지 않아 발생한 것으로, 실제 보드 구성 전압과
대조가 필요한 항목이다.

● Timing 결과 검증

Implemented Design - xc7s75fgga484-1

Tcl ConsoleMessagesLogReportsDesign RunsDRCMethodologyPowerTiming

Design Timing Summary

General Information

Timer Settings

Design Timing Summary

Clock Summary (1)

Methodology Summary (8)

Check Timing (16)

Intra-Clock Paths

Inter-Clock Paths

Other Path Groups

User Ignored Paths

Unconstrained Paths

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS)	999997.812 ns	Worst Hold Slack (WHS)	0.119 ns	Worst Pulse Width Slack (WPWS)	49999
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns	Total Pulse Width Negative Slack (TPWS)	0.000
Number of Failing Endpoints	0	Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	72	Total Number of Endpoints	72	Total Number of Endpoints	36

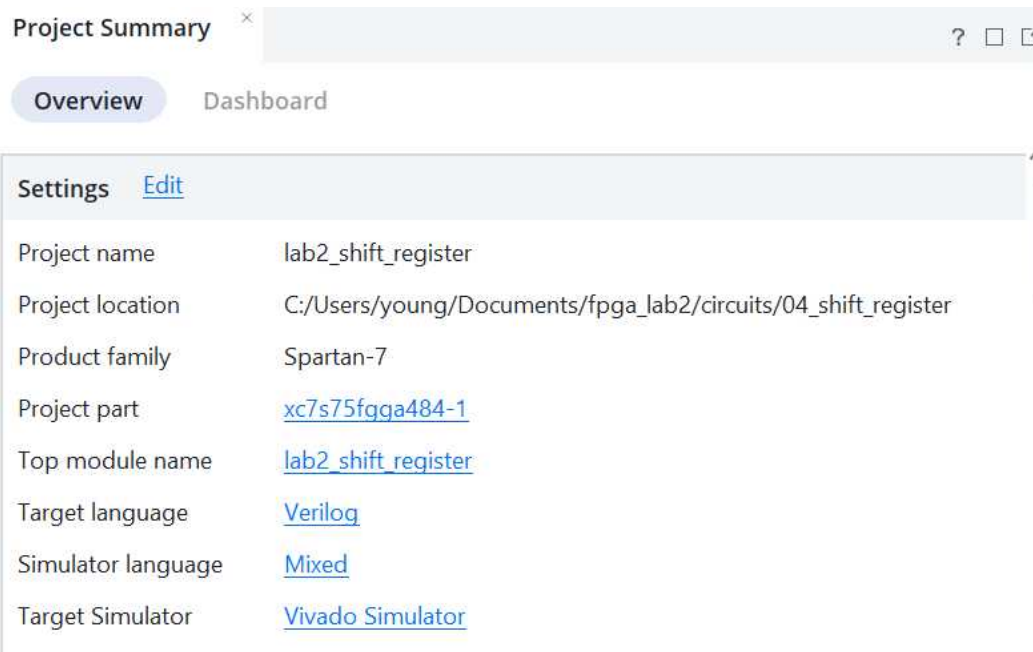
All user specified timing constraints are met.

Vivado Implementation 후 Timing Summary를 확인한 결과, Worst Negative Slack(WNS)은 999997.812 ns, Total Negative Slack(TNS)은 0.000 ns로 나타났으며 Setup Timing의 Failing Endpoint는 0개였다. 또한 Worst Hold Slack(WHS)은 0.119 ns, Total Hold Slack(THS)은 0.000 ns이며 Hold Timing의 Failing Endpoint 역시 0개였다. Pulse Width 항목에서도 Failing Endpoint가 0개로 확인되었다.

따라서 Setup, Hold 및 Pulse Width Timing에서 타이밍 위반이 발생하지 않았으며, Vivado의 "All user specified timing constraints are met." 메시지를 통해 본 설계가 설정된 타이밍 제약을 모두 만족함을 확인하였다.

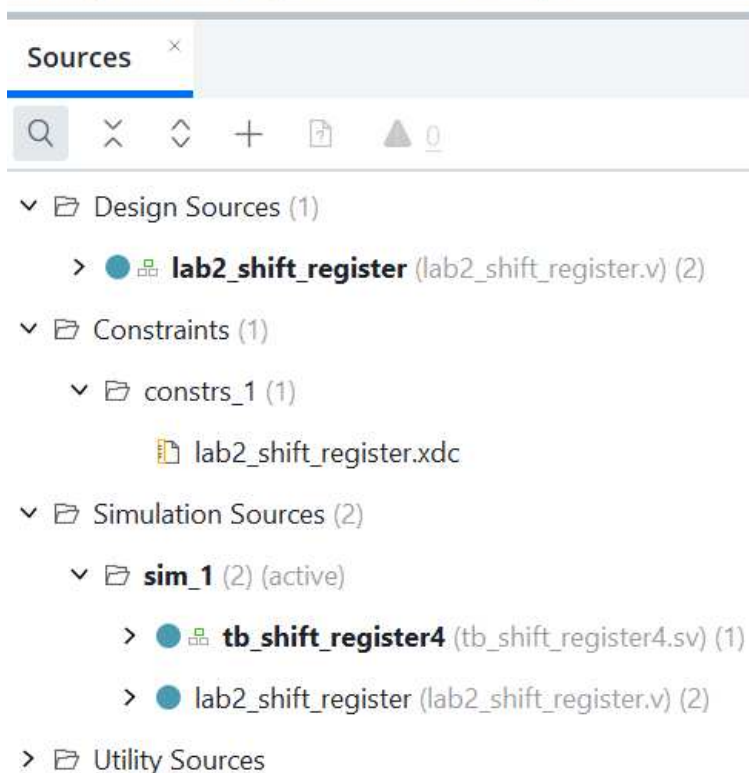
8.4. Shift register

- Vivado 내의 settings는 다음과 같다.

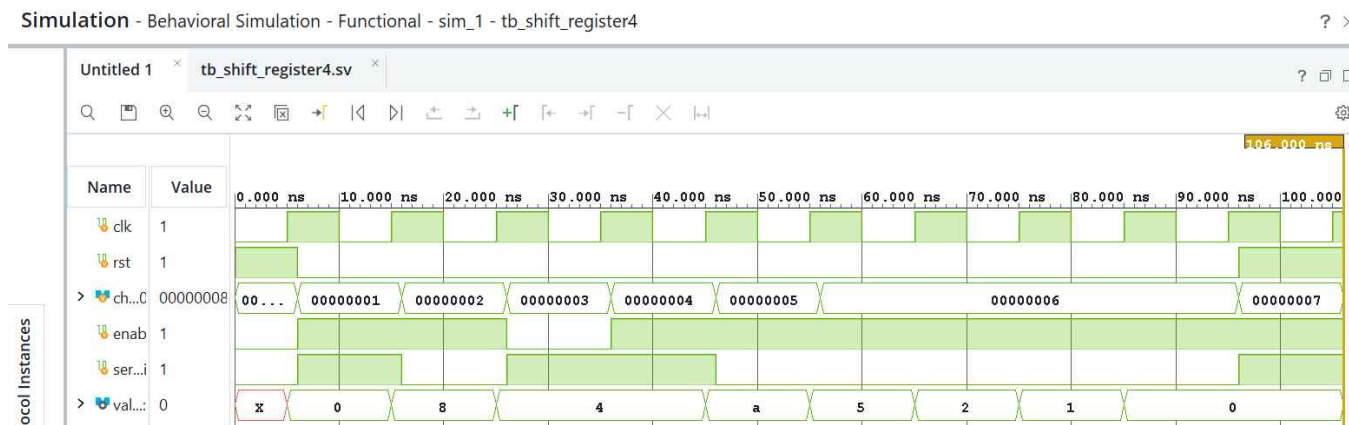


첨부한 사진들과 같이 Top설정 또한 알맞게 진행하였다.

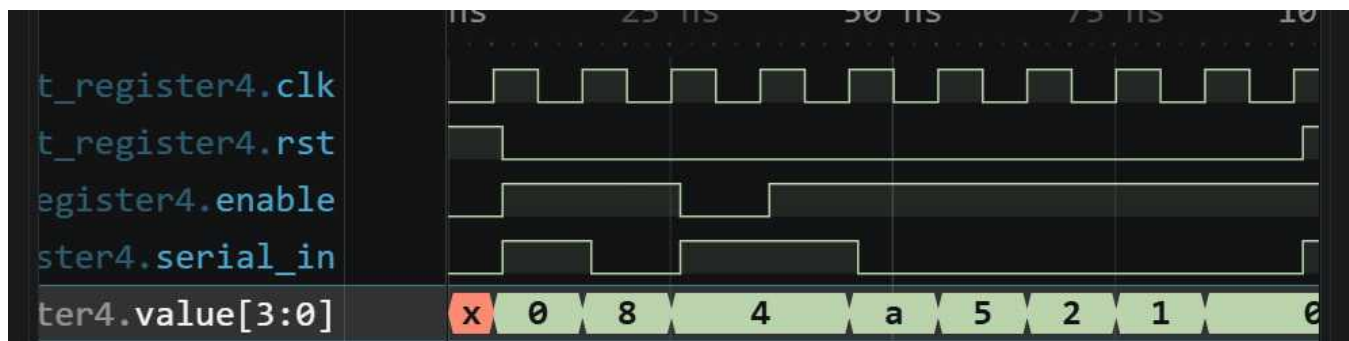
Project Manager - lab2_shift_register



이를 Run simulation 하여 얻은 파형이다.



이렇게 얻은 파형이 올바른지 확인하기 위해 위에서 작성한 vscode 내에서의 시뮬레이션 결과와 비교하였다.



이론에 부합한 결과는 위에서 설명하였고, 그와 vscode의 파형이 일치하였기에 vscode에서의 파형이 올바른 파형이라고 할 수 있다.

이를 vivado파형과 비교해보면, 정확히 같은 파형의 결과를 얻은 것을 알 수 있다.

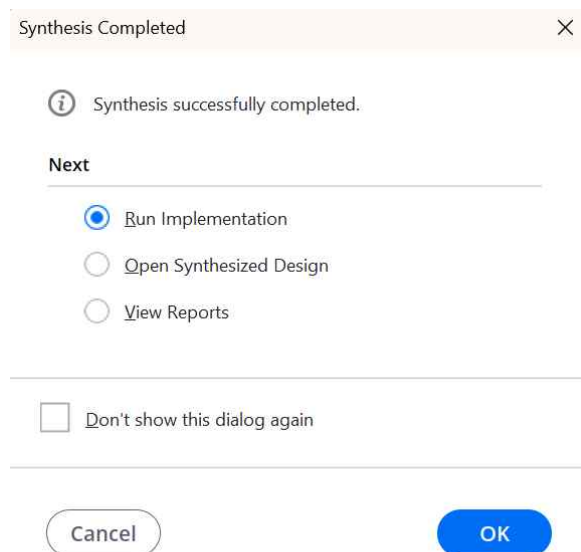
즉, 문제 없이 알맞은 파형을 얻었다.

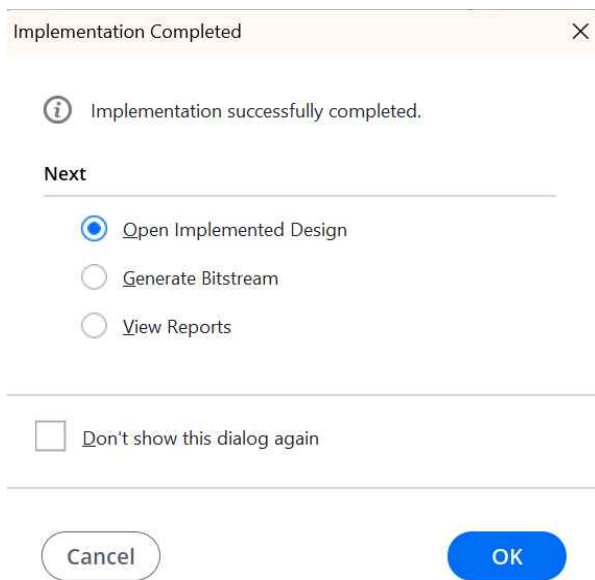
위의 파형을 얻기 위해 시뮬레이션을 행한 결과 아래와 같은 Tcl Console의 결과값을 얻었다.

```
LAB2_PASS shift_register4 checks=8
$finish called at time : 106 ns : File "C:/Users/young/Documents/fpga_lab2/circuits/04_shift_register/sim/tb_shift_register4.sv" Line 30
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_shift_register4_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
| launch_simulation: Time (s): cpu = 00:00:05 ; elapsed = 00:00:08 . Memory (MB): peak = 1756.113 ; gain = 14.883
```

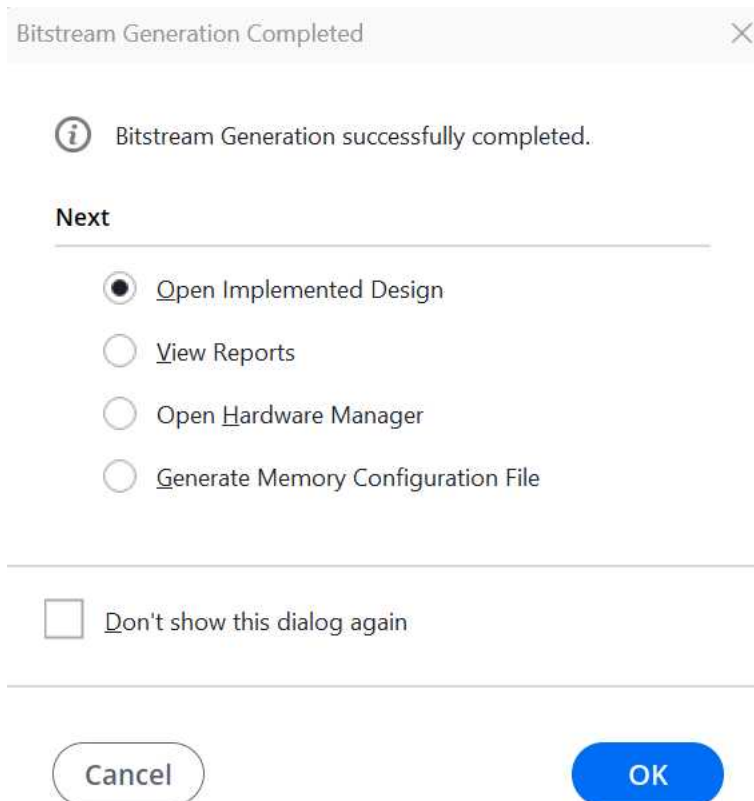
이번에는 정상적으로 8개의 경우를 연산하였고, 출력시간 또한 106ns로 기록되었다.

이후 Run synthesis와 Run Implementation을 진행하였다.





두 결과 모두 성공하였고, 이후 generate bitstream을 진행하였다.



최종적으로 아래와 같은 결과를 오류 없이 얻었다.

Tcl Console × Messages × Log × Reports × Design Runs × ? - □ ▢															
Q × ⬆ ⬅ ⬇ ➡ ➤ ➥ + %															
Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF
✓ synth_1	constrs_1	synth_design Complete!												10	31
✓ impl_1	constrs_1	write_bitstream Complete!	9999%	0.00%	0.122	0.00%		0.000	0.103	0	8 Warn			11	17

사용한 커밋은 이후 차례 <증빙 및 소스 commit 링크>에 주소로 기록하였다.

- Bit 해시:

SHA-256:

686627DDE0FA0A1728FBDE01C65F8CA9CEACEBF81629
FE2B7B051C3D2D12CD95

- Bit 경로

circuits\04_shift_register\lab2_shift_register.runs\
impl_1\lab2_shift_register.bit

Implemented Design - xc7s75fgga484-1

Tcl Console Messages Log Reports Design Runs DRC Methodology Power Timing

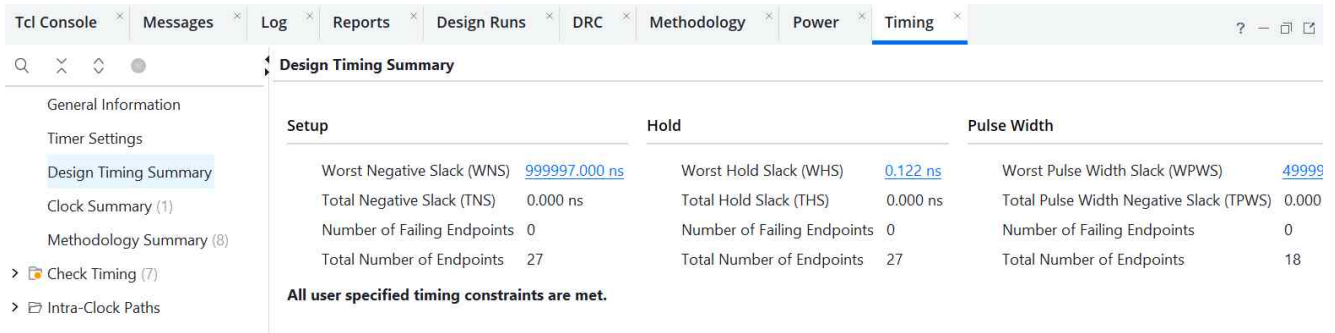
1 Warning Hide All

Name	Severity	Details
All Violations (1)		
Pin Planning (1)		
CFGBVS-1	Warning	Neither the CFGBVS nor CONFIG_VOLTAGE voltage property is set in the current_design. Configuration bank voltage select (CFGBVS) must be set to VCCO or GND, and CONFIG_VOLTAGE must be set to the correct configuration voltage, in order to determine the I/O voltage support for the pins in bank 0. It is suggested to specify these either using the 'Edit Device Properties' function in the GUI or directly in the XDC file using the following syntax: <pre>set_property CFGBVS value1 [current_design] #where value1 is either VCCO or GND set_property CONFIG_VOLTAGE value2 [current_design] #where value2 is the voltage provided to configuration bank 0</pre> Refer to the device configuration user guide for more information.

- DRC 검사 결과 Error는 0건이었으며, CFGBVS-1 Warning 1건이 발생하였다.

해당 경고는 CFGBVS 및 CONFIG_VOLTAGE 속성이 설정되지 않아 발생한 것으로, 실제 보드 구성 전압과 대조가 필요한 항목이다.

● Timing 결과 검증



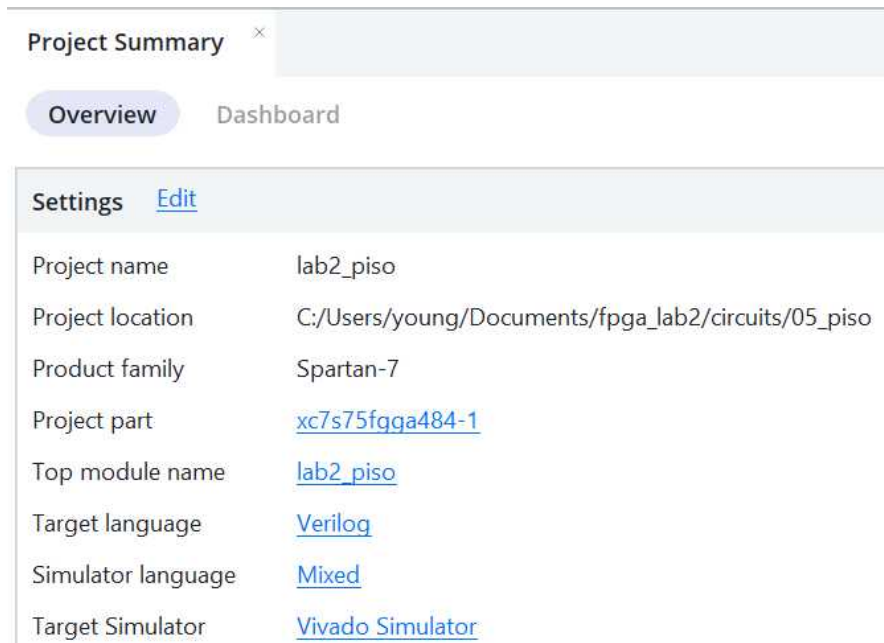
Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS)	999997.000 ns	Worst Hold Slack (WHS)	0.122 ns	Worst Pulse Width Slack (WPWS)	49999
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns	Total Pulse Width Negative Slack (TPWS)	0.000
Number of Failing Endpoints	0	Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	27	Total Number of Endpoints	27	Total Number of Endpoints	18
All user specified timing constraints are met.					

Vivado Implementation 후 Timing Summary를 확인한 결과, Worst Negative Slack(WNS)은 999997.000 ns, Total Negative Slack(TNS)은 0.000 ns이며 Setup Timing의 Failing Endpoint는 0개였다. 또한 Worst Hold Slack(WHS)은 0.122 ns, Total Hold Slack(THS)은 0.000 ns로 나타났으며 Hold Timing의 Failing Endpoint 역시 0개였다. Pulse Width 항목에서도 Failing Endpoint가 0개로 확인되었다.

따라서 Setup, Hold 및 Pulse Width Timing에서 타이밍 위반이 발생하지 않았으며, Vivado의 "All user specified timing constraints are met." 메시지를 통해 본 설계가 설정된 타이밍 제약을 모두 만족함을 확인하였다.

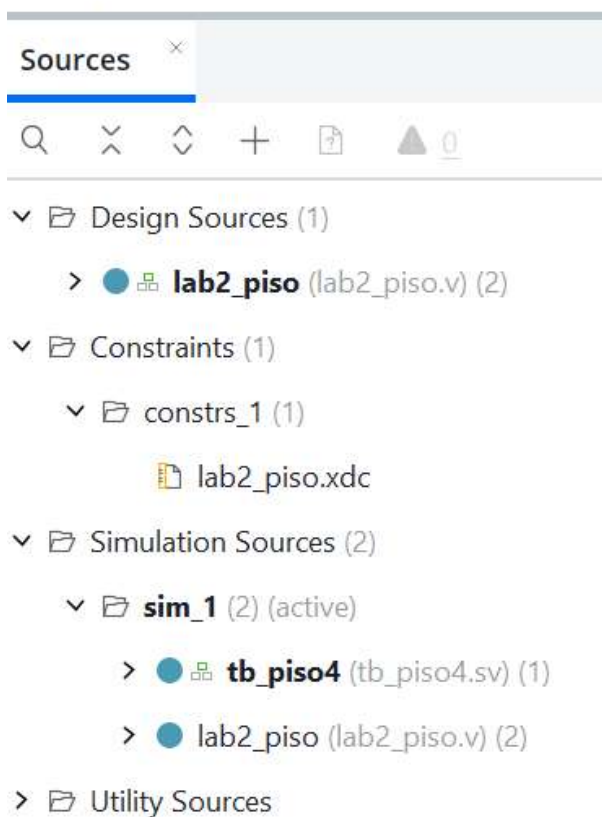
8.5. PISO

- Vivado 내의 settings는 다음과 같다.

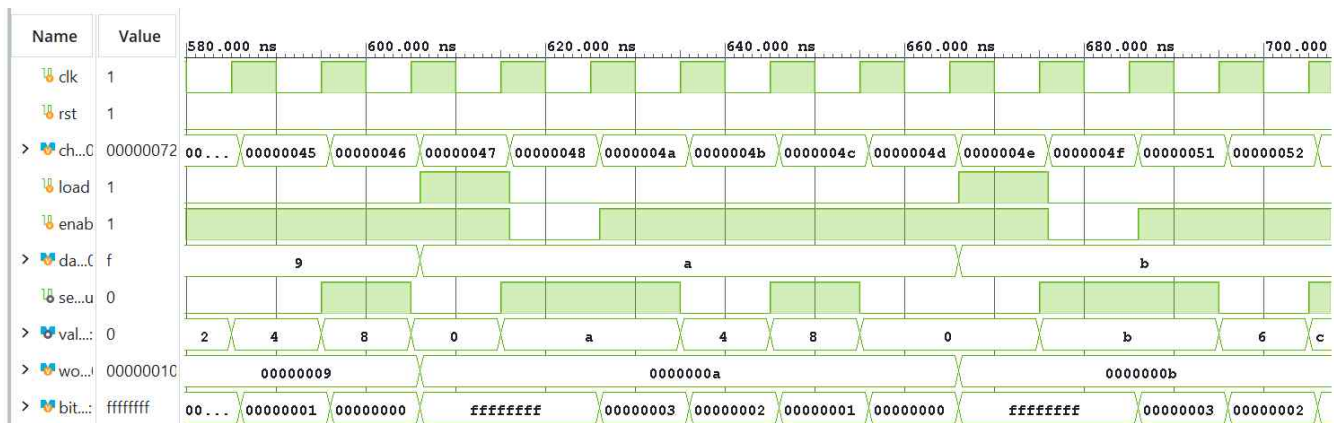


첨부한 사진들과 같이 Top설정 또한 알맞게 진행하였다.

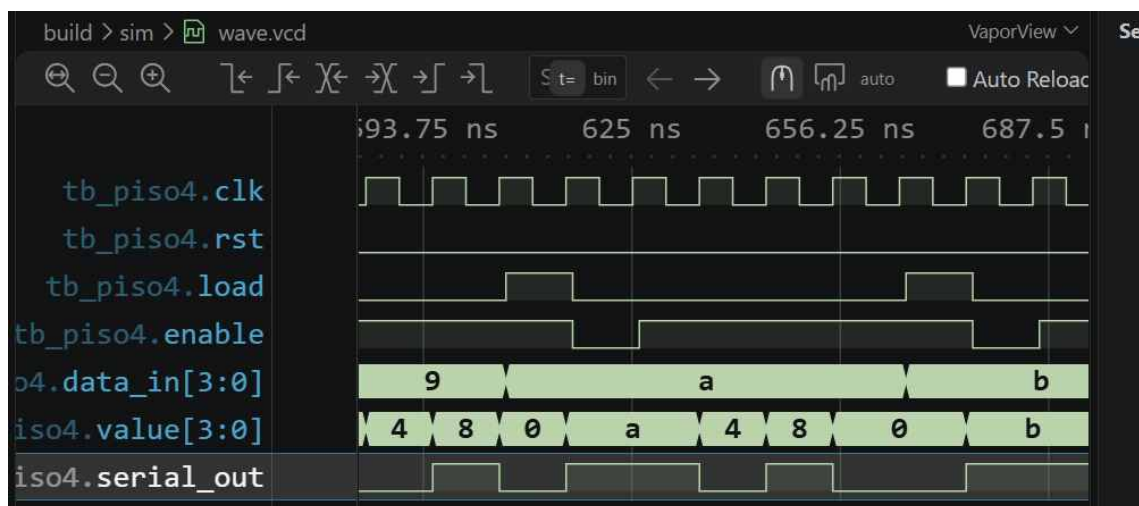
Project Manager - lab2_piso



이를 Run simulation 하여 얻은 파형이다.



이렇게 얻은 파형이 올바른지 확인하기 위해 위에서 작성한 vscode 내에서의 시뮬레이션 결과와 비교하였다.



이론에 부합한 결과는 위에서 설명하였고, 그와 vscode의 파형이 일치하였기에 vscode에서의 파형이 올바른 파형이라고 할 수 있다.

이를 vivado파형과 비교해보면, 정확히 같은 파형의 결과를 얻은 것을 알 수 있다.

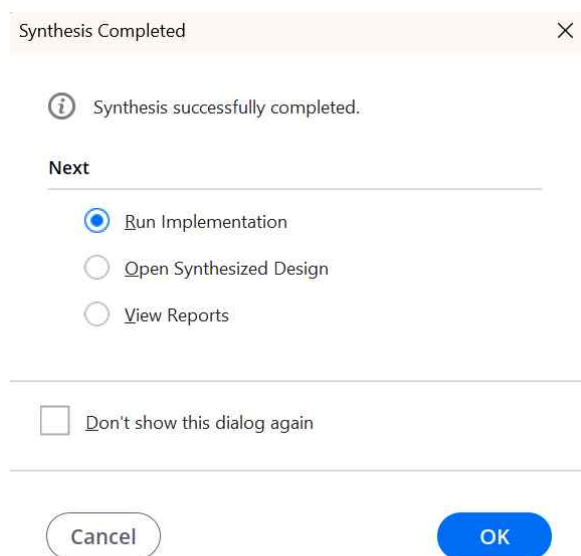
즉, 문제 없이 알맞은 파형을 얻었다.

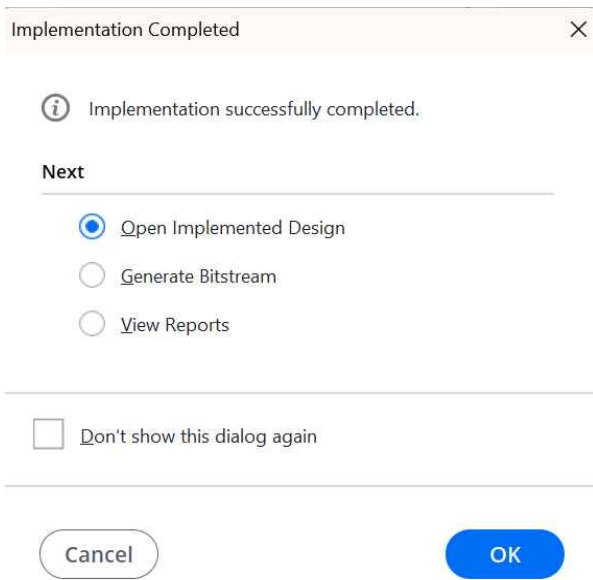
위의 파형을 얻기 위해 시뮬레이션을 행한 결과 아래와 같은 Tcl Console의 결과값을 얻었다.

```
LAB2_PASS piso4 checks=114
$finish called at time : 976 ns : File "C:/Users/young/Documents/fpga_lab2/circuits/05_piso/sim/tb_piso4.sv" Line 30
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_piso4_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:07 ; elapsed = 00:00:08 . Memory (MB): peak = 1754.770 ; gain = 0.000
```

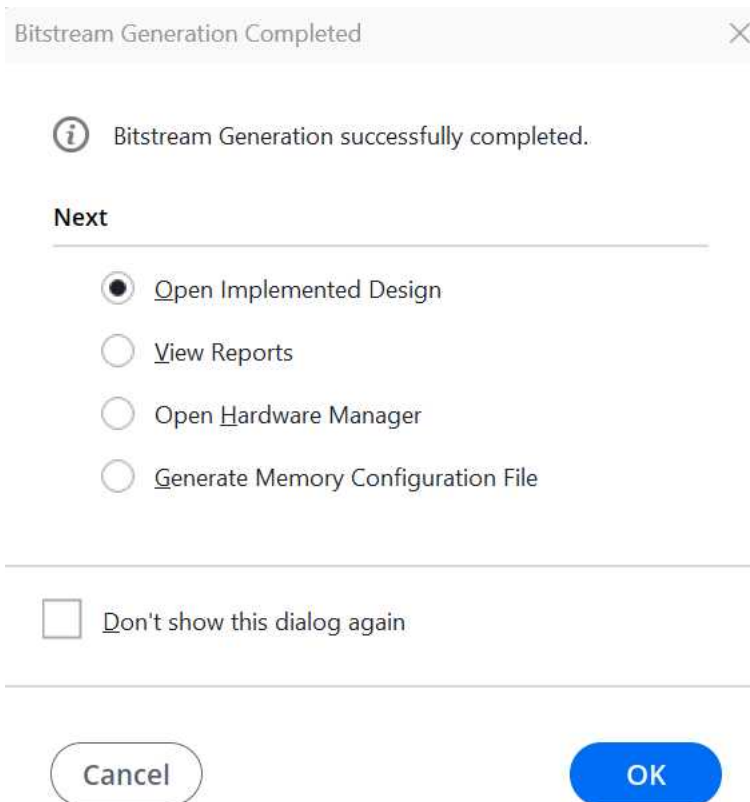
이번에는 정상적으로 114개의 경우를 연산하였고, 출력시간 또한 976ns로 기록되었다.

이후 Run synthesis와 Run Implementation을 진행하였다.





두 결과 모두 성공하였고, 이후 generate bitstream을 진행하였다.



최종적으로 아래와 같은 결과를 오류 없이 얻었다.

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF
✓ synth_1	constrs_1	synth_design Complete!												12	31
✓ impl_1	constrs_1	write_bitstream Complete!	9999%	0.00%	0.121	0.00%		0.000	0.103	0	8 Warn			13	26

사용한 커밋은 이후 차례 <증빙 및 소스 commit 링크>에 주소로 기록하였다.

- Bit 해시:

SHA-256:

763B16E1EAF0026301A2001F461F798F3DC226125899C7
0B5EC2337B81D2F1E8

- Bit 경로

circuits\05_piso\lab2_piso.runs\impl_1\lab2_piso.
bit

Name	Severity	Details
All Violations (1)		
Pin Planning (1)		
CFGBVS-1 (1)		
CFGBVS #1	Warning	Neither the CFGBVS nor CONFIG_VOLTAGE voltage property is set in the current_design. Configuration bank voltage select (CFGBVS) must be set to VCCO or GND, and CONFIG_VOLTAGE must be set to the correct configuration voltage, in order to determine the I/O voltage support for the pins in bank 0. It is suggested to specify these either using the 'Edit Device Properties' function in the GUI or directly in the XDC file using the following syntax: <pre>set_property CFGBVS value1 [current_design] #where value1 is either VCCO or GND</pre> <pre>set_property CONFIG_VOLTAGE value2 [current_design] #where value2 is the voltage provided to configuration bank 0</pre> Refer to the device configuration user guide for more information.

- DRC 검사 결과 Error는 0건이었으며, CFGBVS-1 Warning 1건이 발생하였다.

해당 경고는 CFGBVS 및 CONFIG_VOLTAGE 속성이 설정되지 않아 발생한 것으로, 실제 보드 구성 전압과 대조가 필요한 항목이다.

● Timing 결과 검증

Setup	Hold	Pulse Width
Worst Negative Slack (WNS) 999997.500 ns	Worst Hold Slack (WHS) 0.121 ns	Worst Pulse Width Slack (WPWS) 499997.500 ns
Total Negative Slack (TNS) 0.000 ns	Total Hold Slack (THS) 0.000 ns	Total Pulse Width Negative Slack (TPWS) 0.000 ns
Number of Failing Endpoints 0	Number of Failing Endpoints 0	Number of Failing Endpoints 0
Total Number of Endpoints 42	Total Number of Endpoints 42	Total Number of Endpoints 27

All user specified timing constraints are met.

Vivado Implementation 후 Timing Summary를 확인한 결과, Worst Negative Slack(WNS)은 999997.500 ns, Total Negative Slack(TNS)은 0.000 ns이며 Setup Timing의 Failing Endpoint는 0개였다. 또한 Worst Hold Slack(WHS)은 0.121 ns, Total Hold Slack(THS)은 0.000 ns로 나타났으며 Hold Timing의 Failing Endpoint 역시 0개였다. Pulse Width 항목에서도 Failing Endpoint가 0개로 확인되었다.

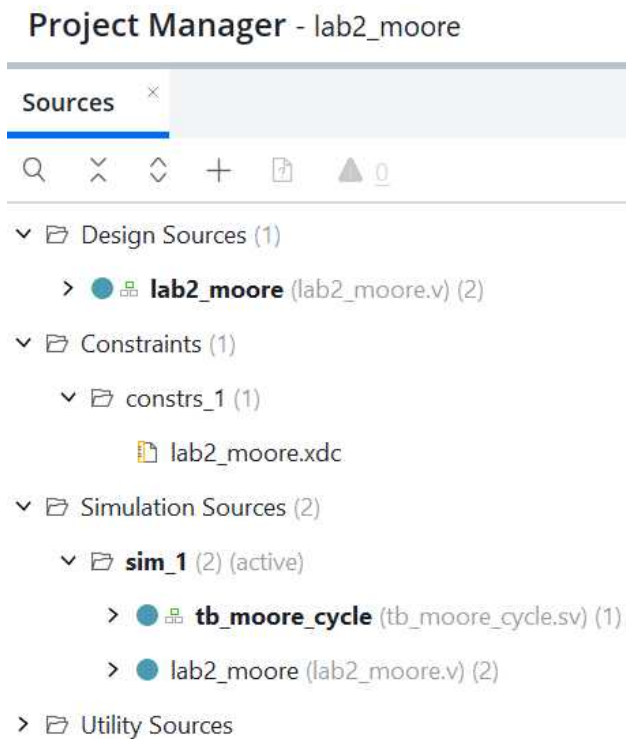
따라서 Setup, Hold 및 Pulse Width Timing에서 타이밍 위반이 발생하지 않았으며, Vivado의 "All user specified timing constraints are met." 메시지를 통해 본 설계가 설정된 타이밍 제약을 모두 만족함을 확인하였다.

8.6. Moore

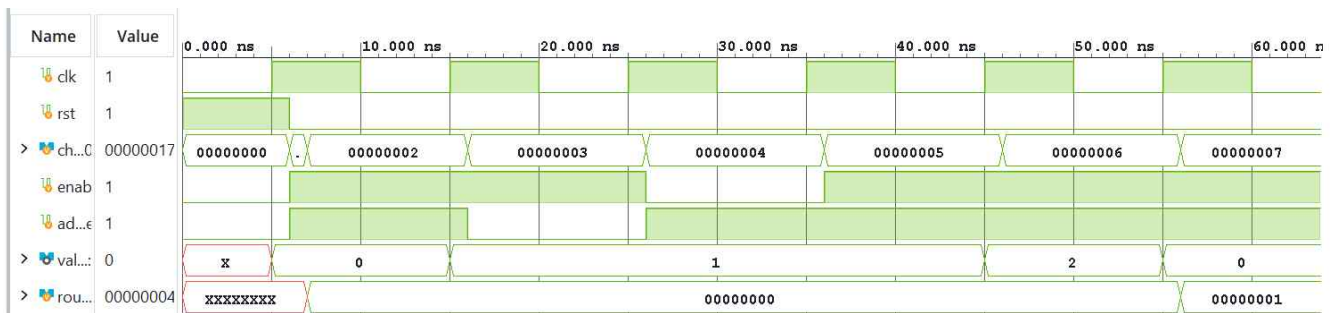
- Vivado 내의 settings는 다음과 같다.

Settings Edit	
Project name	lab2_moore
Project location	C:/Users/young/Documents/fpga_lab2/circuits/06_moore
Product family	Spartan-7
Project part	xc7s75fgga484-1
Top module name	lab2_moore
Target language	Verilog
Simulator language	Mixed
Target Simulator	Vivado Simulator

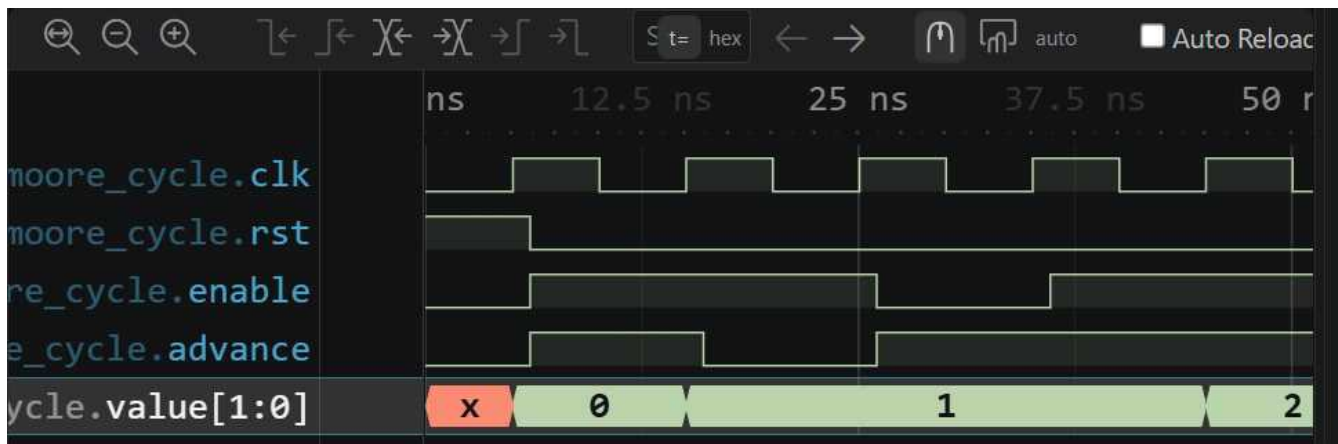
첨부한 사진들과 같이 Top설정 또한 알맞게 진행하였다.



이를 Run simulation 하여 얻은 파형이다.



이렇게 얻은 파형이 올바른지 확인하기 위해 위에서 작성한 vscode 내에서의 시뮬레이션 결과와 비교하였다.



이론에 부합한 결과는 위에서 설명하였고, 그와 vscode의 파형이 일치하였기에 vscode에서의 파형이 올바른 파형이라고 할 수 있다.

이를 vivado파형과 비교해보면, 정확히 같은 파형의 결과를 얻은 것을 알 수 있다.

즉, 문제 없이 알맞은 파형을 얻었다.

위의 파형을 얻기 위해 시뮬레이션을 행한 결과 아래와 같은 Tcl Console의 결과값을 얻었다.

```
LAB2_PASS moore_cycle checks=23
$finish called at time : 226 ns : File "C:/Users/young/Documents/fpga_lab2/circuits/06_moore/sim/tb_moore_cycle.sv" Line 30
xsim: Time (s): cpu = 00:00:06 ; elapsed = 00:00:05 . Memory (MB): peak = 1758.145 ; gain = 5.648
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_moore_cycle_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:07 ; elapsed = 00:00:10 . Memory (MB): peak = 1758.145 ; gain = 5.648
```

그 결과, 정상적으로 23개의 경우를 연산하였고, 출력시간 또한 226ns로 기록되었다.

이후 Run synthesis와 Run Implementation을
진행하였다.

Synthesis Completed X

 Synthesis successfully completed.

Next

☒ Run Implementation

☐ Open Synthesized Design

☐ View Reports

☐ Don't show this dialog again

Cancel OK

Implementation Completed X

 Implementation successfully completed.

Next

☒ Open Implemented Design

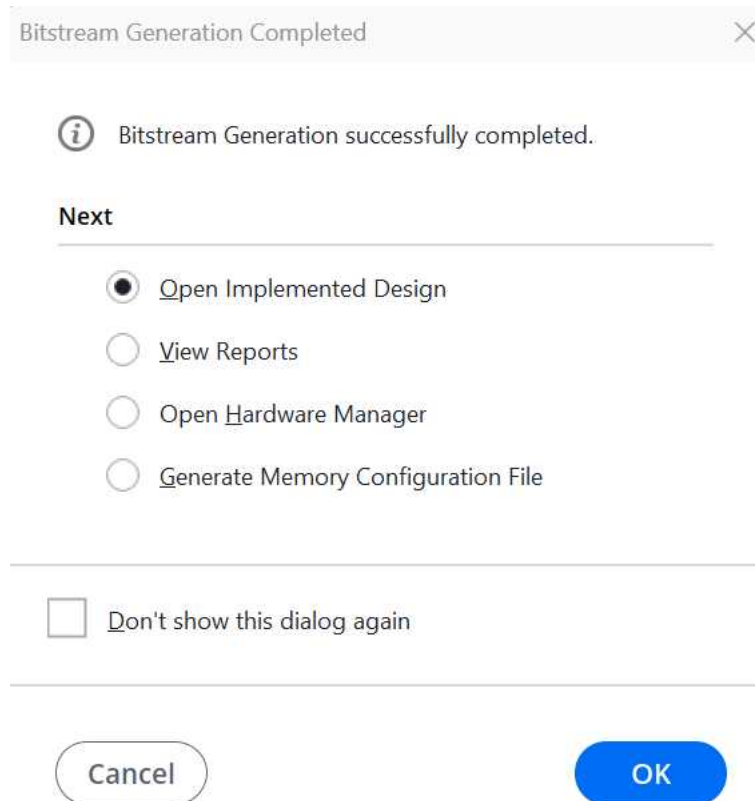
☐ Generate Bitstream

☐ View Reports

☐ Don't show this dialog again

Cancel OK

두 결과 모두 성공하였고, 이후 generate bitstream을 진행하였다.



최종적으로 아래와 같은 결과를 오류 없이 얻었다.

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF
✓ synth_1	constrs_1	synth_design Complete!												11	30
✓ impl_1	constrs_1	write_bitstream Complete!	9999%	0.00%	0.122	0.00%		0.000	0.103	0	8 Warn			12	16

사용한 커밋은 이후 차례 <증빙 및 소스 commit 링크>에 주소로 기록하였다.

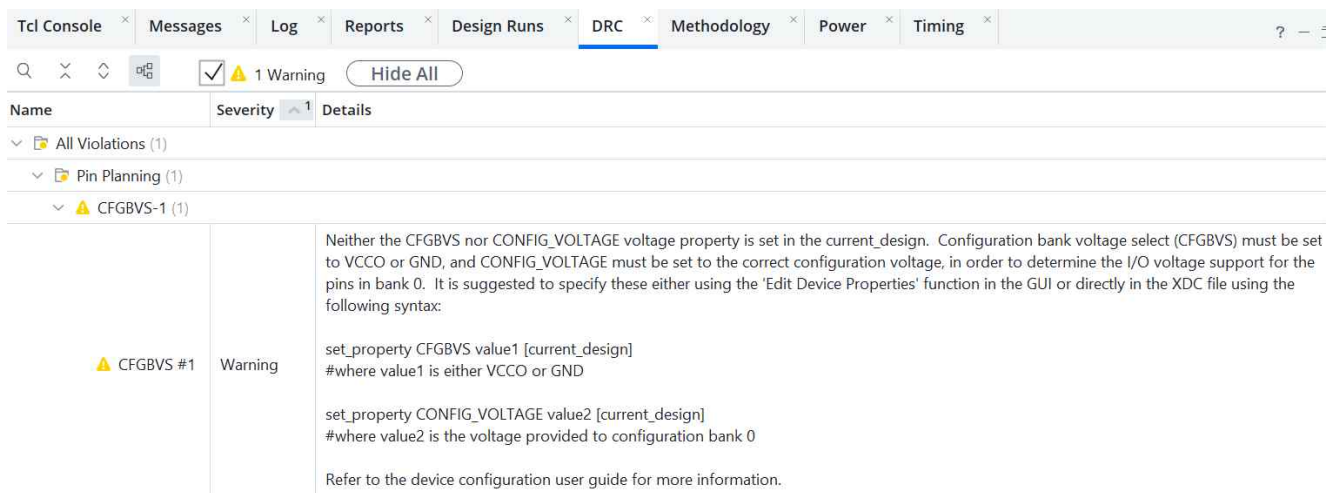
- Bit 해시:

SHA-256:

0DAF25D9D0CE5491D63165A41FDAD90B7B449DE7DA4
0A92FE688A5CF27A3E2DE

- Bit 경로

circuits\06_moore\lab2_moore.runs\impl_1\lab2_
moore.bit

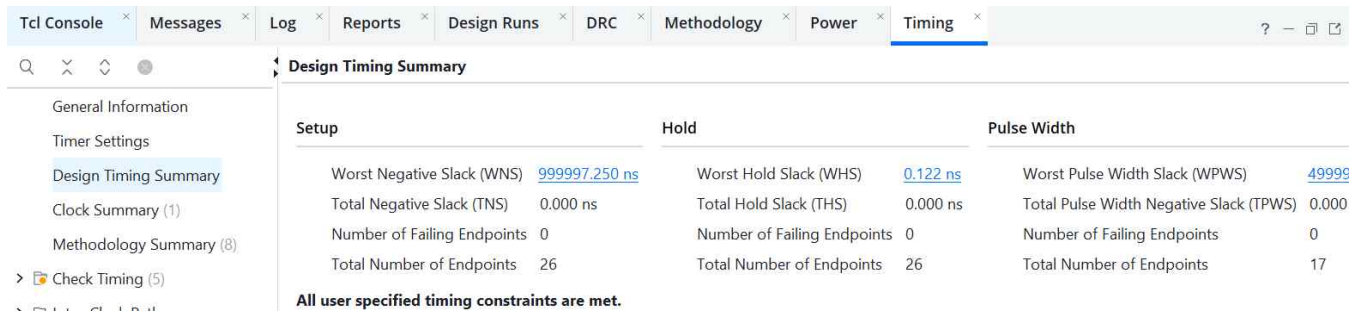


Name	Severity	Details
CFGBVS #1	Warning	Neither the CFGBVS nor CONFIG_VOLTAGE voltage property is set in the current_design. Configuration bank voltage select (CFGBVS) must be set to VCCO or GND, and CONFIG_VOLTAGE must be set to the correct configuration voltage, in order to determine the I/O voltage support for the pins in bank 0. It is suggested to specify these either using the 'Edit Device Properties' function in the GUI or directly in the XDC file using the following syntax: <pre>set_property CFGBVS value1 [current_design] #where value1 is either VCCO or GND set_property CONFIG_VOLTAGE value2 [current_design] #where value2 is the voltage provided to configuration bank 0</pre> Refer to the device configuration user guide for more information.

- DRC 검사 결과 Error는 0건이었으며, CFGBVS-1 Warning 1건이 발생하였다.

해당 경고는 CFGBVS 및 CONFIG_VOLTAGE 속성이 설정되지 않아 발생한 것으로, 실제 보드 구성 전압과 대조가 필요한 항목이다.

● Timing 결과 검증



Setup		Hold		Pulse Width	
Worst Negative Slack (WNS)	999997.250 ns	Worst Hold Slack (WHS)	0.122 ns	Worst Pulse Width Slack (WPWS)	49999 ns
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns	Total Pulse Width Negative Slack (TPWS)	0.000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	26	Total Number of Endpoints	26	Total Number of Endpoints	17

All user specified timing constraints are met.

Vivado Implementation 후 Timing Summary를 확인한 결과, Worst Negative Slack(WNS)은 999997.250 ns, Total Negative Slack(TNS)은 0.000 ns이며 Setup Timing의 Failing Endpoint는 0개였다. 또한 Worst Hold Slack(WHS)은 0.122 ns, Total Hold Slack(THS)은 0.000 ns로 나타났으며 Hold Timing의 Failing Endpoint 역시 0개였다. Pulse Width 항목에서도 Failing Endpoint가 0개로 확인되었다.

따라서 Setup, Hold 및 Pulse Width Timing에서 타이밍 위반이 발생하지 않았으며, Vivado의 "All user specified timing constraints are met." 메시지를 통해 본 설계가 설정된 타이밍 제약을 모두 만족함을 확인하였다.

8.7. Mealy

- Vivado 내의 settings는 다음과 같다.

Project Summary ×

Overview Dashboard

Settings [Edit](#)

Project name	lab2_mealy
Project location	C:/Users/young/Documents/fpga_lab2/circuits/07_mealy
Product family	Spartan-7
Project part	xc7s75fgga484-1
Top module name	lab2_mealy
Target language	Verilog
Simulator language	Mixed
Target Simulator	Vivado Simulator

첨부한 사진들과 같이 Top설정 또한 알맞게 진행하였다.

Sources ×

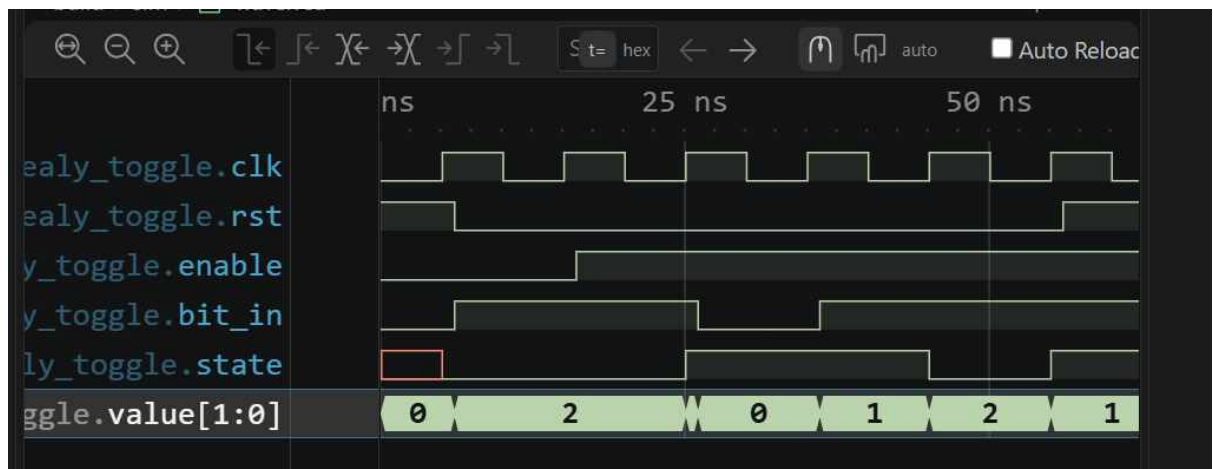
🔍 ✖ ⬆ + 📁 ⬆ 0

- ▼ 📁 Design Sources (1)
 - > ● 📁 **lab2_mealy** (lab2_mealy.v) (2)
- ▼ 📁 Constraints (1)
 - ▼ 📁 constrs_1 (1)
 - 📄 lab2_mealy.xdc
- ▼ 📁 Simulation Sources (2)
 - ▼ 📁 **sim_1** (2) (active)
 - > ● 📁 **tb_mealy_toggle** (tb_mealy_toggle.sv) (1)
 - > ● lab2_mealy (lab2_mealy.v) (2)
- > 📁 Utility Sources

이를 Run simulation 하여 얻은 파형이다.



이렇게 얻은 파형이 올바른지 확인하기 위해 위에서 작성한 vscode 내에서의 시뮬레이션 결과와 비교하였다.



이론에 부합한 결과는 위에서 설명하였고, 그와 vscode의 파형이 일치하였기에 vscode에서의 파형이 올바른 파형이라고 할 수 있다.

이를 vivado파형과 비교해보면, 정확히 같은 파형의 결과를 얻은 것을 알 수 있다.

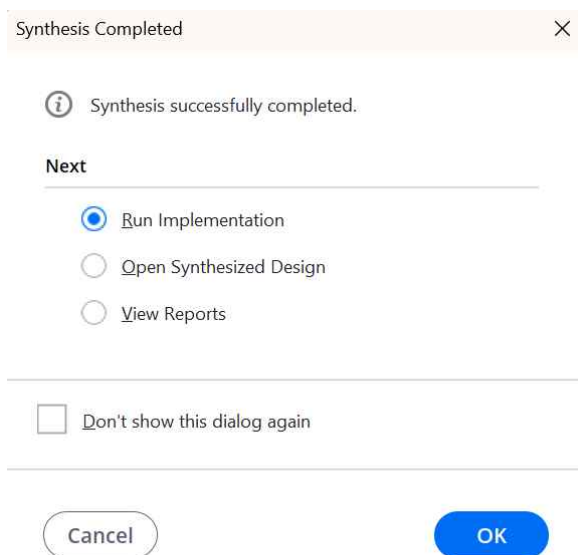
즉, 문제 없이 알맞은 파형을 얻었다.

위의 파형을 얻기 위해 시뮬레이션을 행한 결과 아래와 같은 Tcl Console의 결과값을 얻었다.

```
LAB2_PASS mealy_toggle checks=11
$finish called at time : 67 ns : File "C:/Users/young/Documents/fpga_lab2/circuits/07_mealy/sim/tb_mealy_toggle.sv" Line 30
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_mealy_toggle_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:08 ; elapsed = 00:00:08 . Memory (MB): peak = 1832.605 ; gain = 0.000
```

그 결과, 정상적으로 11개의 경우를 연산하였고, 출력시간 또한 67ns로 기록되었다

이후 Run synthesis와 Run Implementation을 진행하였다.



Implementation Completed

Implementation successfully completed.

Next

☒ Open Implemented Design

☐ Generate Bitstream

☐ View Reports

☐ Don't show this dialog again

CancelOK

Bitstream Generation Completed

Bitstream Generation successfully completed.

Next

☒ Open Implemented Design

☐ View Reports

☐ Open Hardware Manager

☐ Generate Memory Configuration File

☐ Don't show this dialog again

CancelOK

최종적으로 아래와 같은 결과를 오류 없이 얻었다.

Tcl Console Messages Log Reports Design Runs ? - □															
Q × ◇ ⏮ ⏪ ⏩ ⏭ + %															
Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF
✓ synth_1	constrs_1	synth_design Complete!												11	28
✓ impl_1	constrs_1	write_bitstream Complete!	9999%	0.00%	0.122%	0.00%		0.000	0.103	0	8 Warn			11	14

사용한 커밋은 이후 차례 <증빙 및 소스 commit 링크>에 주소로 기록하였다.

- Bit 해시:

SHA-256:

E12AAC2EDD4DCC7307ACFD4F3E92E12CAD00EDEA58
1986ECD72F61FA8335F1AD

- Bit 경로

circuits\07_mealy\lab2_mealy.runs\impl_1\lab2_mealy.bit

All Violations (1)		
Pin Planning (1)		
CFGBVS-1 (1)		
CFGBVS #1	Warning	Neither the CFGBVS nor CONFIG_VOLTAGE voltage property is set in the current design. Configuration bank voltage select (CFGBVS) must be set to VCCO or GND, and CONFIG_VOLTAGE must be set to the correct configuration voltage, in order to determine the I/O voltage support for the pins in bank 0. It is suggested to specify these either using the 'Edit Device Properties' function in the GUI or directly in the XDC file using the following syntax: <pre>set_property CFGBVS value1 [current_design] #where value1 is either VCCO or GND set_property CONFIG_VOLTAGE value2 [current_design] #where value2 is the voltage provided to configuration bank 0</pre> Refer to the device configuration user guide for more information.

- DRC 검사 결과 Error는 0건이었으며, CFGBVS-1 Warning 1건이 발생하였다.

해당 경고는 CFGBVS 및 CONFIG_VOLTAGE 속성이 설정되지 않아 발생한 것으로, 실제 보드 구성 전압과 대조가 필요한 항목이다.

● Timing 결과 검증

Setup	Hold	Pulse Width
Worst Negative Slack (WNS) 999997.750 ns	Worst Hold Slack (WHS) 0.122 ns	Worst Pulse Width Slack (WPWS) 499997.750 ns
Total Negative Slack (TNS) 0.000 ns	Total Hold Slack (THS) 0.000 ns	Total Pulse Width Negative Slack (TPWS) 0.000 ns
Number of Failing Endpoints 0	Number of Failing Endpoints 0	Number of Failing Endpoints 15
Total Number of Endpoints 22	Total Number of Endpoints 22	Total Number of Endpoints 15

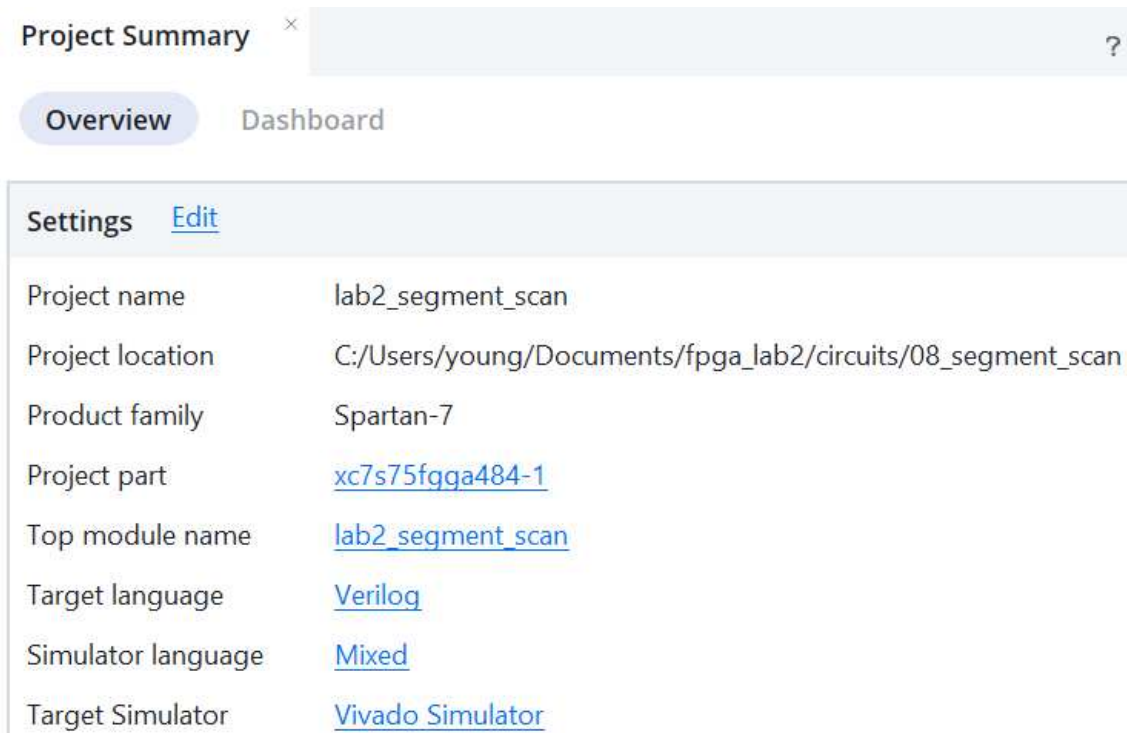
All user specified timing constraints are met.

Vivado Implementation 후 Timing Summary를 확인한 결과, Worst Negative Slack(WNS)은 999997.750 ns, Total Negative Slack(TNS)은 0.000 ns이며 Setup Timing의 Failing Endpoint는 0개였다. 또한 Worst Hold Slack(WHS)은 0.122 ns, Total Hold Slack(THS)은 0.000 ns로 나타났으며 Hold Timing의 Failing Endpoint 역시 0개였다. Pulse Width 항목에서도 Failing Endpoint가 0개로 확인되었다.

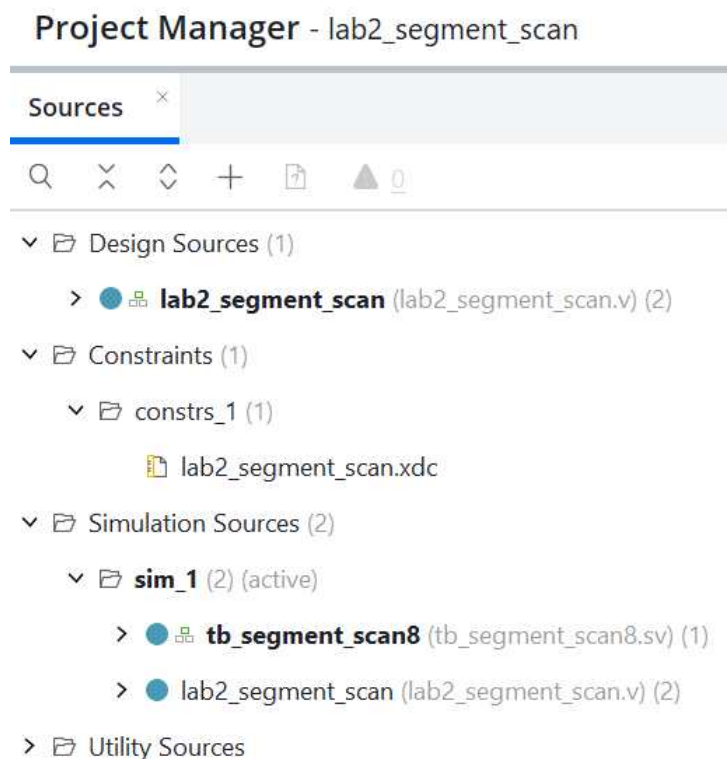
따라서 Setup, Hold 및 Pulse Width Timing에서 타이밍 위반이 발생하지 않았으며, Vivado의 "All user specified timing constraints are met." 메시지를 통해 본 설계가 설정된 타이밍 제약을 모두 만족함을 확인하였다.

8.8. Segment Scan

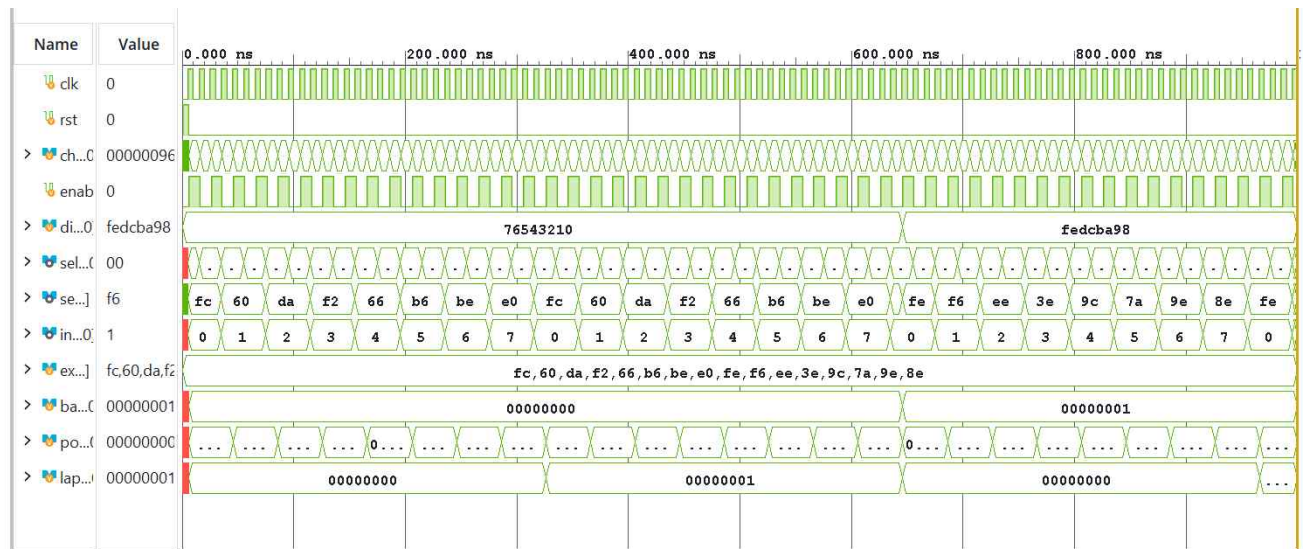
- Vivado 내의 settings는 다음과 같다.



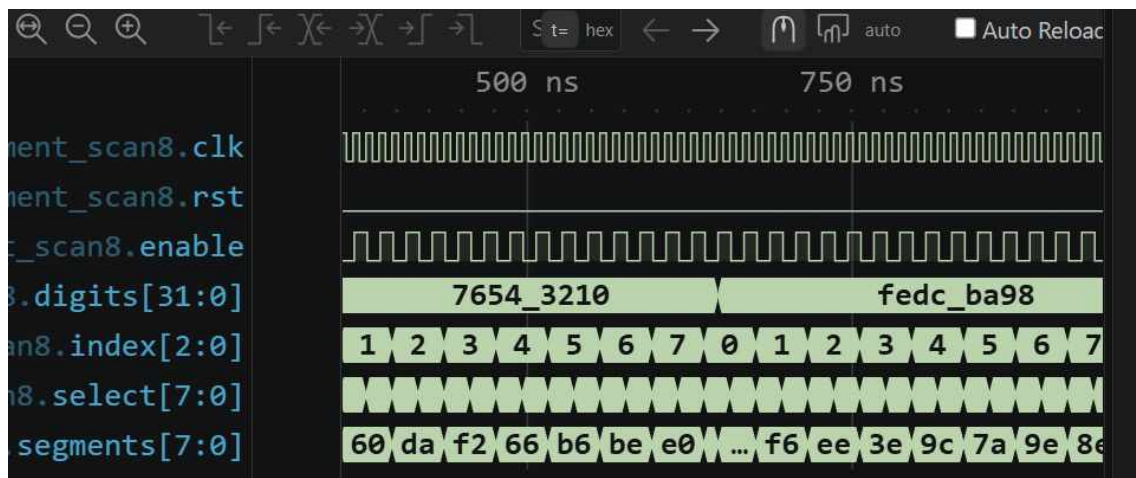
첨부한 사진들과 같이 Top설정 또한 알맞게 진행하였다.



이를 Run simulation 하여 얻은 파형이다.



이렇게 얻은 파형이 올바른지 확인하기 위해 위에서 작성한 vscode 내에서의 시뮬레이션 결과와 비교하였다.



이론에 부합한 결과는 위에서 설명하였고, 그와 vscode의 파형이 일치하였기에 vscode에서의 파형이 올바른 파형이라고 할 수 있다.

이를 vivado파형과 비교해보면, 정확히 같은 파형의 결과를 얻은 것을 알 수 있다.

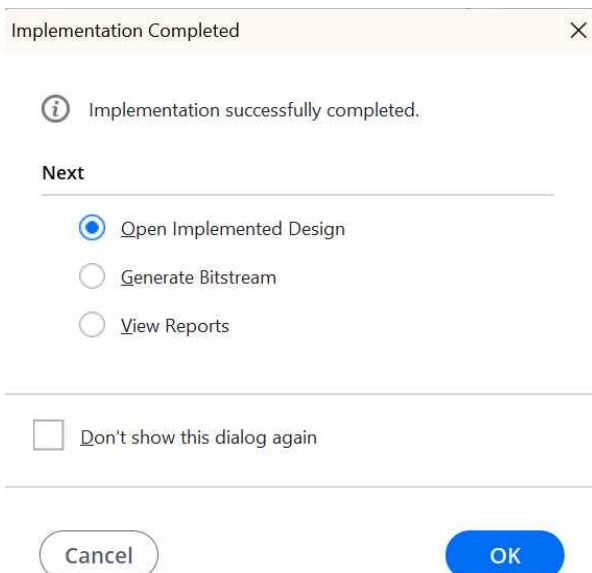
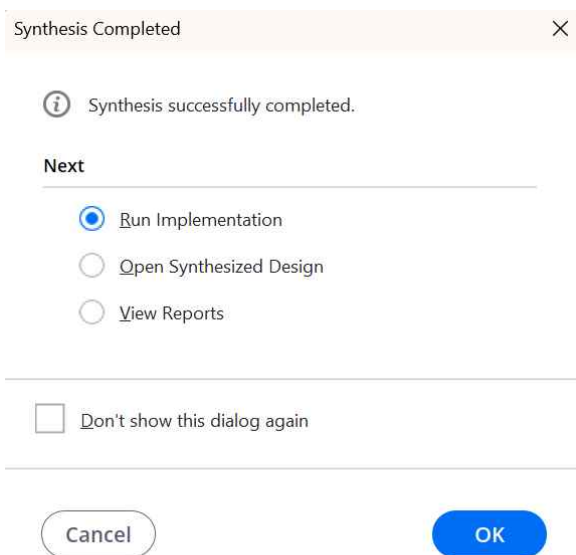
즉, 문제 없이 알맞은 파형을 얻었다.

위의 파형을 얻기 위해 시뮬레이션을 행한 결과 아래와 같은 Tcl Console의 결과값을 얻었다.

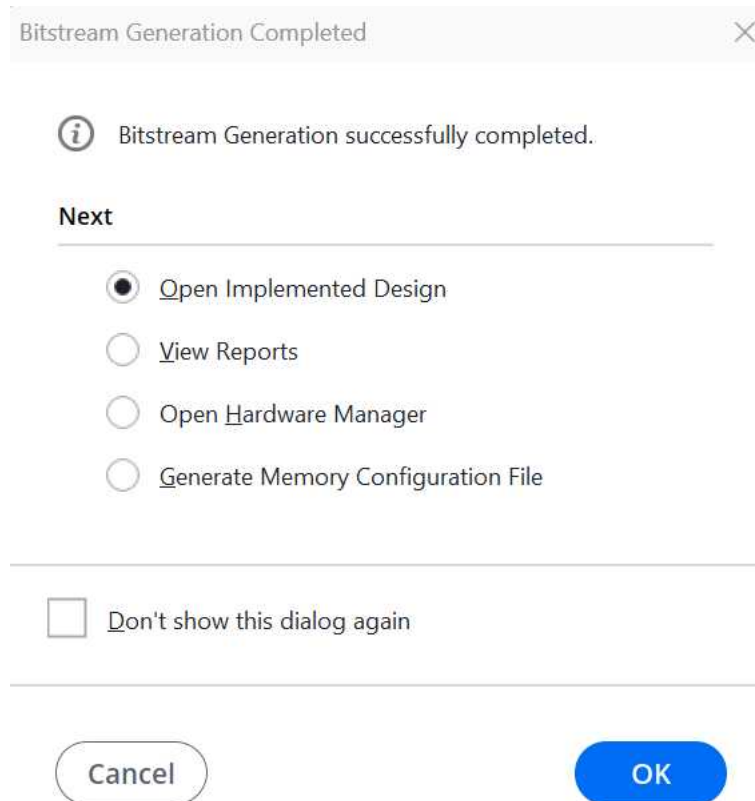
```
run all
LAB2_PASS segment_scan8 checks=194
$finish called at time : 1306 ns : File "C:/Users/young/Documents/fpga_lab2/circuits/08_segment_scan/sim/tb_segment_scan8.sv" Line 30
```

이번에는 정상적으로 194개의 경우를 연산하였고,
출력시간 또한 1306ns로 기록되었다.

이후 Run synthesis와 Run Implementation을
진행하였다.



두 결과 모두 성공하였고, 이후 generate bitstream을 진행하였다.



최종적으로 아래와 같은 결과를 오류 없이 얻었다.

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF
✓ synth_1	constrs_1	synth_design Complete!												12	24
✓ impl_1	constrs_1	write_bitstream Complete!	9999%	0.00%	0.119	0.00%		0.000	0.103	0	24 Warn			14	14

사용한 커밋은 이후 차례 <증빙 및 소스 commit 링크>에 주소로 기록하였다.

- Bit 해시:

SHA-256:

B589174F278CE1CEFC5A7903028CF0C46AD69BADC492
F0B7A98C81E6F760D3B2

- Bit 경로

circuits\08_segment_scan\lab2_segment_scan.run
s\impl_1\lab2_segment_scan.bit

Tcl Console Messages Log Reports Design Runs DRC Methodology Power Timing		
1 Warning Hide All		
Name	Severity	Details
All Violations (1)		
Pin Planning (1)		
CFGBVS-1 (1)		
CFGBVS #1	Warning	Neither the CFGBVS nor CONFIG_VOLTAGE voltage property is set in the current_design. Configuration bank voltage select (CFGBVS) must be set to VCCO or GND, and CONFIG_VOLTAGE must be set to the correct configuration voltage, in order to determine the I/O voltage support for the pins in bank 0. It is suggested to specify these either using the 'Edit Device Properties' function in the GUI or directly in the XDC file using the following syntax: <pre>set_property CFGBVS value1 [current_design] #where value1 is either VCCO or GND set_property CONFIG_VOLTAGE value2 [current_design] #where value2 is the voltage provided to configuration bank 0</pre> Refer to the device configuration user guide for more information.

- DRC 검사 결과 Error는 0건이었으며, CFGBVS-1 Warning 1건이 발생하였다.

해당 경고는 CFGBVS 및 CONFIG_VOLTAGE 속성이
설정되지 않아 발생한 것으로, 실제 보드 구성 전압과
대조가 필요한 항목이다.

● Timing 결과 검증

Implemented Design - xc7s75fgga484-1

Tcl Console Messages Log Reports Design Runs DRC Methodology Power Timing

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS) 999998.062 ns	Worst Hold Slack (WHS) 0.119 ns	Worst Pulse Width Slack (WPWS) 499998.062 ns
Total Negative Slack (TNS) 0.000 ns	Total Hold Slack (THS) 0.000 ns	Total Pulse Width Negative Slack (TPWS) 0.000 ns
Number of Failing Endpoints 0	Number of Failing Endpoints 0	Number of Failing Endpoints 0
Total Number of Endpoints 17	Total Number of Endpoints 17	Total Number of Endpoints 15

All user specified timing constraints are met.

Vivado Implementation 후 Timing Summary를 확인한 결과, Worst Negative Slack(WNS)은 999998.062 ns, Total Negative Slack(TNS)은 0.000 ns이며 Setup Timing의 Failing Endpoint는 0개였다. 또한 Worst Hold Slack(WHS)은 0.119 ns, Total Hold Slack(THS)은 0.000 ns로 나타났으며 Hold Timing의 Failing Endpoint 역시 0개였다. Pulse Width 항목에서도 Failing Endpoint가 0개로 확인되었다.

따라서 Setup, Hold 및 Pulse Width Timing에서 타이밍 위반이 발생하지 않았으며, Vivado의 "All user specified timing constraints are met." 메시지를 통해 본 설계가 설정된 타이밍 제약을 모두 만족함을 확인하였다.

9. 실험 결과와 예상 결과 간의 비교

9.1. Counter

보드 위에 4비트 업, 다운 Counter를 구현한 후, Push Switch를 눌러 카운트 입력을 준 뒤, DIP Switch로 증가·감소 방향을 선택했다. LED 4개에 표시되는 2진수 형태로 현재 카운터 값을 확인했다.

처음 실험을 시작했을 때 LED는 0000을 나타냈다. 증가 모드로 설정한 상태에서 Push Switch를 여러 번 누르니 값이 하나씩 증가했고, 영상에서도 0011(3), 0110(6), 1110(14) 등의 값을 확인할 수 있었다. 이후 계속 입력하자 1111(15)까지 올라간 뒤 다시 0000(0)으로 바뀌었다. 즉, 4-bit로 표현할 수 있는 최대값인 15를 넘으면 다시 0부터 카운트되는 것을 알 수 있었다.

이후 DIP Switch를 조작해 감소 모드로 전환하였다. 0000(0) 상태에서 감소 입력을 한 번 주자 LED가 1111(15)로 바뀌었다. 앞선 경우와 반대의 방향으로 진행되는 것을 알 수 있다.

즉, 증가 모드에서는 15 다음에 0으로 돌아가고 감소 모드에서는 0 다음에 15로 바뀌는 동작이 구현되었다. 이는 0~15 범위에서 반복되는 modulo-16 카운터의 특성과 일치한다고 할 수 있다. 또한 Push Switch 입력에 따라 LED 값이 바뀌었고, DIP Switch를 통해 카운트 방향도 정상적으로 전환되는 것을 확인하였다.

보드 시연 영상:

[fpga_lab2/evidence/01_counter/board/videos/01_counter_demo.mp4 at main · dalkomys777/fpga_lab2](https://www.youtube.com/watch?v=777fpga_lab2)

9.2. Clock divider

FPGA 보드에 Clock Divider 회로를 구현한 뒤, LED를 통해 분주된 클럭 신호가 실제로 어떻게 동작하는지 확인해 보았다. 입력 클럭은 1 kHz를 사용했고, 이를 기준으로 div2, div10, div50, divided, tick 신호가 만들어지도록 설계하였다. 각 출력은 분주비에 따라 약 500 Hz, 100 Hz, 20 Hz, 1 Hz 정도의 주파수를 갖도록 구성했다.

실험 영상에서는 Push Switch를 누른 뒤 LED가 동작하기 시작했다. LED마다 변화하는 속도가 조금씩 달랐는데, 특히 낮은 주파수로 분주된 신호가 연결된 LED는 시간이 지나면서 켜졌다 꺼지는 모습이 확인되었다. 이를 통해 입력 클럭보다 훨씬 느린 주기의 신호가 만들어지고 있다는 것을 알 수 있었다. 즉, 카운터를 이용해 입력 클럭의 속도를 낮추는 Clock Divider 회로가 FPGA 보드에서 정상적으로 동작한 것으로 볼 수 있다.

영상 속에서 500 Hz, 100 Hz, 20 Hz처럼 매우 빠른 신호는 눈으로 점멸을 구분하기 어렵다. 또한, 휴대폰으로 촬영한 영상의 프레임으로도 정확히 담기기 힘들다. 따라서 영상에서는 해당 LED가 계속 켜져 있거나 밝기가 거의 일정한 것처럼 보일 수 있다.

tick 신호는 분주 과정이 끝나는 시점에 잠깐 발생하는 펄스이다. 펄스 간격이 매우 짧아서 영상만으로는 모든 발생 순간을 확인하기 어렵다. 따라서 촬영한 실험 영상은 회로가 전반적으로 제대로 동작하는지 확인하는 용도로 활용할 수 있고, 정확한 출력 주파수나 tick 펄스 간격은 시뮬레이션 결과 혹은 오실로스코프를 이용한 측정을 통해 확인해야 할 것이다.

실험을 통해 하나의 입력 클럭이 여러 분주비에 따라 서로 다른 속도의 출력 신호로 변환되는 것을 확인할 수 있었다. 또한 분주비가 커질수록 LED의 변화 속도도 느려지는 특징을 관찰했다.

결과적으로 설계한 Clock Divider 회로가 실제 FPGA 하드웨어에서도 의도한 대로 동작한다는 점을 확인하였다.

보드 시연 영상:

[fpga_lab2/evidence/02_clock_divider/board/videos/02_clock_divider_demo.mp4_at_main · dalkomys777/fpga_lab2](https://fpga_lab2/evidence/02_clock_divider/board/videos/02_clock_divider_demo.mp4_at_main%20dalkomys777/fpga_lab2)

9.3. Register

FPGA 보드에 4비트 레지스터 회로를 구현한 후, DIP Switch와 Push Switch를 각각 이용해서 데이터가 저장되고 전달되는 과정을 확인했다. sw[7:4]는 입력 데이터로 사용했고, sw[0]과 sw[1]은 각각 Load와 Transfer 동작을 제어하는 신호로 설정했다. 그렇게 출력된 결과는 LED로 확인했고, 이후, 오른쪽 4개 LED에는 저장된 값이, 왼쪽 4개 LED에는 현재 값이 표시되도록 회로를 구성했다.

실험을 시작하기 전, Reset 신호를 먼저 인가했다. 이때 두 레지스터의 출력이 모두 0000으로 표시되어 초기화가 정상적으로 이루어진 것을 확인할 수 있었다. 이후 DIP Switch를 조작해 입력 데이터와 제어 신호를 바꾸고, Push Switch를 눌러 동작 입력을 주었다. 이 과정을 통해 입력된 데이터가 레지스터에 저장되고, 설정한 제어 신호에 따라 다른 레지스터로 전달되는 특성을 관찰하였다.

입력 및 동작	LED 출력 (stored, value)	결과
Reset	00	두 레지스터 모두 초기화
A1 입력 후 실행	A0	A가 stored에 저장됨
32 입력 후 실행	AA	기존 stored=A가 value로 전달됨
33 입력 후 실행	3A	새로운 값 3은 stored에 저장되고, 이전 값 A가 value로 전달됨
33 다시 실행	33	기존 stored=30이 value로 전달됨
F0 입력	33 유지	Load와 Transfer가 비활성화되어 값 유지
Reset	00	두 레지스터 모두 다시 초기화

가장 눈에 띄었던 부분은 입력을 처음 33으로 설정했을 때 LED가 바로 33이 아니라 3A로 표시된 것이다. Load와 Transfer 신호가 같은 클록에서 동시에 활성화되면 stored 레지스터에는 새 입력값인 3이 저장되지만, value 레지스터는 갱신되기 직전 stored에 들어 있던 값인 A를 전달받게 된다. 이러한 과정을 거쳐서 오른쪽 4비트에는 3이 나타나고 왼쪽 4비트에는 A가 표시되어 최종 출력이 3A가 된다.

이 결과는 두 레지스터가 같은 클록 엣지에서 동작하는 순차회로의 특성으로 이해할 수 있다. value는 해당 클록 엣지가 들어오기 전에 stored에 저장되어 있던 값에 맞추어 바뀌기 때문에, 같은 순간에 새로 저장된 데이터는 바로 전달되지 않고 다음 동작에서 반영된다. 영상에서

확인할 수 있듯이 실제로 같은 33 입력을 한 번 더
인가했을 때는 stored에 이미 3이 들어 있는 상태였으므로
LED 출력도 33으로 변경되는 것을 확인할 수 있었다.
외에도 입력 데이터를 F로 변경한 상태에서 Load와
Transfer를 모두 비활성화했을 때는 LED에 기존 값인
33이 그대로 표시되었다. 이를 통해 제어 신호가 들어오지
않으면 레지스터가 이전 값을 계속 유지한다는 점도
확인했다.

이 과정을 통해서 Reset에 따른 초기화, Load 신호를
통한 데이터 저장, Transfer 신호에 따른 레지스터 간
데이터 전달, 또 제어 신호가 없을 때의 값 유지까지 모두
설계한 내용과 일치했다. Load와 Transfer를 동시에
활성화했을 때 새 데이터는 stored에 저장되고, value에는
이전 stored 값이 전달되는 동작이 FPGA 보드에서도
정상적으로 구현된 것을 확인할 수 있었다.

보드 시연 영상:

fpga_lab2/evidence/03_register/board/videos/03_register_demo.mp4 at [main](#) · [dalkomys777/fpga_lab2](#)

9.4. Shift register

FPGA 보드에 4-bit Shift Register를 구현한 뒤 DIP Switch와 Push Switch를 이용해 직렬 데이터가 이동하는 과정을 확인해 보았다. Push Switch를 한 번 누를 때마다 레지스터에 저장된 값이 오른쪽으로 1비트씩 이동하도록 만들었고, DIP Switch로 설정한 serial_in 값은 최상위 비트(MSB)에 새로 들어가도록 설계했다. 현재 레지스터 상태는 LED 4개를 통해 확인하였다.

먼저 Reset을 인가한 뒤 LED가 모두 꺼진 0000 상태가 되는 것을 확인했다. 이후 serial_in 값을 설정하고 버튼을 여러 번 누르자 LED의 점등 위치와 패턴이 순서대로 바뀌었다. 이를 통해 레지스터에 저장된 비트가 버튼 입력에 맞춰 한 자리씩 오른쪽으로 이동하는 것을 볼 수 있었다.

serial_in을 1로 설정한 뒤 Shift 동작을 계속하면, MSB에 1이 새로 입력되고 기존 비트는 오른쪽으로 밀려난다. 예상되는 값의 변화는 다음과 같다.

0000 → 1000 → 1100 → 1110 → 1111

반대로 serial_in을 0으로 바꾸고 Shift 동작을 계속하면 MSB에는 0이 들어간다. 기존에 저장되어 있던 1들은 오른쪽으로 한 칸씩 이동하다가 순서대로 사라진다.

1111 → 0111 → 0011 → 0001 → 0000

실험 영상에서도 버튼을 누를 때마다 LED 패턴이 순차적으로 이동하는 모습을 확인할 수 있었다. 또한 버튼을 누르지 않는 동안에는 LED 출력이 바뀌지 않았는데, 이로부터 Shift 입력이 없을 때 레지스터가 이전 상태를 그대로 유지한다는 점도 알 수 있었다.

결과적으로 Push Switch 입력마다 4-bit 데이터가 오른쪽으로 이동하고, DIP Switch의 직렬 입력값이 MSB에 삽입되는 Shift Register 동작이 정상적으로 구현되었다. Reset에 따른 초기화와 입력이 없을 때의 상태 유지 역시 의도한 대로 동작했다.

보드 시연 영상:

[fpga_lab2/evidence/04_shift_register/board/videos/04_shift_register_demo.mp4 at main · dalkomys777/fpga_lab2](https://www.youtube.com/watch?v=04_shift_register_demo.mp4)

9.5. PISO

FPGA 보드에 4-bit PISO(Parallel-In Serial-Out) Shift Register를 구현한 뒤, DIP Switch와 Push Switch를 이용해 병렬 데이터 저장과 직렬 출력 동작을 확인해 보았다.

먼저 DIP Switch에서 병렬 입력값을 $1010_2(A)$ 로 설정한 후 Load 동작을 실행했다. 입력한 1010 값이 레지스터에 저장되었고, LED에도 저장된 값인 1010이 표시되는 것을 확인할 수 있었다.

이후 Shift 모드로 바꾼 뒤 Push Switch를 여러 번 눌렀다. 버튼을 누를 때마다 레지스터에 들어 있던 데이터는 왼쪽으로 1비트씩 이동했고, 오른쪽에 비는 자리에는 0이 들어가도록 동작했다. 영상에서는 레지스터 값이 다음과 같이 변하는 모습을 확인했다.

$1010 \rightarrow 0100 \rightarrow 1000 \rightarrow 0000$

각 상태에서 회로는 최상위 비트(MSB)를 직렬 출력으로 내보냈다. 따라서 처음 저장한 병렬 데이터 1010의 비트는 $1 \rightarrow 0 \rightarrow 1 \rightarrow 0$ 순서로 출력되었다.

이를 통해 PISO Shift Register가 병렬로 저장한 데이터를 MSB부터 한 비트씩 직렬로 전송하는 방식으로 정상 동작한다는 것을 확인하였다.

단계	레지스터 값	Serial Out	의미
Load	1010 (A)	1	병렬 데이터 저장 및 MSB 출력
Shift 1	0100 (4)	0	한 비트 Shift
Shift 2	1000 (8)	1	다음 데이터 출력
Shift 3	0000 (0)	0	마지막 데이터 출력
이후 Shift	0000	0	모든 데이터 출력 완료

영상에서는 Load 직후 LED가 A, 즉 1010 패턴을 표시했다. 이후 버튼을 누를 때마다 LED 값이 4, 8, 0 순서로 바뀌는 모습을 확인할 수 있었다. 별도로 연결한 직렬 출력 LED도 각 Shift 단계마다 1 → 0 → 1 → 0 순서로 상태가 변했다. 이를 통해 병렬로 저장된 데이터가 한 비트씩 직렬 데이터로 바뀌어 출력되는 과정을 확인했다.

실제 FPGA 보드에서도 4-bit 병렬 데이터가 레지스터에 정상적으로 저장되었고, Shift 입력이 들어올 때마다 데이터가 한 비트씩 이동하면서 MSB부터 직렬로 출력되었다. 모든 비트가 출력된 뒤에는 레지스터가 0000 상태를 유지했다. 따라서 빈 비트 자리에 0을 채우는 zero-fill 방식의 PISO Shift 동작 역시 설계한 대로 구현된 것을 확인할 수 있었다.

보드 시연 영상:

fpga_lab2/evidence/05_piso/board/videos/05_piso_demo.mp4 at [main](#) · [dalkomys777/fpga_lab2](#)

9.6. Moore

FPGA 보드에 Moore FSM을 구현한 뒤, DIP Switch와 Push Switch를 이용해 상태가 어떻게 바뀌는지 확인해보았다. 먼저 Reset 신호를 인가했을 때 LED는 00을 표시했고, FSM도 초기 상태인 S0으로 돌아가는 것을 확인할 수 있었다.

advance 입력을 활성화한 상태에서 Push Switch를 누르자 LED 출력이 00에서 01로 바뀌었다. 즉, FSM이 S0에서 S1으로 이동한 것이다. 같은 조건에서 버튼을 한 번 더 누르니 출력은 10으로 바뀌었고, FSM은 S2 상태에 도달했다. 이후 다시 버튼을 누르자 LED가 00으로 변하면서 FSM이 초기 상태인 S0으로 돌아왔다. 상태 전이 순서는 다음과 같았다.

$S0(00) \rightarrow S1(01) \rightarrow S2(10) \rightarrow S0(00)$

반대로 advance를 0으로 설정했을 때는 버튼을 눌러도 LED 출력이 바뀌지 않았다. 회로가 현재 상태를 그대로 유지했기 때문에, advance가 1인 상태에서 Push Switch 입력이 들어올 때만 다음 상태로 이동한다는 점을 확인할 수 있었다.

이 동작은 Moore FSM의 특징과도 일치한다. Moore FSM은 입력값이 바뀌었다고 바로 출력이 변하는 방식이 아니라, 현재 FSM이 어떤 상태에 있는지에 따라 출력이

결정된다. 실제 실험에서도 DIP Switch만 조작했을 때는 LED 값이 유지되었고, 버튼 입력으로 유효한 상태 전이가 발생한 뒤에만 해당 상태에 맞는 출력이 나타났다.

결과적으로 실제 FPGA 보드에서 Reset에 따른 초기화, advance=1일 때의 상태 전이, advance=0일 때의 상태 유지, 그리고 00 → 01 → 10 → 00으로 반복되는 동작을 모두 확인했다. 이를 통해 구현한 Moore FSM이 의도한 대로 정상 동작한다는 것을 검증하였다.

보드 시연 영상:

fpga_lab2/evidence/06_moore/board/videos/06_moore_demo.mp4 at [main](#) · [dalkomys777/fpga_lab2](#)

9.7. Mealy

보드 위에 밀리 회로를 구현한 뒤, Push Switch와 DIP Switch를 사용해 상태 전이와 출력 변화를 확인해 보았다. 먼저 Reset 신호를 인가했을 때 상태와 출력을 나타내는 LED가 모두 꺼졌다. 이를 통해 회로가 초기 상태로 정상적으로 초기화된 것을 확인할 수 있었다.

Push Switch로 클럭 입력을 주자 상태 표시 LED가 순서대로 바뀌었다. 영상에서는 초기 상태에서 다음 상태로 넘어갈 때 LED7이 켜졌고, 그다음에는 LED6이 켜지는 모습이 확인되었다. 즉, 버튼 입력에 따라 FSM 내부 상태가 정상적으로 전이하고 있다는 것을 알 수 있었다.

특히 LED6이 켜진 상태에서 DIP Switch 입력을 바꿨을 때 Mealy FSM의 특징이 잘 나타났다. 상태를 표시하는 LED6은 계속 켜져 있었지만, 출력 LED8은 입력값에 따라 즉시 바뀌었다.

이 과정에서는 내부 상태가 바뀌지 않았는데도 DIP Switch 입력만 조작하면 LED8의 출력이 달라졌다. 밀리 회로는 현재 상태 외에도 현재 입력값도 반영해서 출력을 내놓기 때문에, 같은 상태를 유지하는 중에도 입력이 달라진다면, 이에 따라 다른 출력이 나올 수 있다.

밀리 회로 구현을 통해 Reset에 따른 초기화와 Push Switch 입력에 따른 상태 전이가 정상적으로 이루어지는 것을 확인했다. 또한 상태가 유지되는 동안에도 DIP Switch 입력에 따라 출력이 바로 변하는 Mealy FSM의 특성 역시 FPGA 보드에서 확인할 수 있었다.

보드 시연 영상:

[fpga_lab2/evidence/07_mealy/board/videos/07_mealy_demo.mp4](https://github.com/dalkomys777/fpga_lab2/tree/main/evidence/07_mealy/board/videos/07_mealy_demo.mp4) at main · dalkomys777/fpga_lab2

9.8. Segment scan

FPGA 보드에 8자리 7-Segment Scan 회로를 구현한 뒤, DIP Switch를 조작하면서 표시 데이터와 다중화 동작을 확인해 보았다. 실험을 시작했을 때 첫 번째 자리에는 0이 표시됐고, 나머지 자리에는 1부터 7까지 숫자가 순서대로 나타났다. 따라서 전체 7-Segment에서는 01234567 형태가 표시되는 것을 확인할 수 있었다.

DIP Switch 값을 바꾸자 첫 번째 7-Segment 숫자만 변경됐으며, 나머지 일곱 자리는 기존 숫자를 그대로 유지했다. 영상에서도 DIP Switch를 조작할 때 첫 번째 자리가 5, 6, 7, 8 등으로 바뀌는 모습이 확인되었다. 이를 통해 입력한 4-bit 값이 첫 번째 표시 자리의 데이터로 정상 처리된다는 점을 알 수 있었다. 나머지 자리는 고정된 데이터를 계속 표시했기 때문에, 회로가 각 자리의 데이터를 구분해서 처리하고 있다는 점도 확인했다.

이 회로는 8개의 7-Segment를 동시에 계속 켜는 방식이 아니라, 각 자리를 아주 짧은 간격으로 차례대로 선택하는 Scan 방식을 사용한다. 내부적으로는 한 순간에 하나의 자리만 활성화되지만, 스캔 속도가 충분히 빠르기 때문에 사람 눈에는 8개 숫자가 동시에 켜져 있는 것처럼 보인다. 실제 보드에서도 모든 자리가 안정적으로 표시되어 다중화 구동이 정상적으로 이루어지고 있음을 확인할 수 있었다. 영상 일부 장면에서는 특정 숫자가 조금 어둡게 보이거나

일부 Segment가 약하게 표시되기도 했다. 이는 빠른 스캔 주기와 카메라 촬영 시점이 맞지 않아 생기는 현상으로 볼 수 있다. 실제 보드에서는 각 자리가 반복적으로 정상 표시되는 것을 확인했다.

결과적으로 DIP Switch 입력값은 첫 번째 7-Segment에 반영됐고, 나머지 자리에는 고정된 1~7 데이터가 유지되었다. 또한 8개 자리가 Scan 방식으로 순차 구동되는 것도 확인했으며, 이를 통해 설계한 데이터 변환 및 자리 선택 다중화 회로가 의도한 대로 동작함을 검증하였다.

보드 시연 영상:

[fpga_lab2/evidence/08_segment_scan/board/videos/08_segment_scan_demo.mp4](https://www.youtube.com/watch?v=...) at
main · dalkomys777/fpga_lab2

<LAB 01 미완료 실험 추가 진행 결과>

<9.9. Decoder 3x8, 9.10. SEVEN SEGMENT DECODER>

(Decoder 3x8)

FPGA 보드에 3-to-8 Decoder 회로를 구현한 뒤, DIP Switch로 3-bit 입력값을 차례대로 바꾸면서 LED 출력을 확인해 보았다. 회로는 입력 abc 값에 따라 8개 출력 중 하나만 1이 되도록 구성했고, 각 출력은 LED1부터 LED8까지 연결하였다.

실험 영상에서는 DIP Switch 입력을 변경할 때마다 하나의 LED만 켜지는 것을 확인할 수 있었다. 입력값을 증가시키자 켜지는 LED의 위치도 순서대로 이동했다. 입력이 000일 때는 LED8이 켜졌고, 이후에는 LED7, LED6 순으로 점등 위치가 이동했다. 마지막으로 입력이 111이 되었을 때는 LED1이 켜졌다.

즉, 입력값에 따라 하나의 출력만 선택되는 one-hot 방식의 디코더 동작이 FPGA 보드에서 정상적으로 구현된 것을 확인하였다.

3-bit 입력 abc	10진수	점등 LED	출력 특성
000	0	LED8	하나의 출력만 활성화
001	1	LED7	one-hot 출력
010	2	LED6	one-hot 출력
011	3	LED5	one-hot 출력
100	4	LED4	one-hot 출력
101	5	LED3	one-hot 출력
110	6	LED2	one-hot 출력
111	7	LED1	하나의 출력만 활성화

영상에서 입력 조합을 바꿀 때마다 기존에 켜져 있던 LED는 꺼지고, 새로운 위치의 LED 하나만 켜지는 모습을 확인했다. 여러 LED가 동시에 켜지지 않았기 때문에, 입력값에 해당하는 하나의 출력만 선택되는 Decoder의 특성이 정상적으로 나타났다고 볼 수 있다.

입력값을 000에서 111까지 순서대로 증가시키자 점등 위치는 LED8에서 LED1 방향으로 이동했다. 이는 설계한 출력 비트의 순서와 FPGA 보드 LED의 연결 순서가 서로 맞게 구현되었다는 의미이다.

실제 보드 실험을 통해 3-bit 입력의 8가지 조합마다 서로 다른 하나의 출력만 활성화되는 3-to-8 Decoder 동작을 확인하였다. 결과적으로 RTL로 구현한 조합논리 Decoder 회로가 FPGA 하드웨어에서도 설계한 대로 정상 동작함을 검증하였다.

보드 시연 영상:

fpga_lab1/evidence/09/board/videos/09_decoder3to8_demo.mp4 at [main](https://github.com/dalkomys777/fpga_lab1) · [dalkomys777/fpga_lab1](https://github.com/dalkomys777/fpga_lab1)

(SEVEN SEGMENT DECODER)

FPGA 보드에 4-bit 입력을 7-Segment 표시 패턴으로 변환하는 Decoder 회로를 구현한 뒤, DIP Switch로 입력값을 바꾸면서 출력 상태를 확인해 보았다.

회로는 4-bit 입력값에 맞춰 필요한 Segment를 켜고, 0~9 숫자와 A~F의 16진수 문자를 표시하도록 구성하였다.

실험 영상에서는 DIP Switch 상태를 변경하자 7-Segment의 점등 패턴도 바로 바뀌는 모습을 확인할 수 있었다. 처음에는 0에 해당하는 형태가 나타났고, 입력값을 변경하자 오른쪽 두 Segment가 켜진 1의 형태가 표시되었다. 이후에는 7개 Segment가 모두 켜지는 8의 형태도 확인되었다. 영상 후반에는 A와 F에 해당하는 문자 역시 정상적으로 표시되었다.

특히 8을 표시할 때 모든 Segment가 켜졌기 때문에, 7개 Segment가 각각 정상적으로 동작한다는 점을 확인할 수 있었다. 또한 10 이상의 입력값에서는 숫자 대신 A~F 형태의 16진수 문자가 표시되었다. 이를 통해 회로가 0000~1111 전체 입력 범위를 처리하는 Hex Decoder로 정상 동작하고 있음을 알 수 있었다.

영상에서 확인한 결과는 아래와 같다.

4-bit 입력에 해당하는 값	7-Segment 표시	관찰 내용
0	0	가운데 Segment를 제외한 숫자 0 형태
1	1	우측 두 Segment 중심으로 점등
8	8	모든 주요 Segment 점등
A	A	16진수 A 형태 정상 표시
F	F	16진수 F 형태 정상 표시

DIP Switch를 조작할 때는 각 스위치가 완전히 동시에 바뀌는 것이 아니기 때문에, 전환 과정에서 잠깐 중간 입력값에 해당하는 문자가 표시될 수 있다. 하지만 스위치 조작이 끝난 뒤에는 7-Segment가 설정한 4-bit 입력값에 맞는 패턴을 안정적으로 나타냈다. 이를 통해 이 회로는 이전 값을 저장하는 방식이 아니라, 현재 입력값을 바로 7-Segment 출력으로 변환하는 조합논리 회로라는 점을 확인할 수 있었다.

실제 FPGA 보드에서도 입력값을 바꾸자 그에 맞는 7-Segment 패턴이 정상적으로 표시되었다. 0~9 숫자뿐 아니라 A~F의 16진수 문자도 확인할 수 있었다. 따라서 구현한 4-bit to 7-Segment Decoder가 설계한 대로 정상 동작함을 검증하였다.

보드 시연 영상:

fpga_lab1/evidence/10/board/videos/10_seven_segment_demo.mp4 at [main](#) · dalkomys777/fpga_lab1

10. 증빙 및 source commit 링크(Github)

10.1. Counter

VS Code simulation log:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/01_counter/vscode/simulation.txt

VS Code waveform VCD:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/01_counter/vscode/wave.vcd

VS Code waveform image:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/01_counter/vscode/wave.png

VS Code waveform xdc:

https://github.com/dalkomys777/fpga_lab2/blob/main/circuits/01_counter/constraints/lab2_counter.xdc

10.2. Clock divider

VS Code simulation log:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/02_clock_divider/vscode/simulation.txt

VS Code waveform VCD:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/02_clock_divider/vscode/wave.vcd

VS Code waveform image:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/02_clock_divider/vscode/wave.png

VS Code waveform xdc:

https://github.com/dalkomys777/fpga_lab2/blob/main/circuits/02_clock_divider/constraints/lab2_clock_divider.xdc

10.3. register

VS Code simulation log:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/03_register/vscode/simulation.txt

VS Code waveform VCD:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/03_register/vscode/wave.vcd

VS Code waveform image:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/03_register/vscode/wave.png

VS Code waveform xdc:

https://github.com/dalkomys777/fpga_lab2/blob/main/circuits/03_register/constraints/lab2_register.xdc

10.4. Shift register

VS Code simulation log:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/04_shift_register/vscode/simulation.txt

VS Code waveform VCD:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/04_shift_register/vscode/wave.vcd

VS Code waveform image:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/04_shift_register/vscode/wave.png

VS Code waveform xdc:

https://github.com/dalkomys777/fpga_lab2/blob/main/circuits/04_shift_register/constraints/lab2_shift_register.xdc

10.5. PISO

VS Code simulation log:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/05_piso/vscode/simulation.txt

VS Code waveform VCD:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/05_piso/vscode/wave.vcd

VS Code waveform image:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/05_piso/vscode/wave.png

VS Code waveform xdc:

https://github.com/dalkomys777/fpga_lab2/blob/main/circuits/05_piso/constraints/lab2_piso.xdc

10.6. Moore

VS Code simulation log:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/06_moore/vscode/simulation.txt

VS Code waveform VCD:

[fpga_lab2/evidence/06_moore/vscode/wave.vcd at main · dalkomys777/fpga_lab2](#)

VS Code waveform image:

[fpga_lab2/evidence/06_moore/vscode/wave.png at main · dalkomys777/fpga_lab2](#)

VS Code waveform xdc:

[fpga_lab2/circuits/06_moore/constraints/lab2_moore.xdc at main · dalkomys777/fpga_lab2](#)

10.7. Mealy

VS Code simulation log:

[fpga_lab2/evidence/07_mealy/vscode/simulation.txt](#) at main · dalkomys777/fpga_lab2

VS Code waveform VCD:

[fpga_lab2/evidence/07_mealy/vscode/wave.vcd](#) at main · dalkomys777/fpga_lab2

VS Code waveform image:

[fpga_lab2/evidence/07_mealy/vscode/wave.png](#) at main · dalkomys777/fpga_lab2

VS Code waveform xdc:

[fpga_lab2/circuits/07_mealy/constraints/lab2_mealy.xdc](#) at main · dalkomys777/fpga_lab2

10.8. Segment scan

VS Code simulation log:

[fpga_lab2/evidence/08_segment_scan/vscode/simulation.txt](#) at main · [dalkomys777/fpga_lab2](#)

VS Code waveform VCD:

[fpga_lab2/evidence/08_segment_scan/vscode/wave.vcd](#) at main · [dalkomys777/fpga_lab2](#)

VS Code waveform image:

[fpga_lab2/evidence/08_segment_scan/vscode/wave.png](#) at main · [dalkomys777/fpga_lab2](#)

VS Code waveform xdc:

[fpga_lab2/circuits/08_segment_scan/constraints/lab2_segment_scan.xdc](#) at main · [dalkomys777/fpga_lab2](#)

전체 소스 커밋:

[Complete LAB2 circuits and evidence · dalkomys777/fpga_lab2@11c334e](#)

LAB_02 github 주소:

[dalkomys777/fpga_lab2](#)

LAB_02 제출 태그:

fpga_lab2-reports-post